

Single Source Shortest Path

Data Structures and Algorithms

Housekeeping:

- **No** recitation **this** week
- Midterms **this** week. Thursday.
 - Check your seat and venue!
 - Midterm announcement on Coursemology has details.
 - Past years released.



Other Details:

- Eldon away for ICPC APAC Championships, will be back Wednesday.

Comments from Nodle:

Hey guys! Sorry if the slides lack colour. Doing this without many hours of time between the competition events.

(at 12:24AM 09-Mar)



Game Plan for Today:

- New problem: Shortest pathfinding for directed, weighted/unweighted graphs.
- On Wednesday, a short 1 hour review + problem solving for midterm.

BFS for SSSP

Finding short paths from a source
node s .

The starting point for 3.5
algorithms today.

Problem statement:

Input: Given a **directed**, **unweighted** graph $G = (V, E)$ in the form of an adjacency list, with **n** nodes and **m** edges, and a source node **s**.

Output: An array **dist**[] such that for all nodes **v**, **dist**[**v**] is the shortest distance from node **s** to node **v**.

Unweighted SSSP:

Idea: starting from node **s**, we explore the nodes based on nodes with distance 1, then distance 2, then distance 3,... and so on.

(That's how BFS does its traversal)

Unweighted SSSP:

Maintain a distance array `dist[]`. Set `dist[s] = 0`.

Enqueue `s` into the `queue`. Mark `s` as visited.

While the `queue` is not empty:

- Pop `current_node` from `queue`

- For all neighbours `n` of `current_node`:

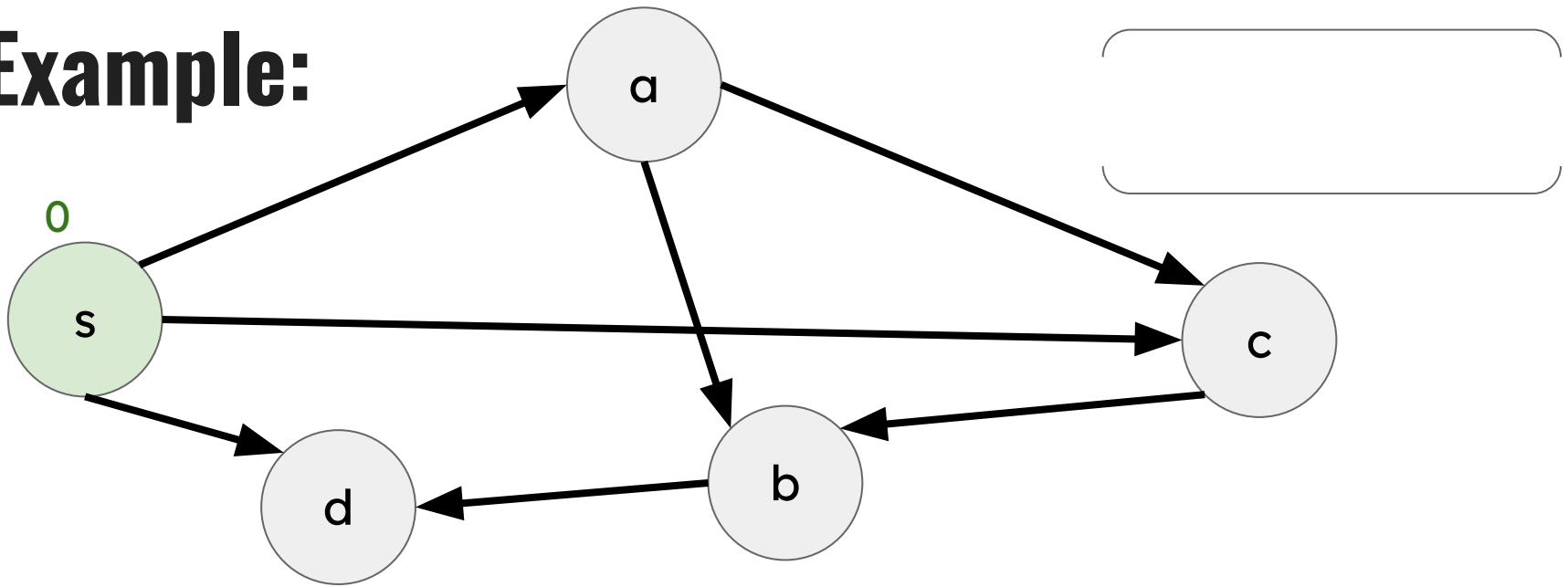
 - If `n` is `visited` before, `continue`;

 - `dist[n] = dist[current_node] + 1`

 - Mark `n` as visited.

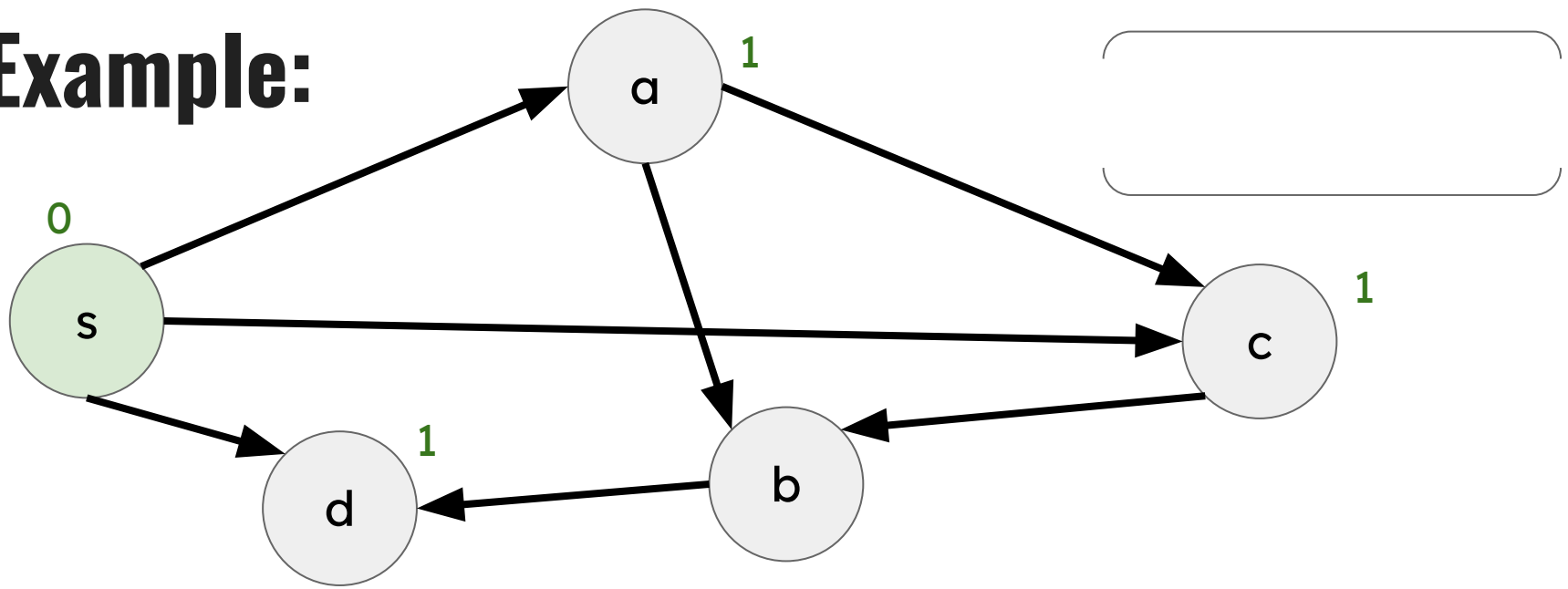
 - Enqueue into `queue`

Example:



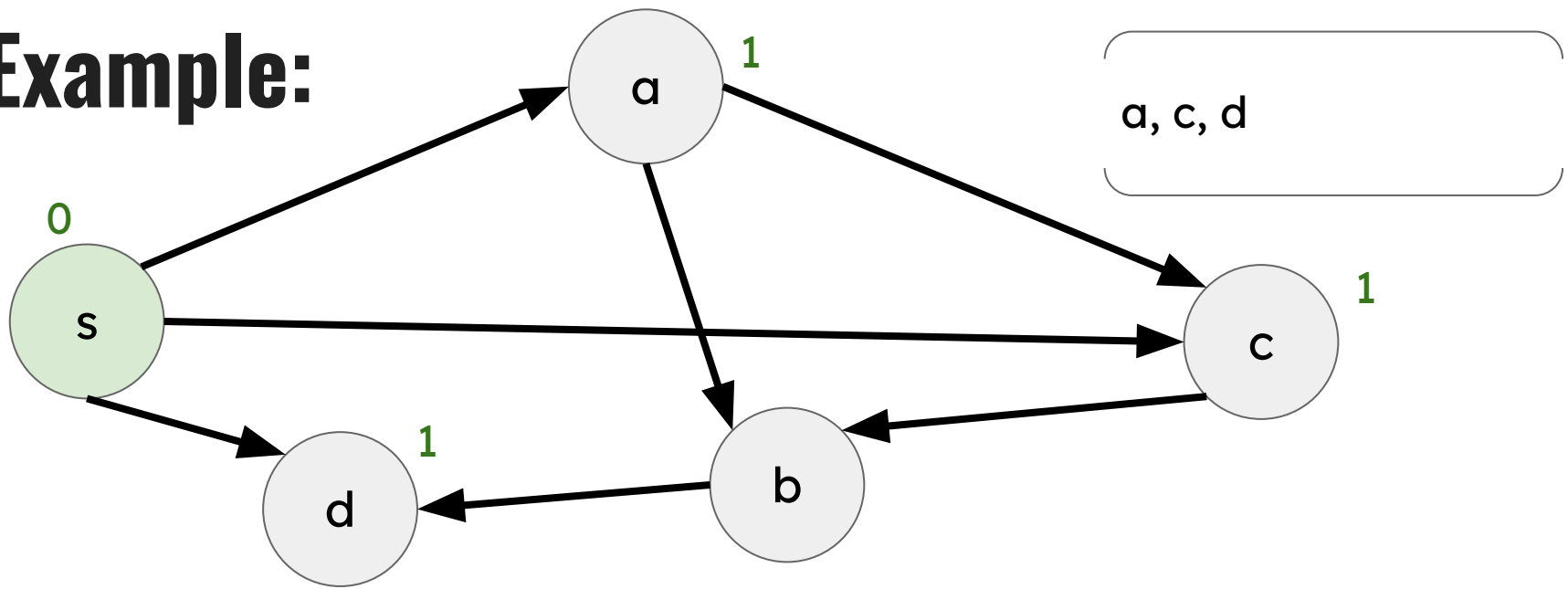
Start from node s, mark it as having distance 0.

Example:



Go through its neighbours, in this case **a**, **c**, and **d**. Neither have been visited. Set their distance to $0 + 1$.

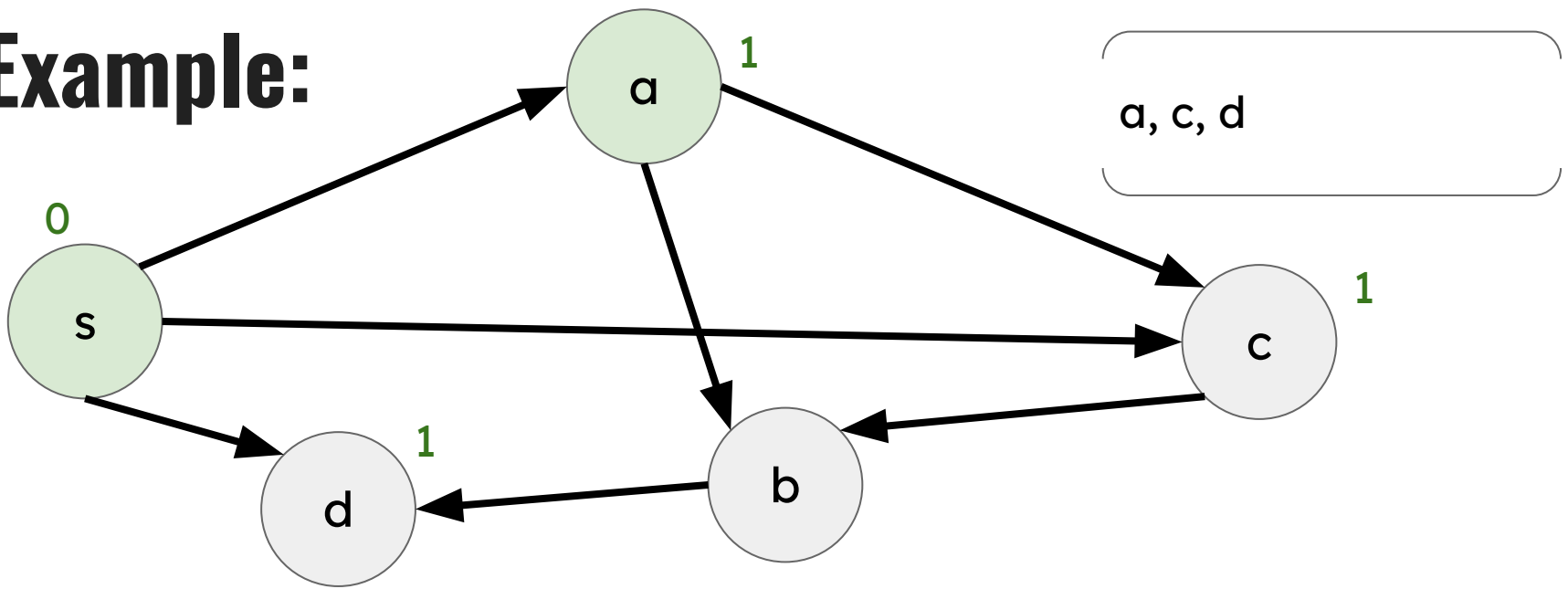
Example:



Go through its neighbours, in this case **a**, **c**, and **d**. Neither have been visited. Set their distance to $0 + 1$.

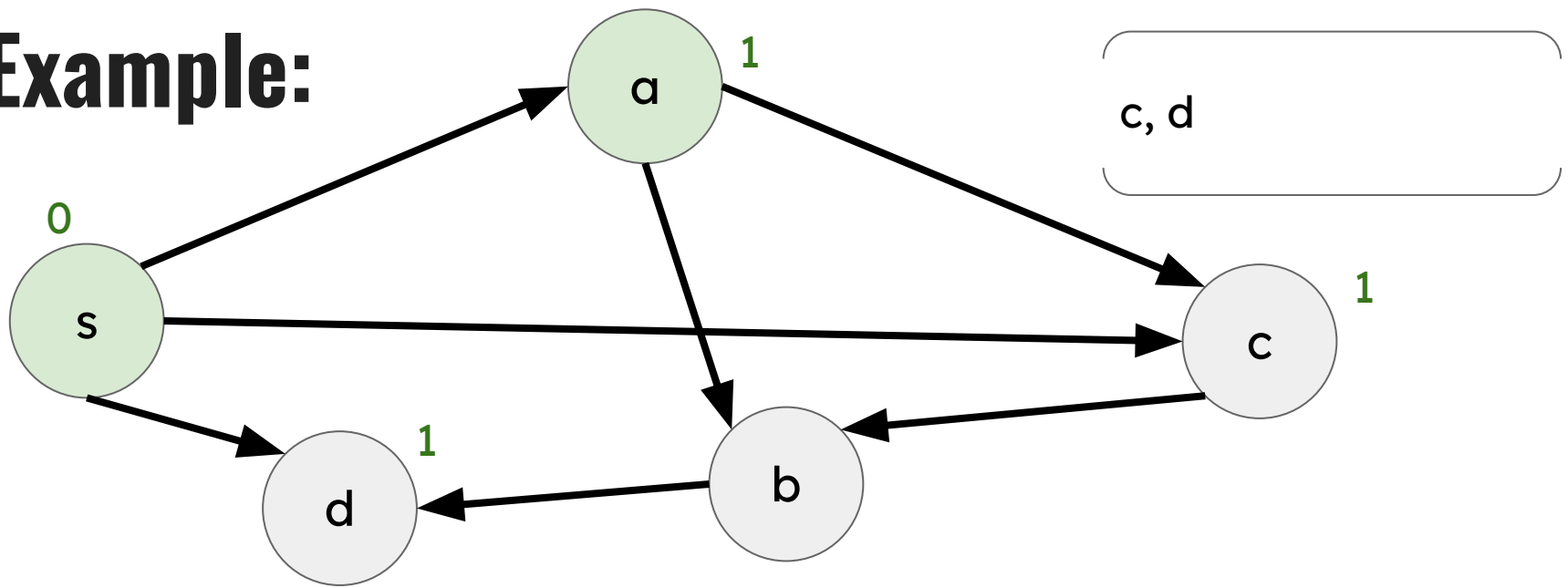
Then enqueue them.

Example:



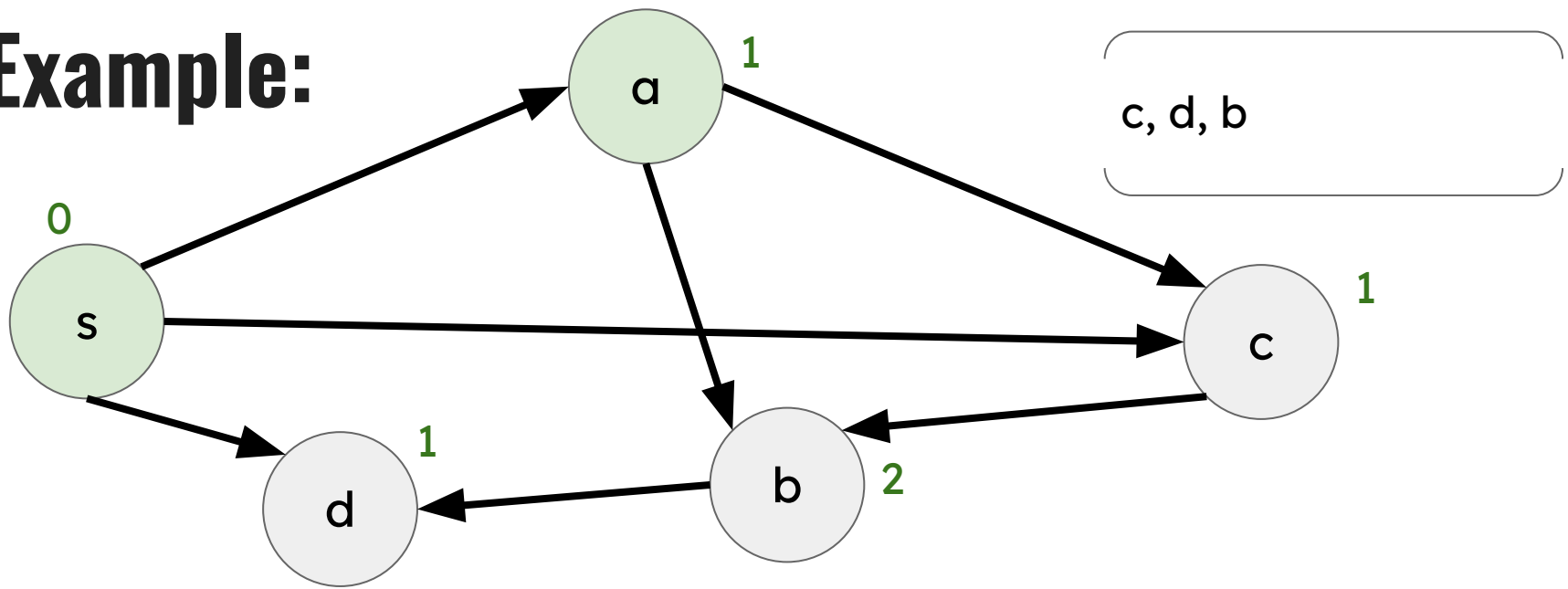
Next, we dequeue and get node a.

Example:



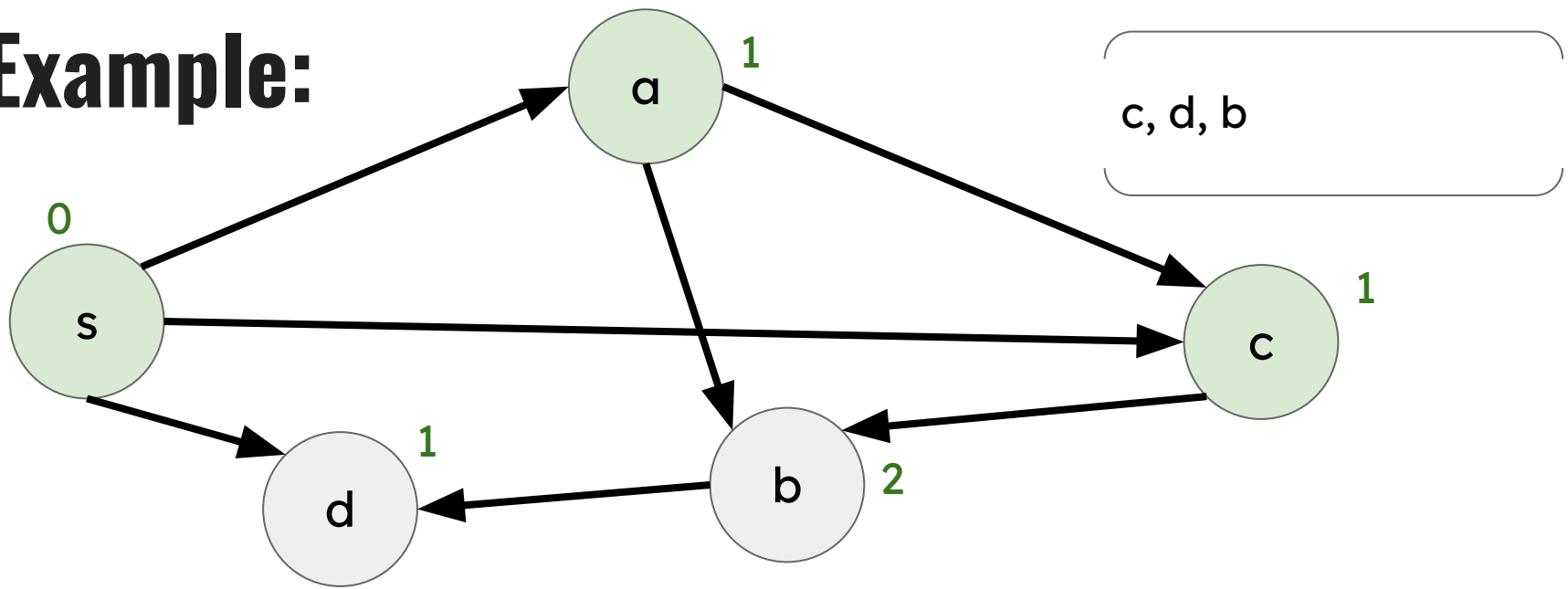
Process its neighbours. Notice that since c has been visited before. We won't do anything to it.

Example:



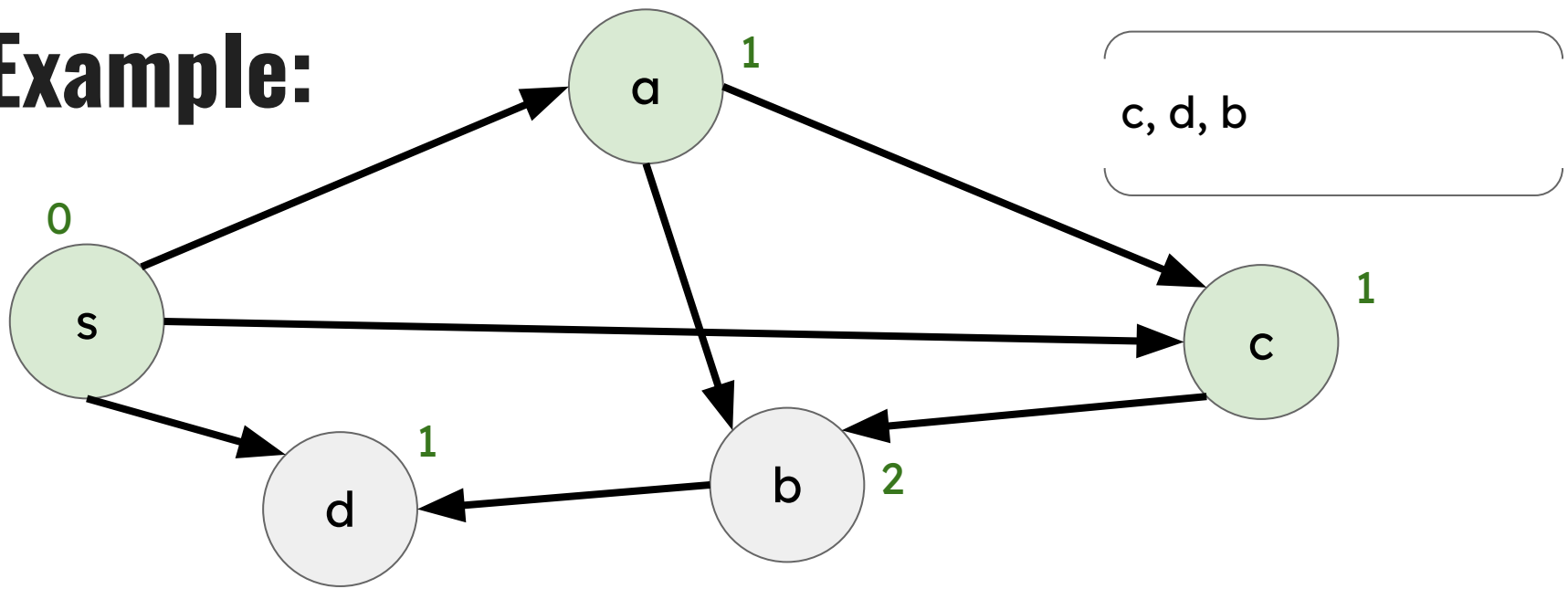
On the other hand, node b has not been visited before. We set its distance to $1 + 1 = 2$. Then mark it as visited, then enqueue.

Example:



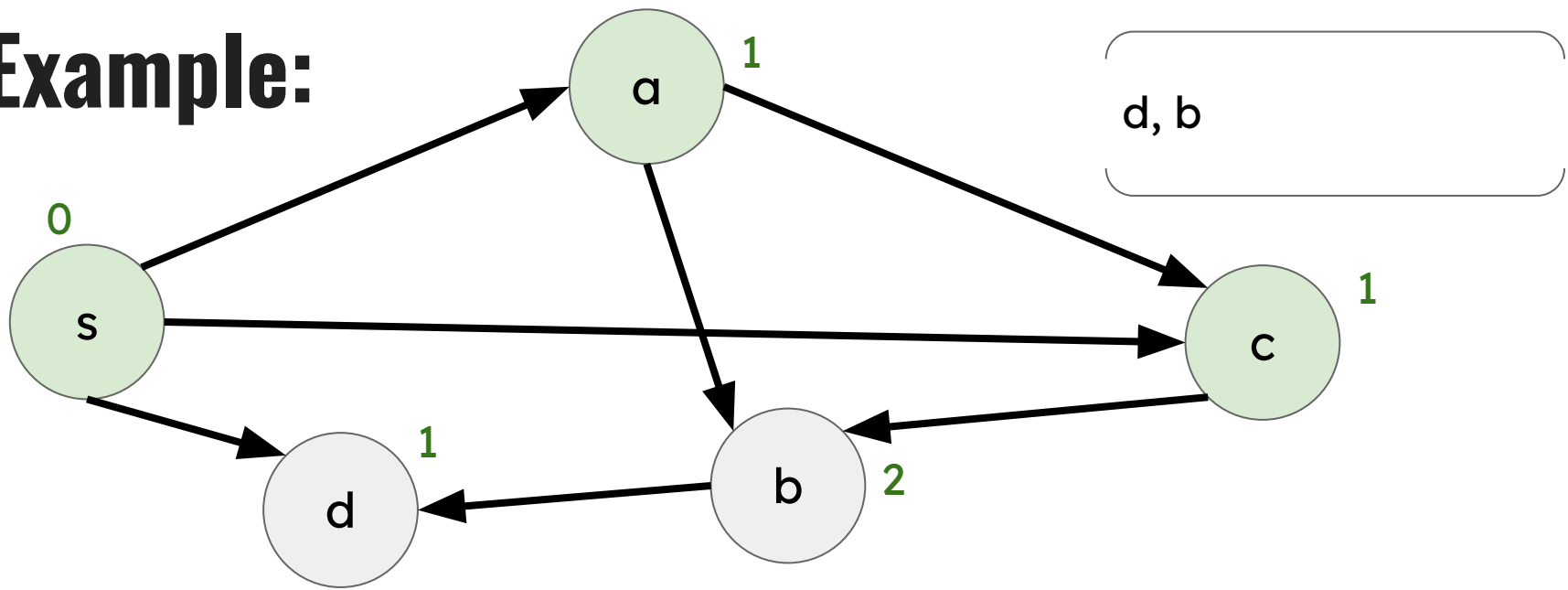
Next, we dequeue and get node c.

Example:



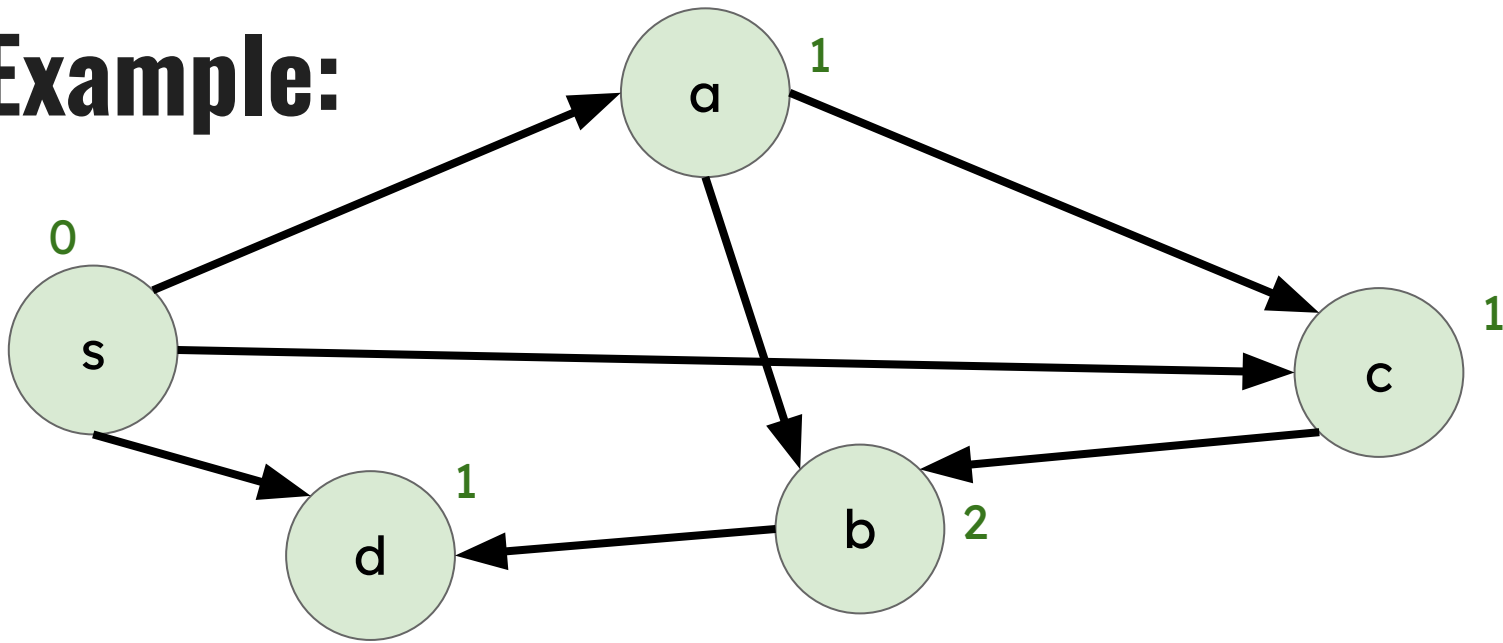
Notice that its only neighbour is b, which is already visited. So there's nothing else to do.

Example:



The same story goes for nodes d and b. When we eventually dequeue them, there's nothing left to process.

Example:



So we will arrive at the correct distances for all of the nodes.

Summary so far:

Case	Complexity
Directed, unweighted	$O(m + n)$

Single Source Shortest Path

Finding short paths from a source node s .

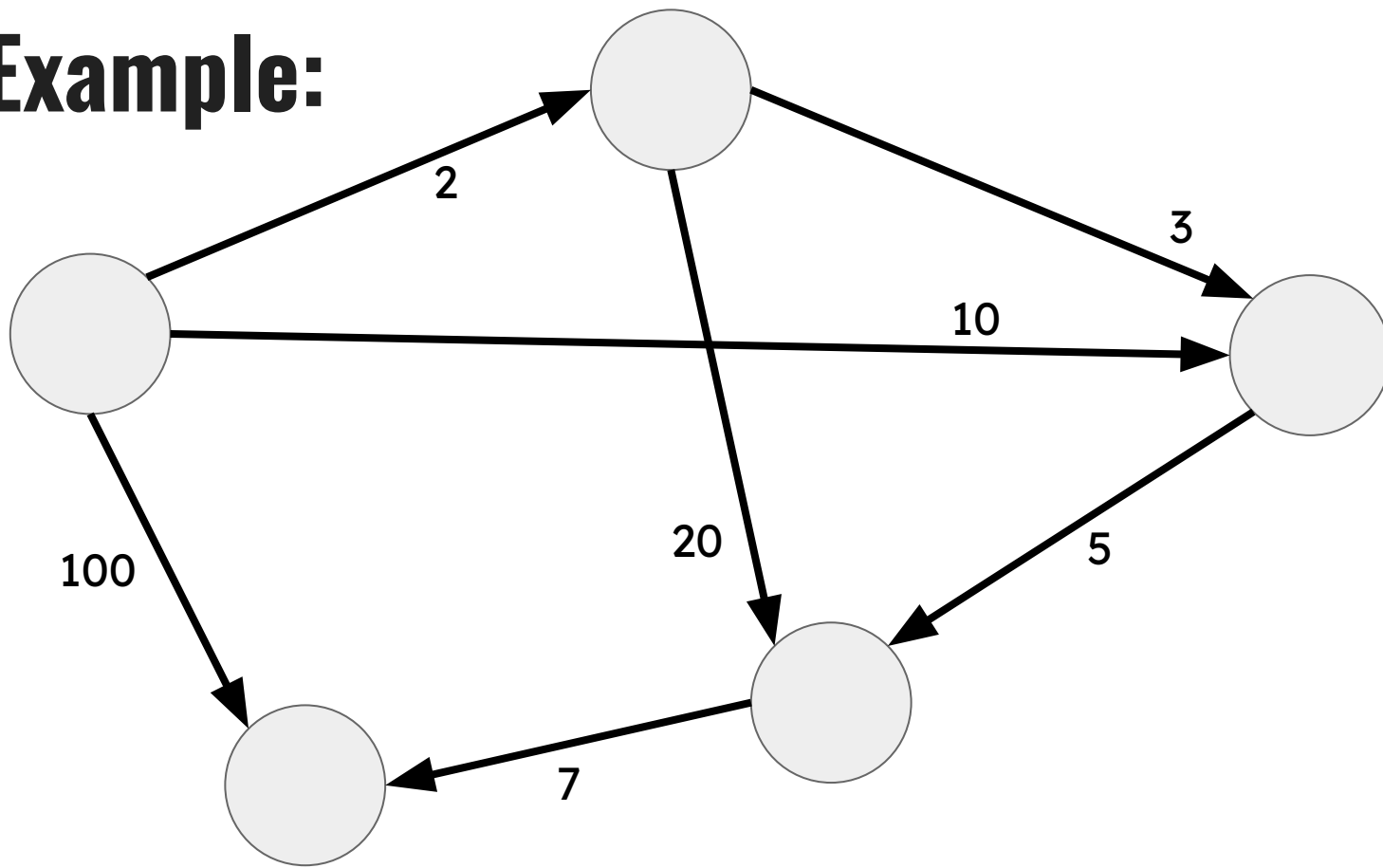
Longest name for a single problem I've heard.

Problem statement:

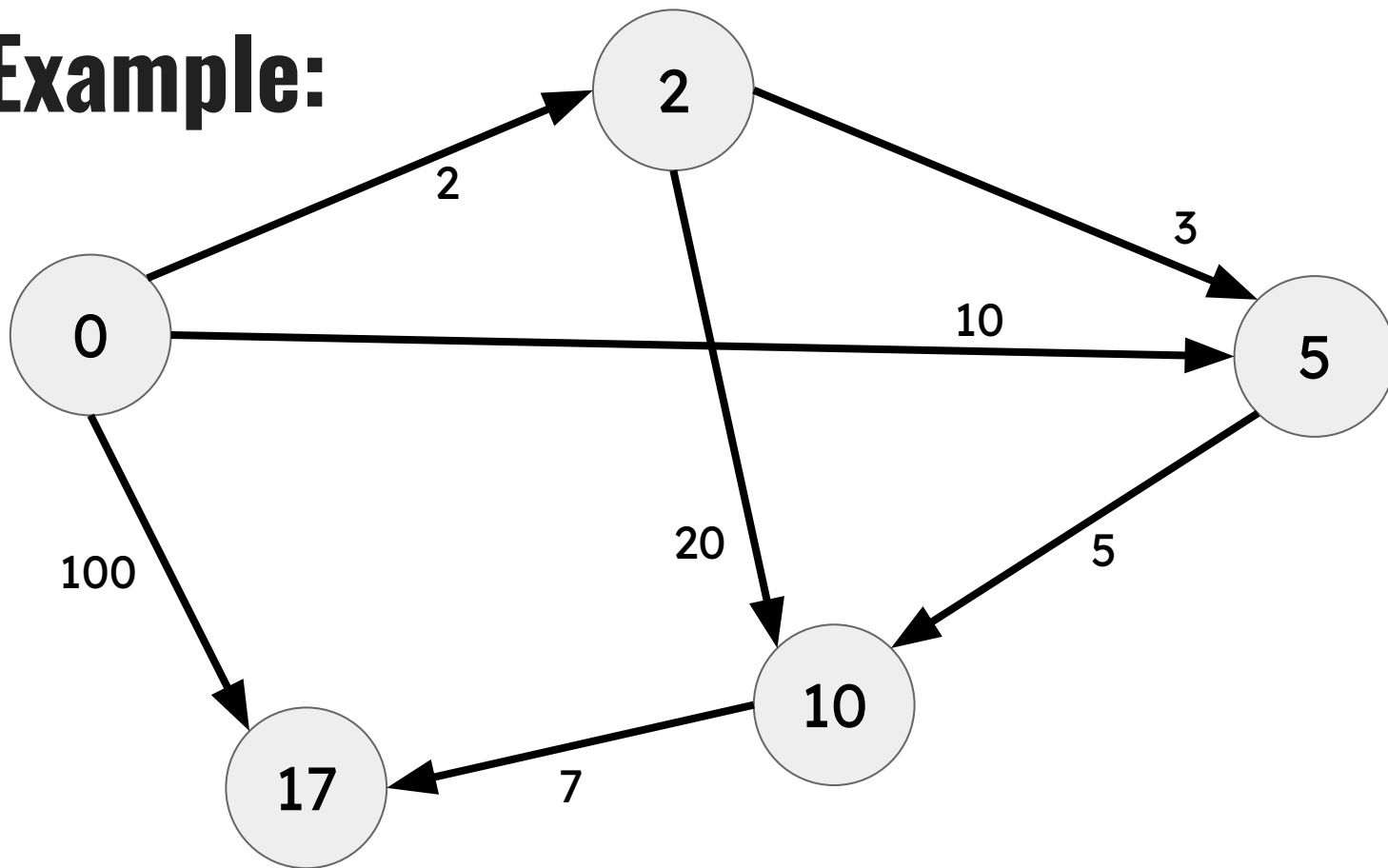
Input: Given a **directed, weighted** graph $G = (V, E)$ in the form of an adjacency list, with **n** nodes and **m** edges, and a source node **s**.

Output: An array **dist[]** such that for all nodes **v**, **dist[v]** is the shortest distance from node **s** to node **v**.

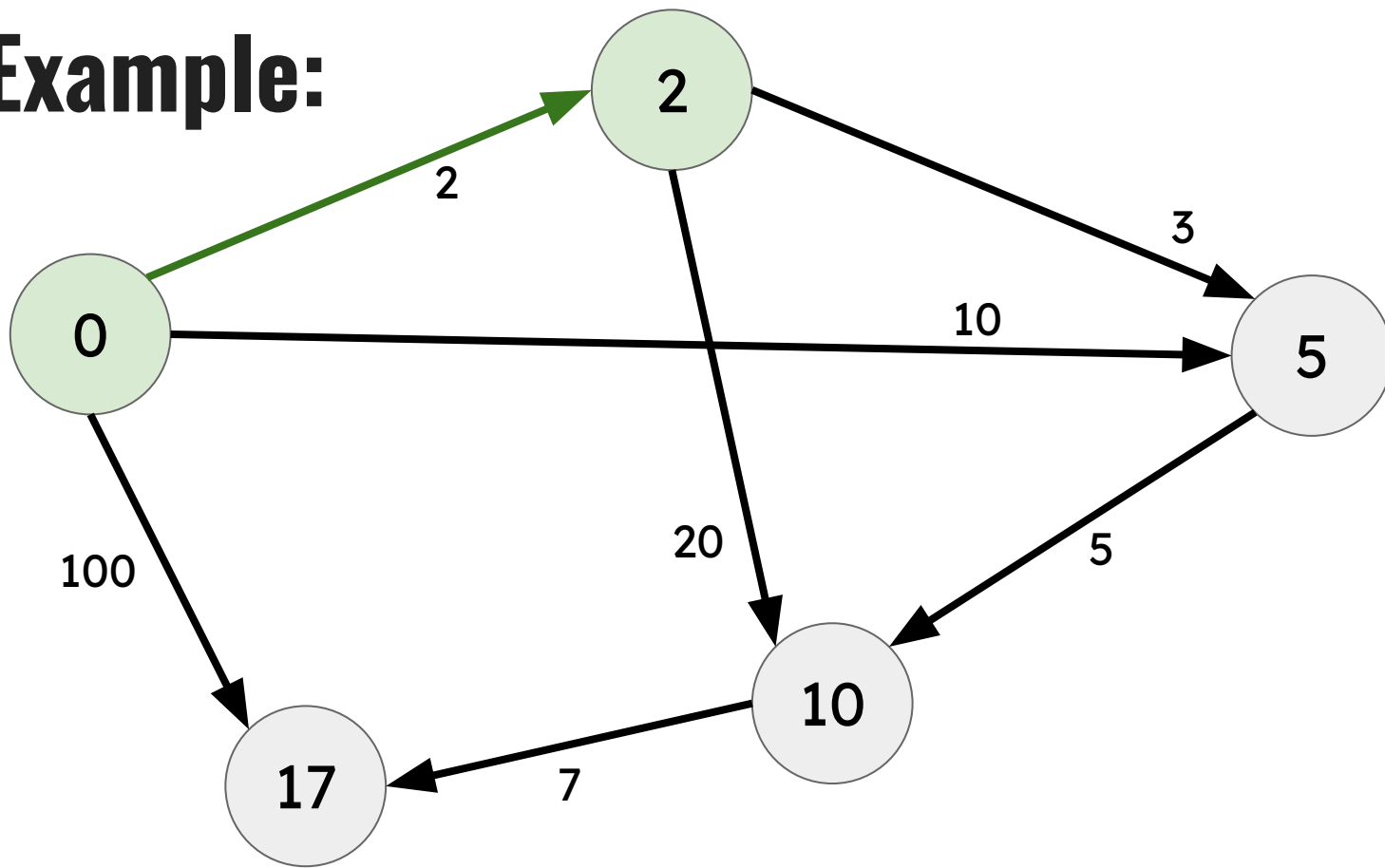
Example:



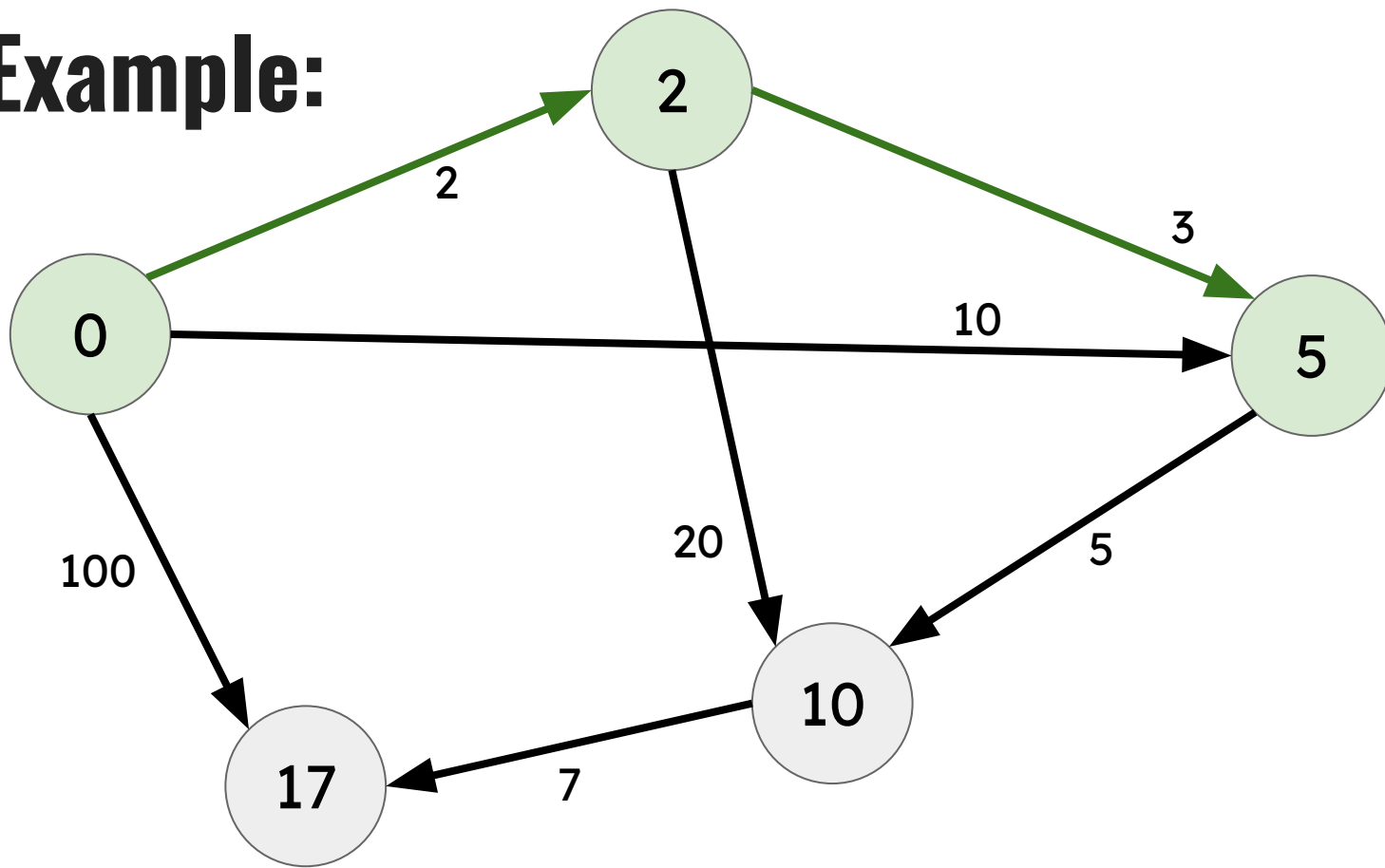
Example:



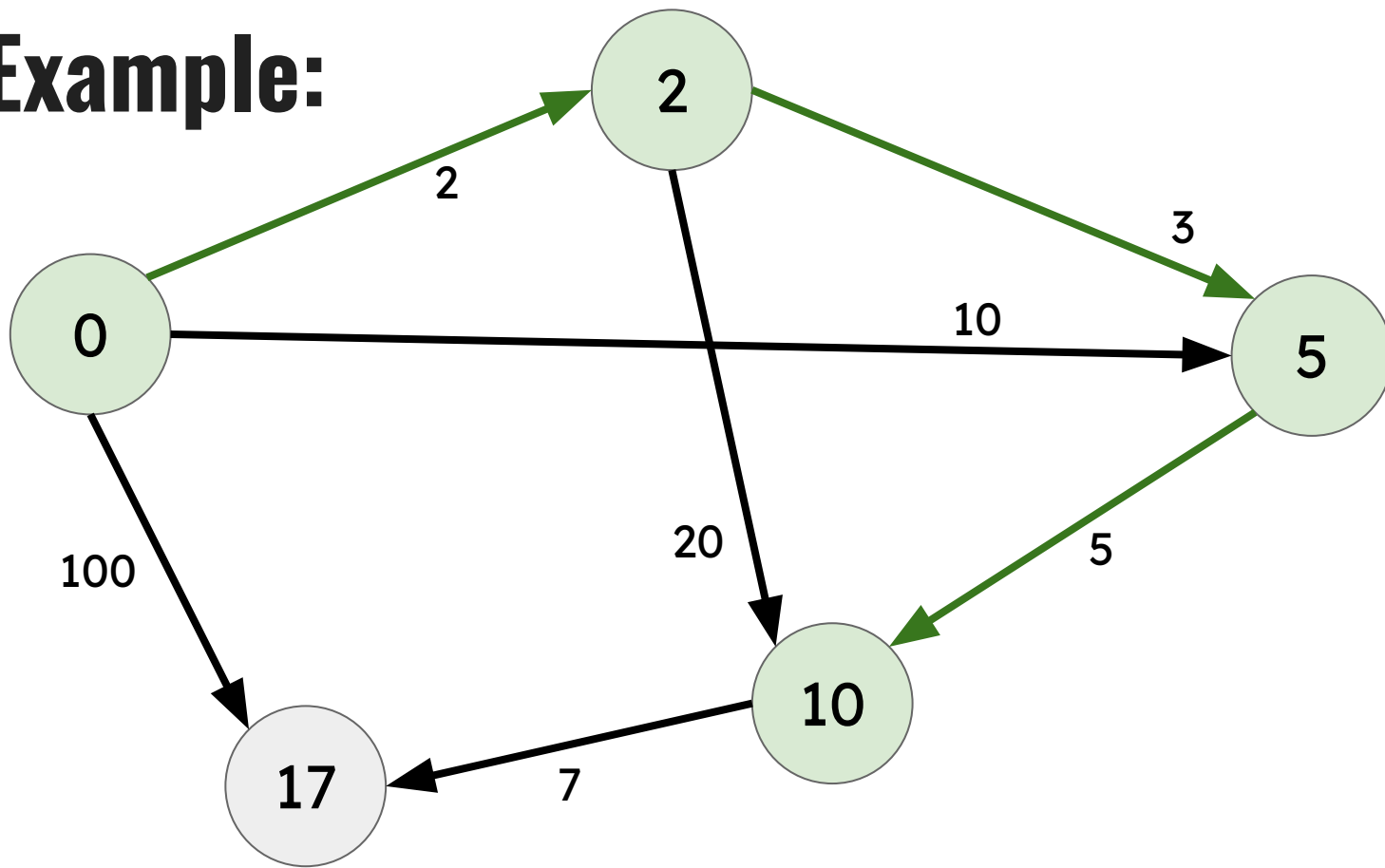
Example:



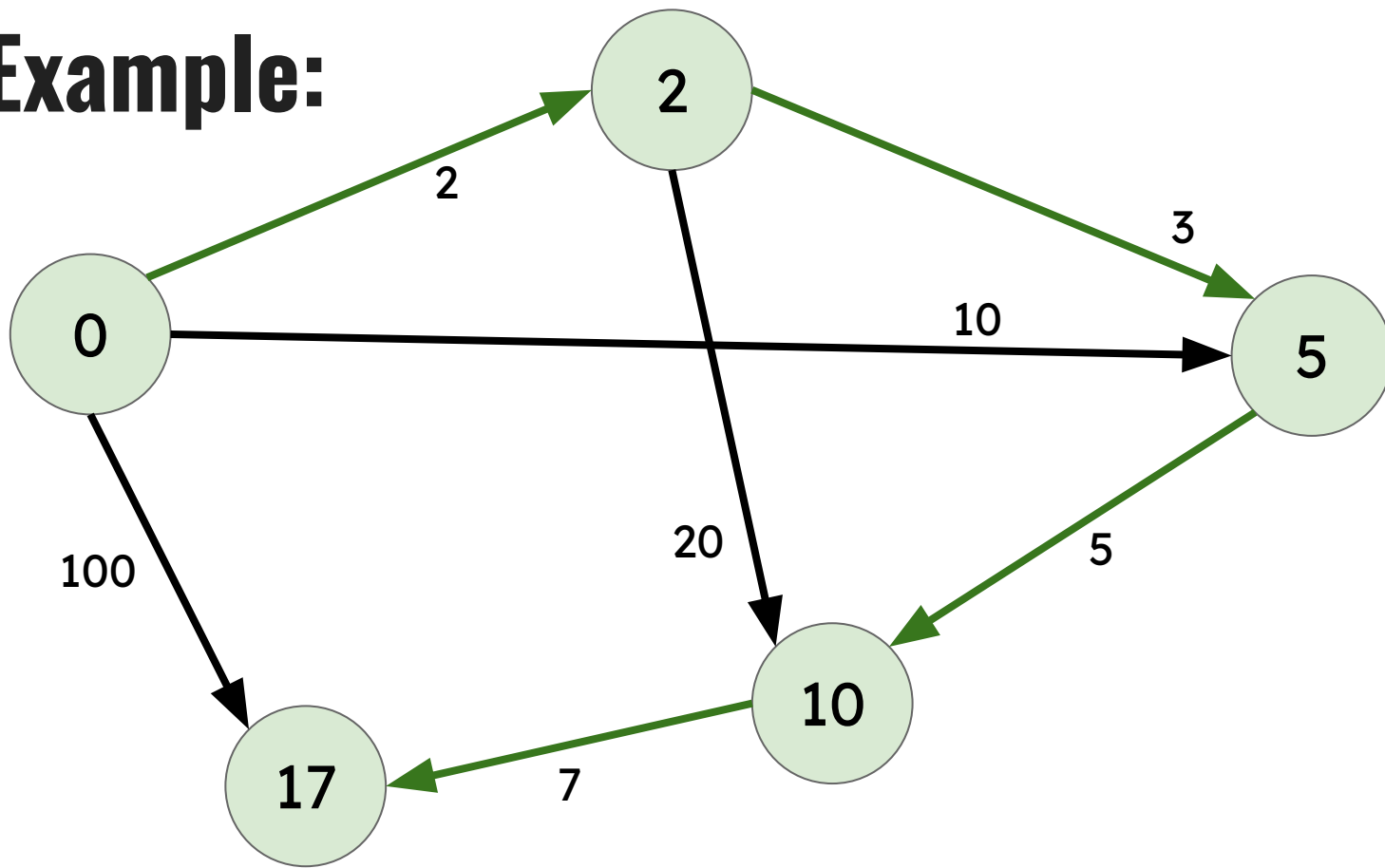
Example:

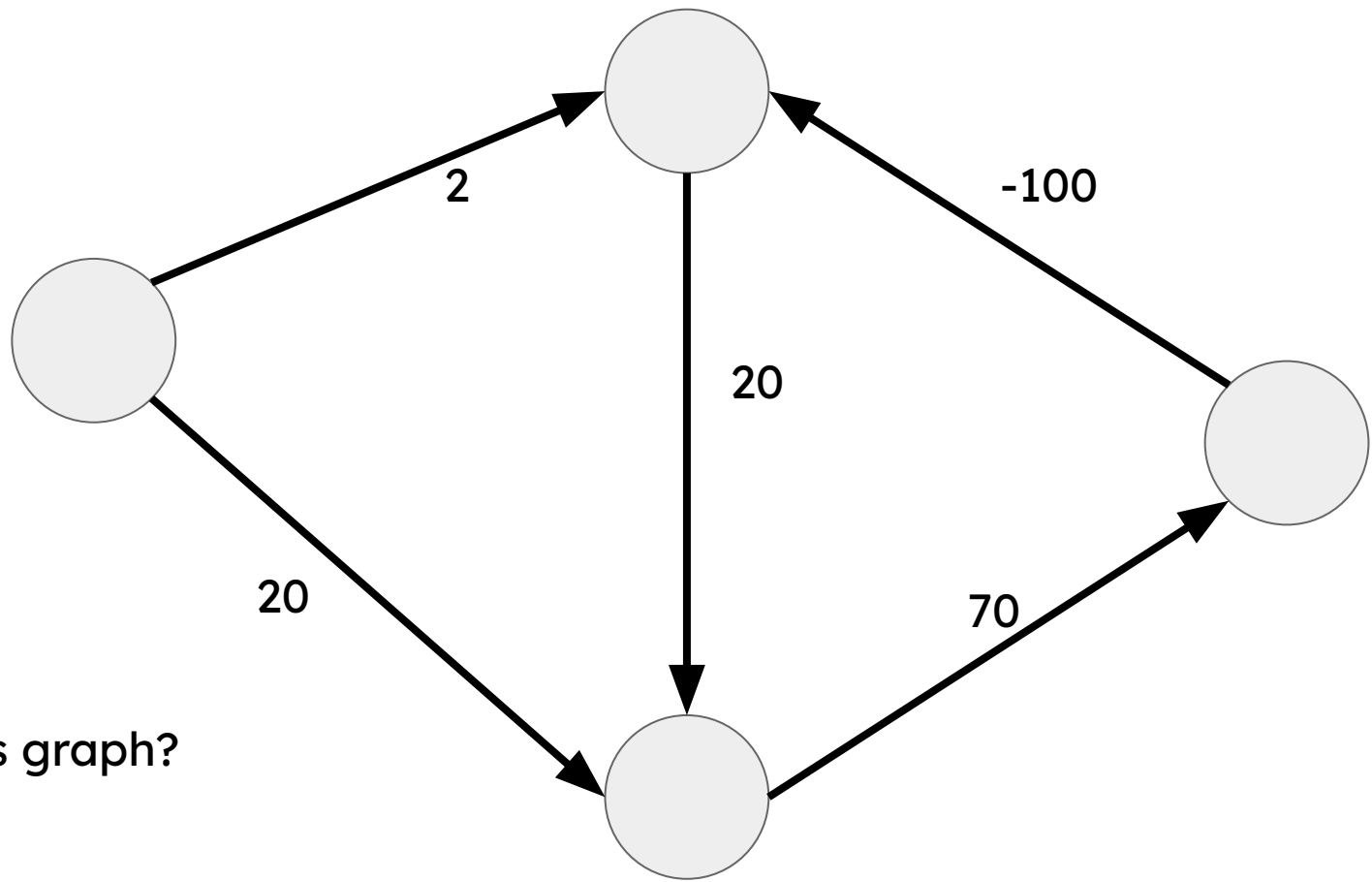


Example:

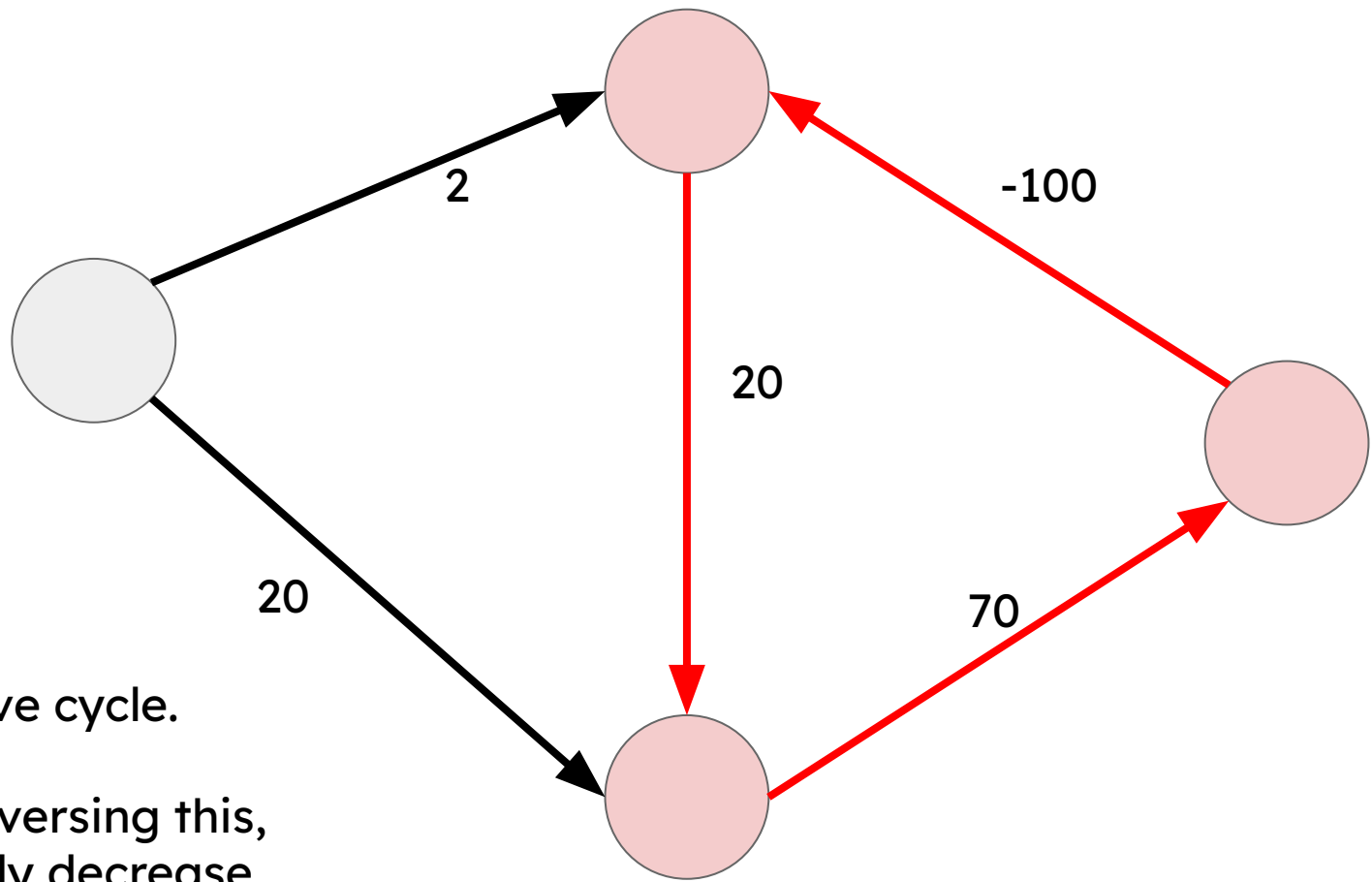


Example:





What about this graph?



There's a negative cycle.

So if we kept traversing this,
we could infinitely decrease
our distance.

Bellman-Ford

Finding short paths from a source node s . (With no negative cycles)

Problem Statement:

Input: Given a **directed, weighted** graph $G = (V, E)$ in the form of an adjacency list, with **n** nodes and **m** edges, and a source node **s** . You are promised the graph has **no negative cycles**.

Output: An array **dist[]** such that for all nodes **v** , **dist[v]** is the shortest distance from node **s** to node **v** .

A few observations:

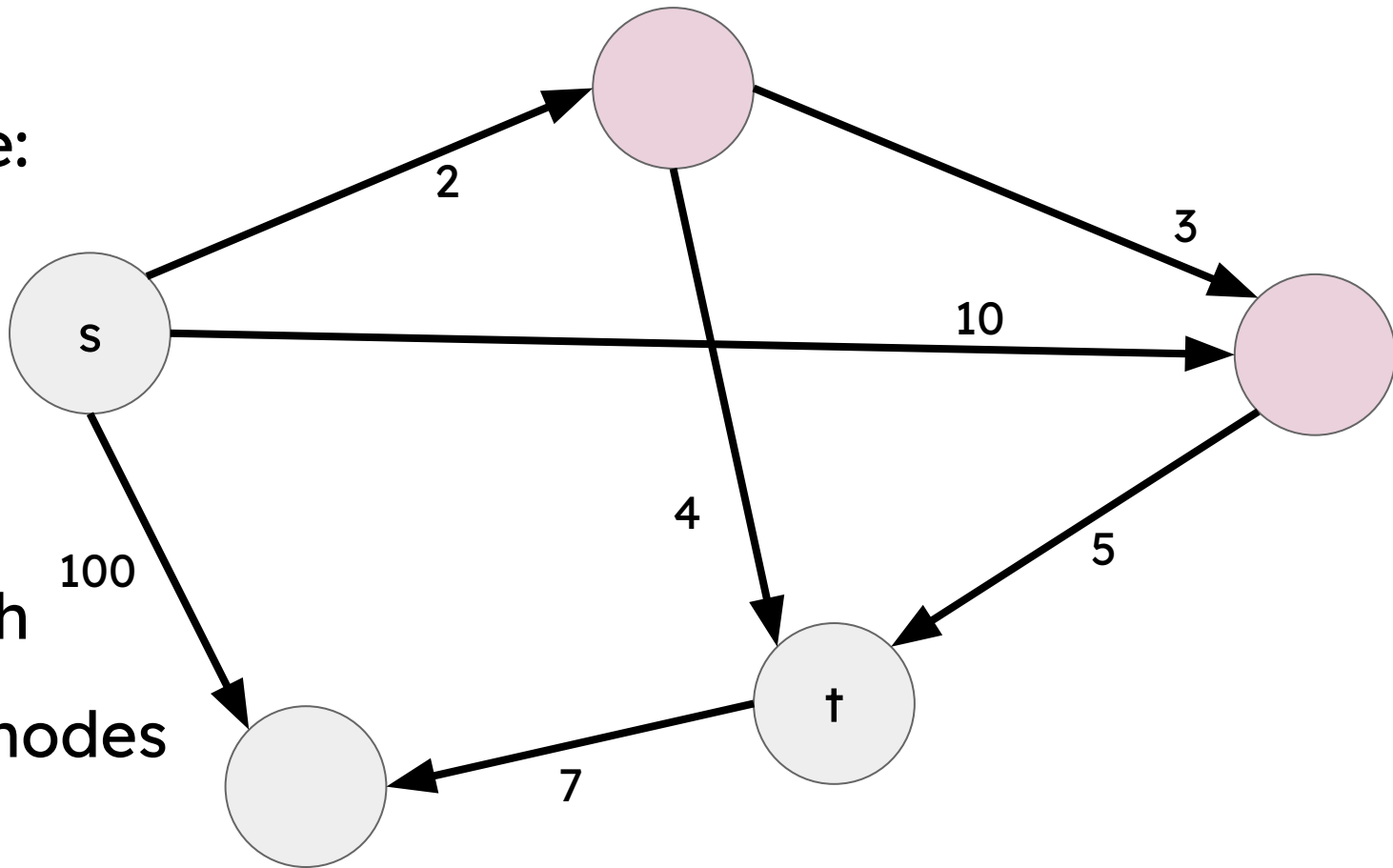
Conjecture #1: Given a node t that we want to know the shortest distance to.

Consider $P(t)$ as the set of nodes that have some directed edge to node t .

Then the shortest distance from s to t is the minimum of:

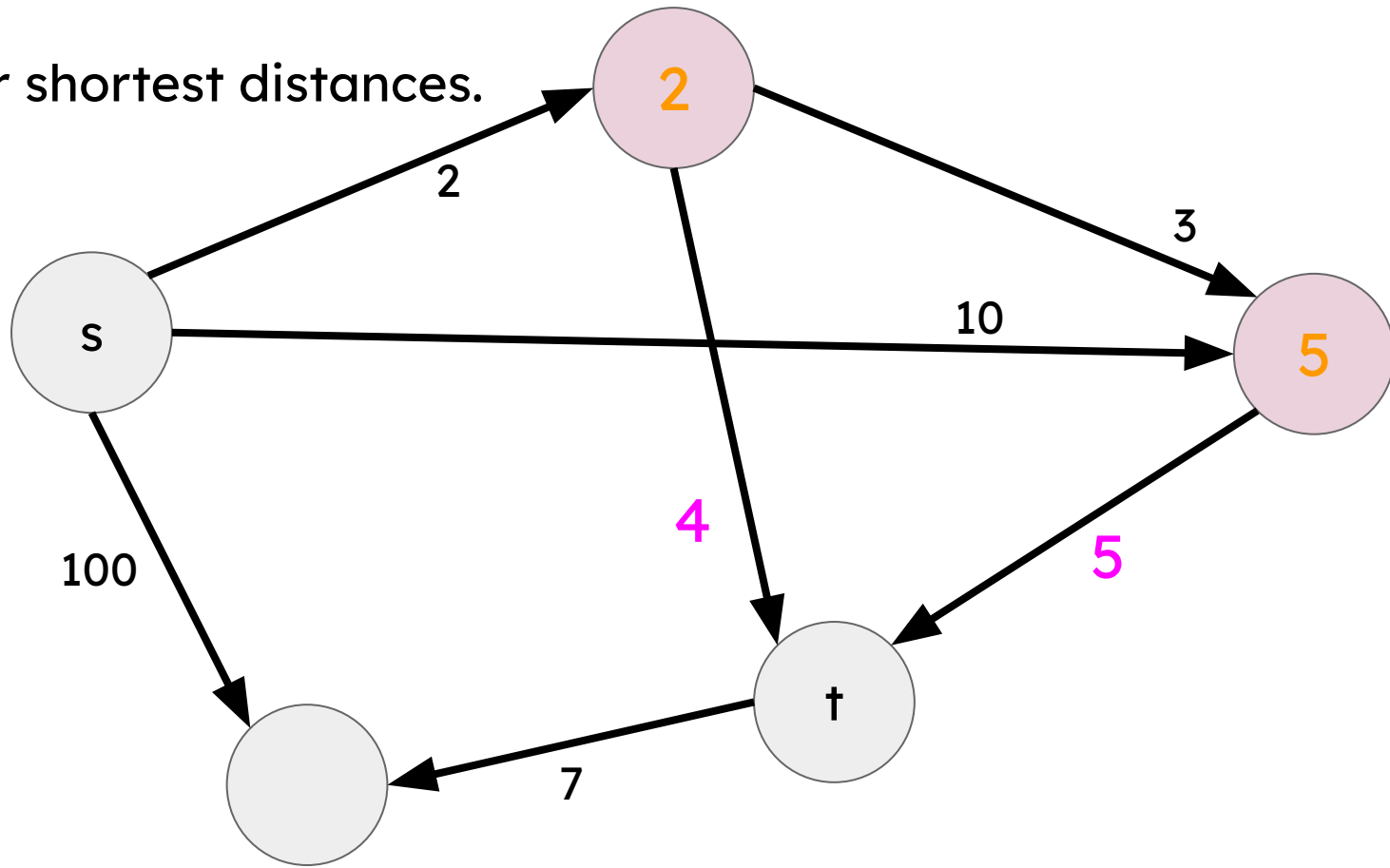
The shortest distance from s to p and from p to t , over all p in $P(t)$.

For example:

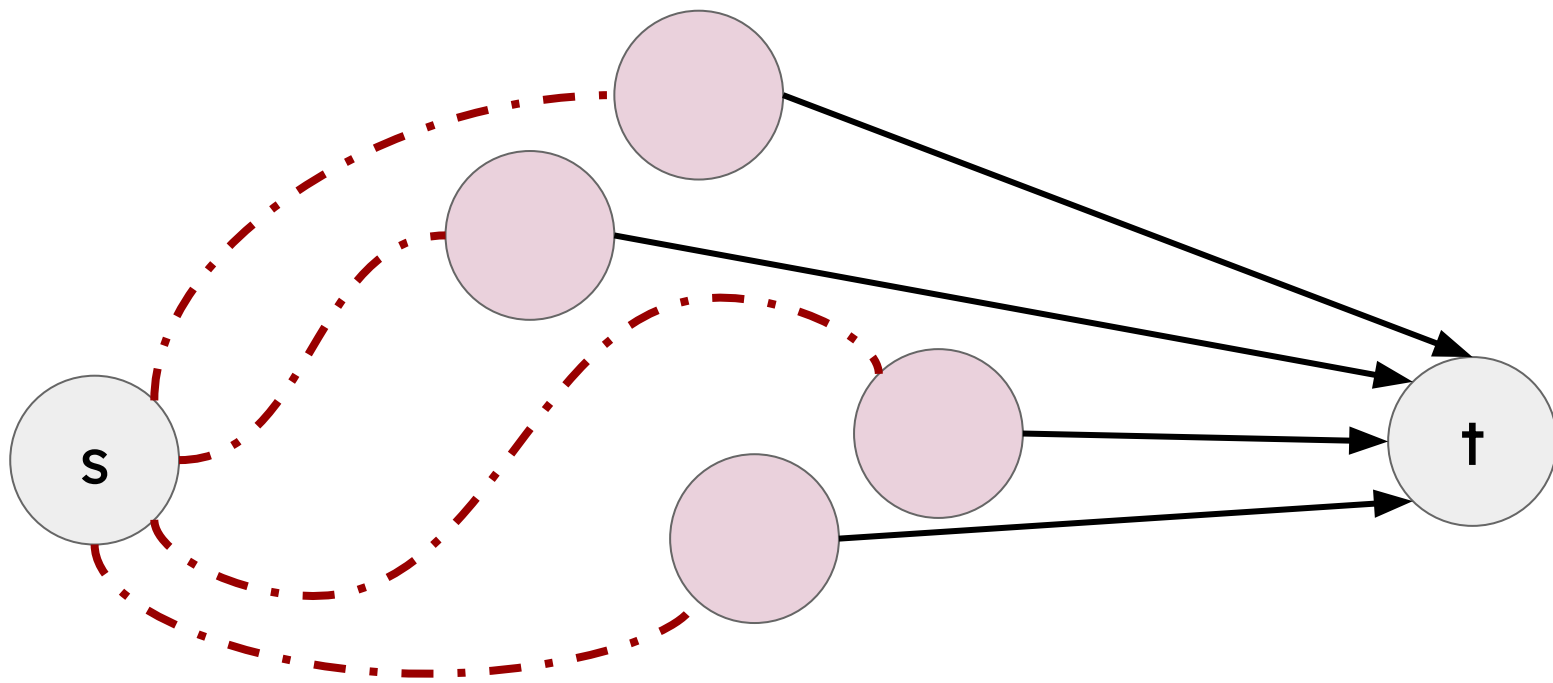


In this graph
there are 2 nodes
in $P(t)$.

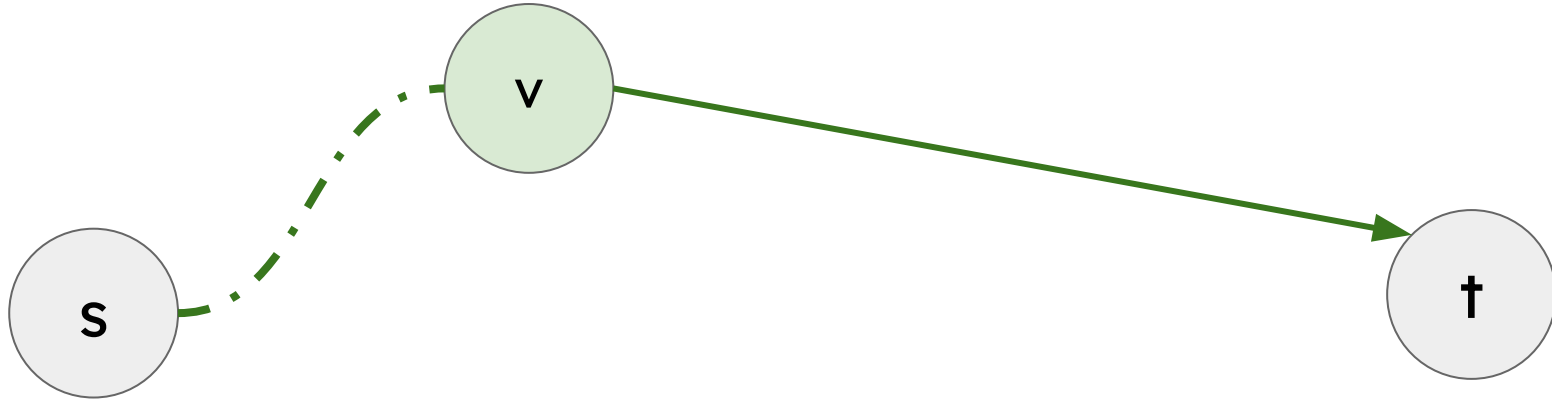
Consider their shortest distances.



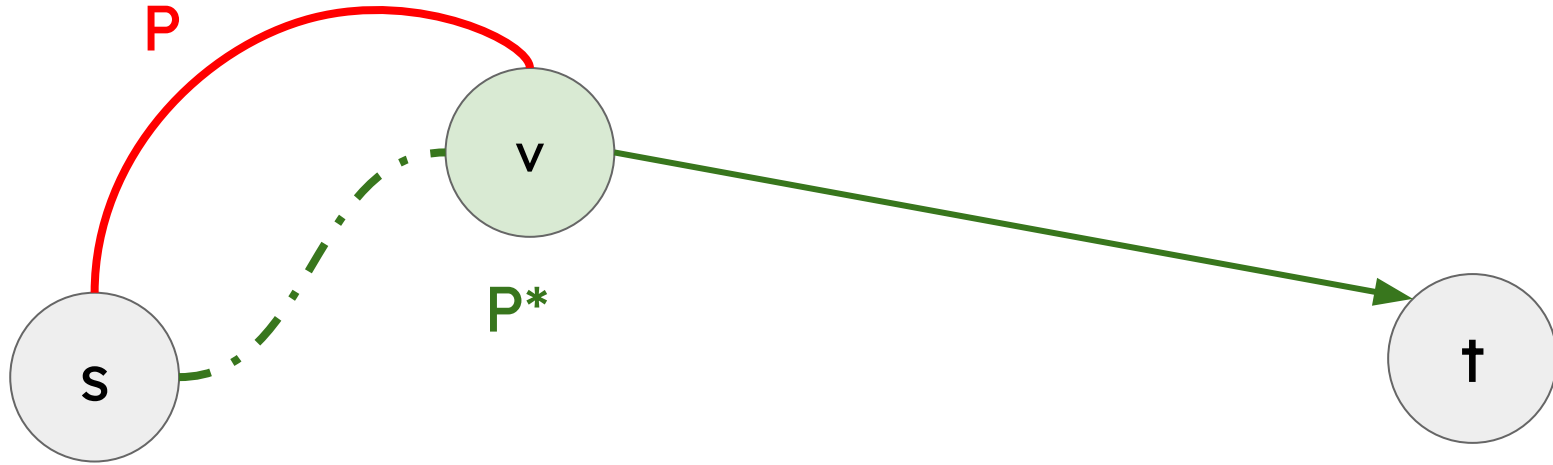
Then $\text{dist}[t] = \min(2 + 4, 5 + 5)$



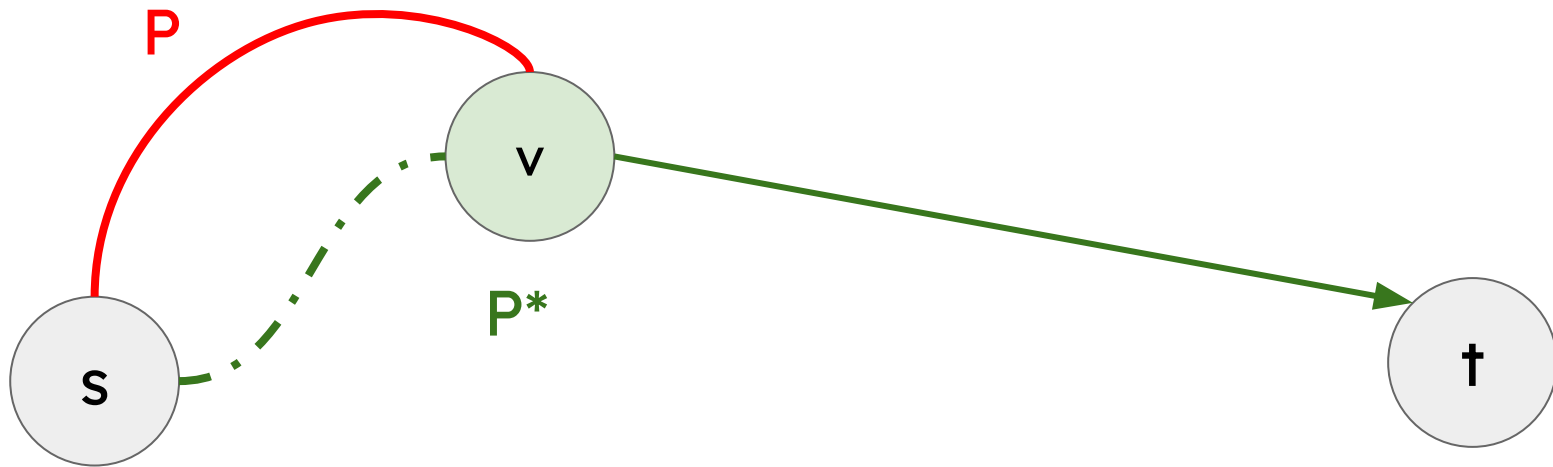
In general, let's consider the set of nodes $P(t)$. Consider the distance they are from s , and the distance they are to node t .



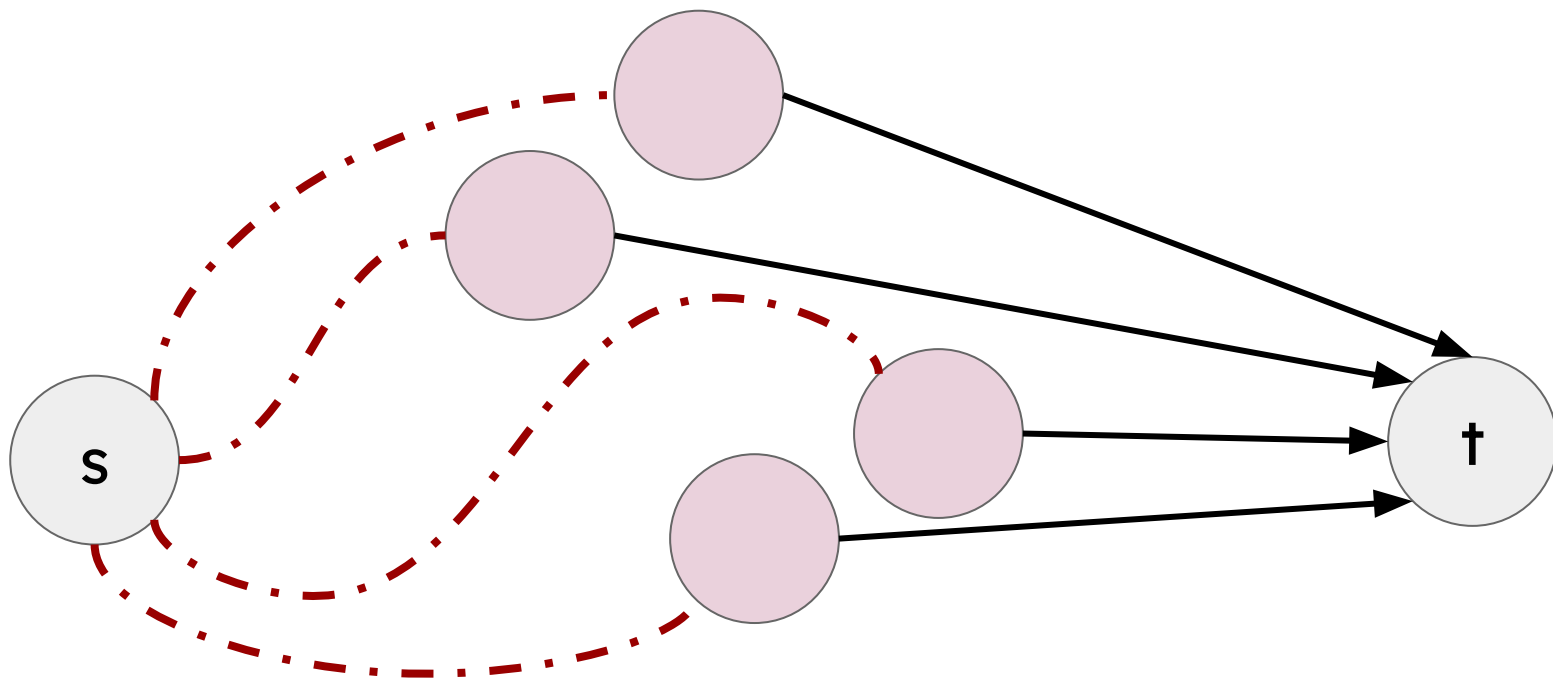
Consider a shortest path P^* from node s to t . WLOG assume it goes through some node v .



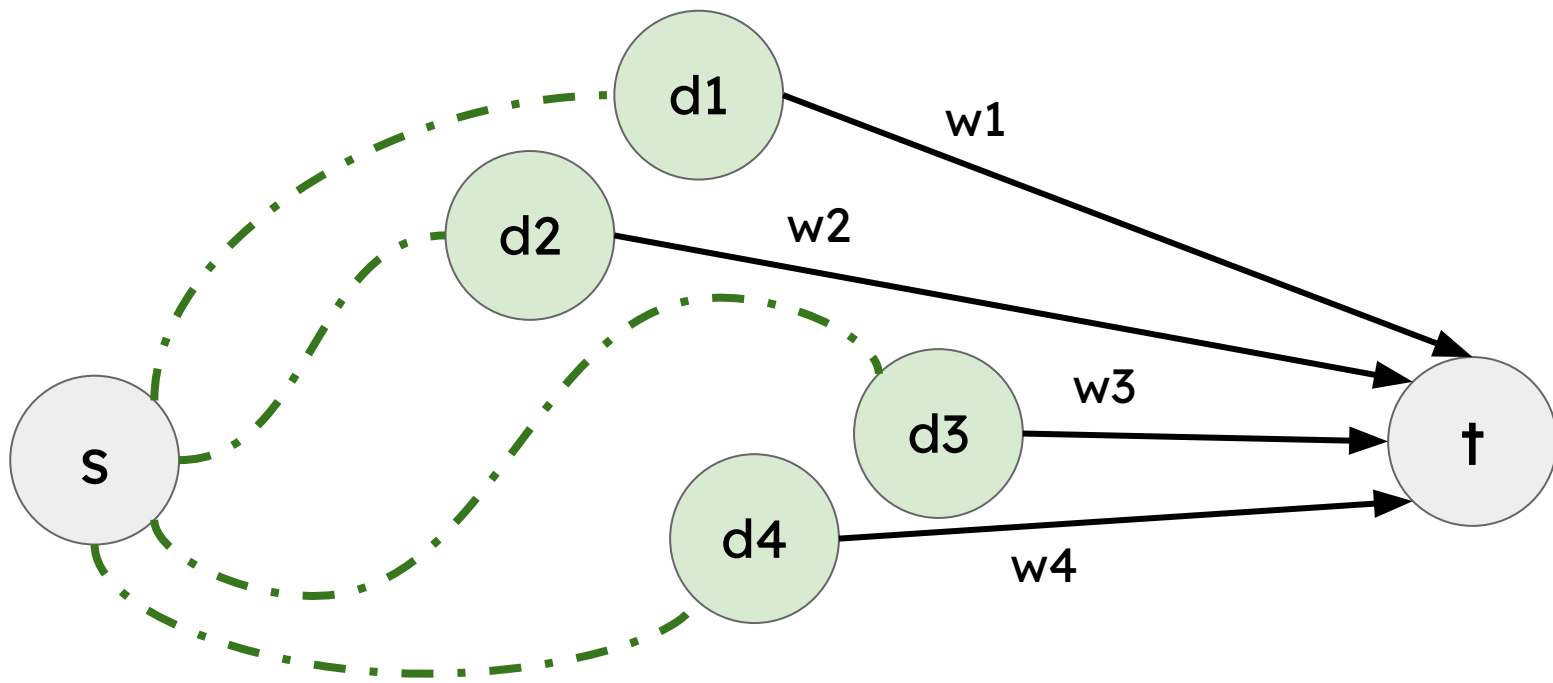
What happens if a shortest path P from s to v is somehow longer than the path from s to v according to P^* ?



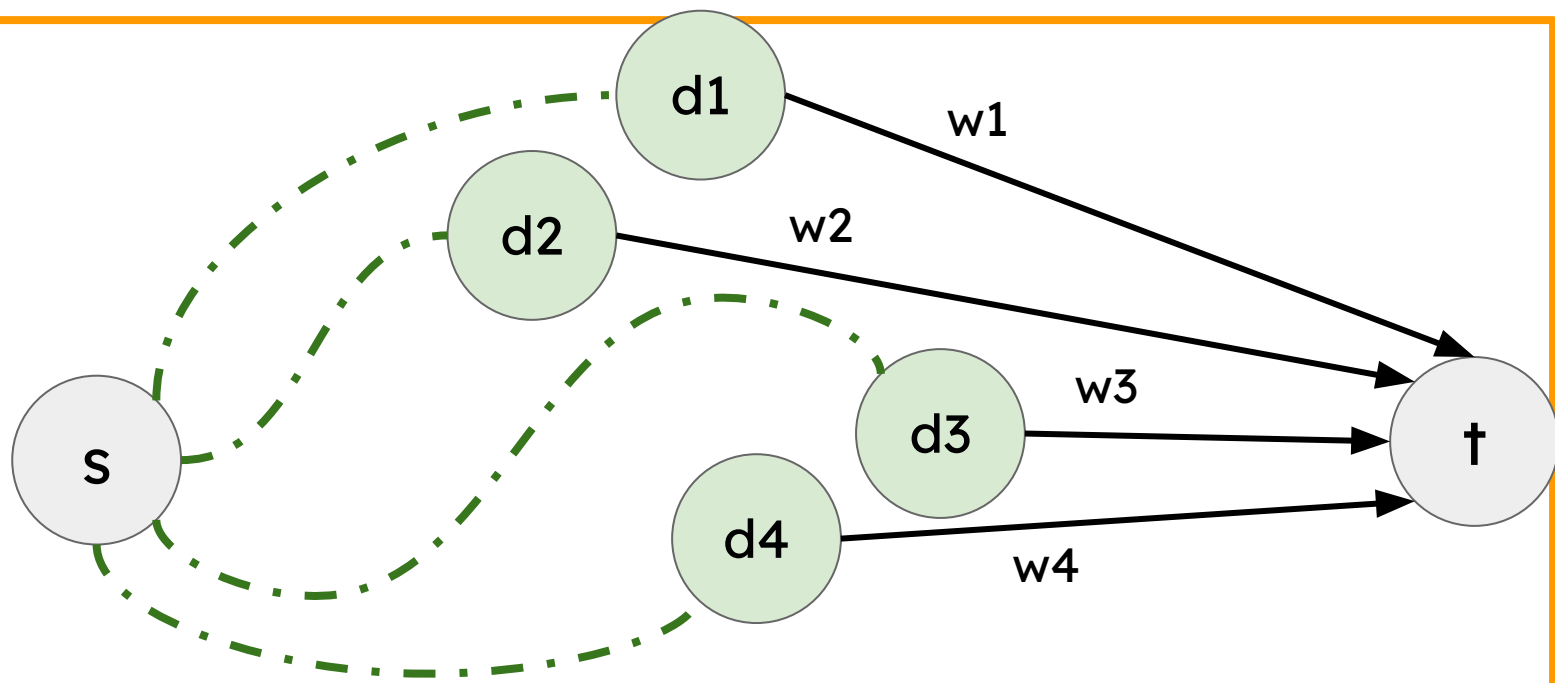
Then we could have used P^* to get an even shorter path from s to v .



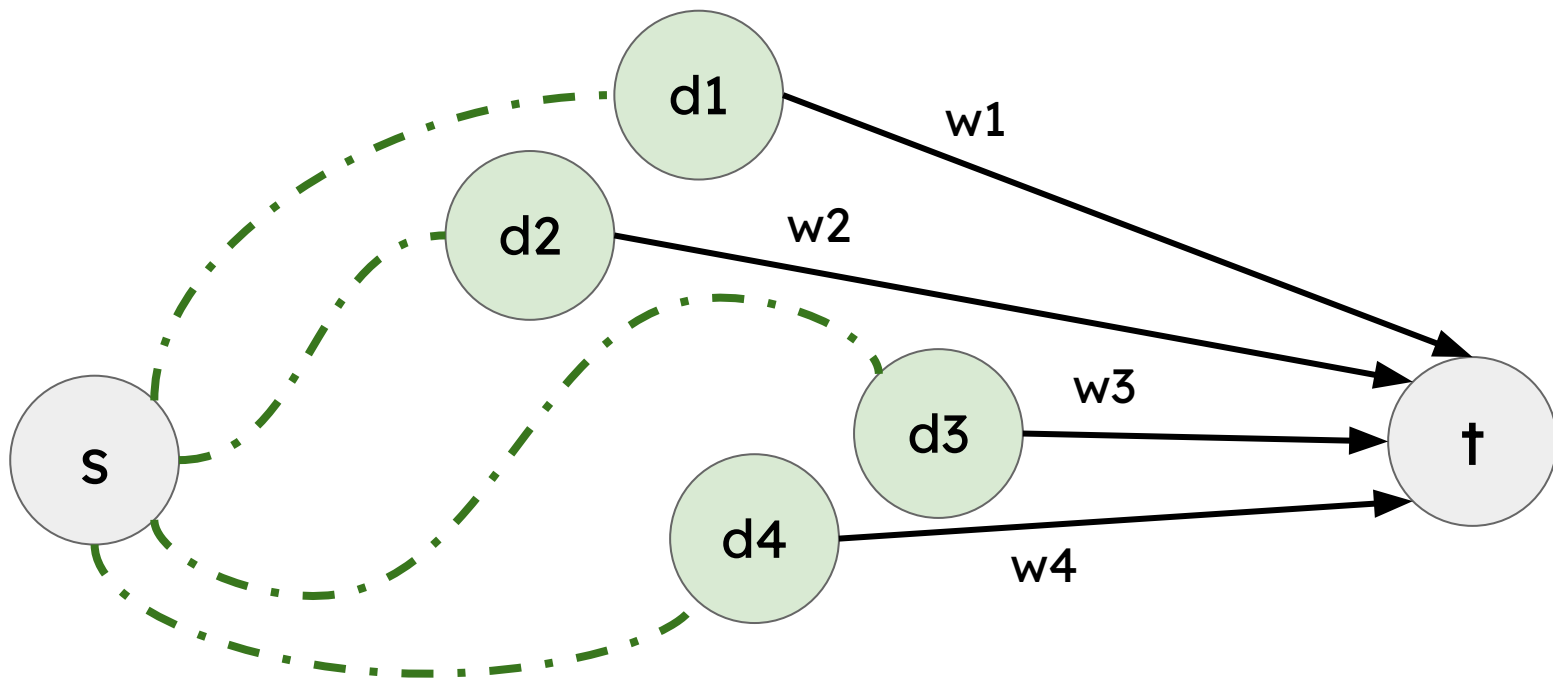
So to figure out a shortest path from s to t , we should figure out a shortest paths from s to $P(t)$ first.



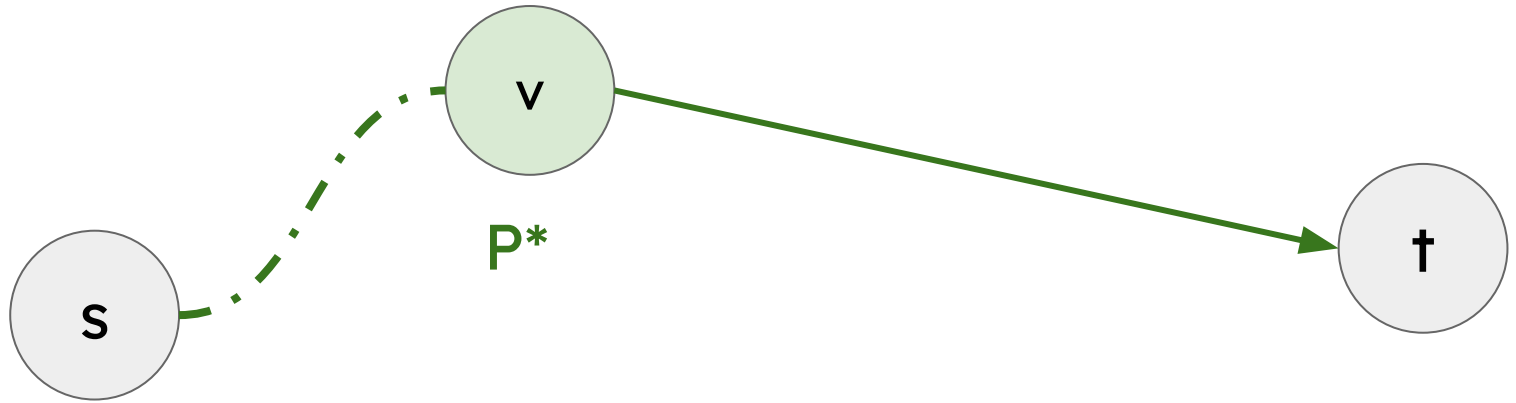
Then, among all the nodes t in $P(t)$, **after** we know their shortest paths, we can try to figure out a shortest path to t .



In general, how should we figure out a shortest path to t ? Using shortest distances $d1$, $d2$, $d3$, $d4$, and if we know their edge weights are $w1$, $w2$, $w3$, $w4$?

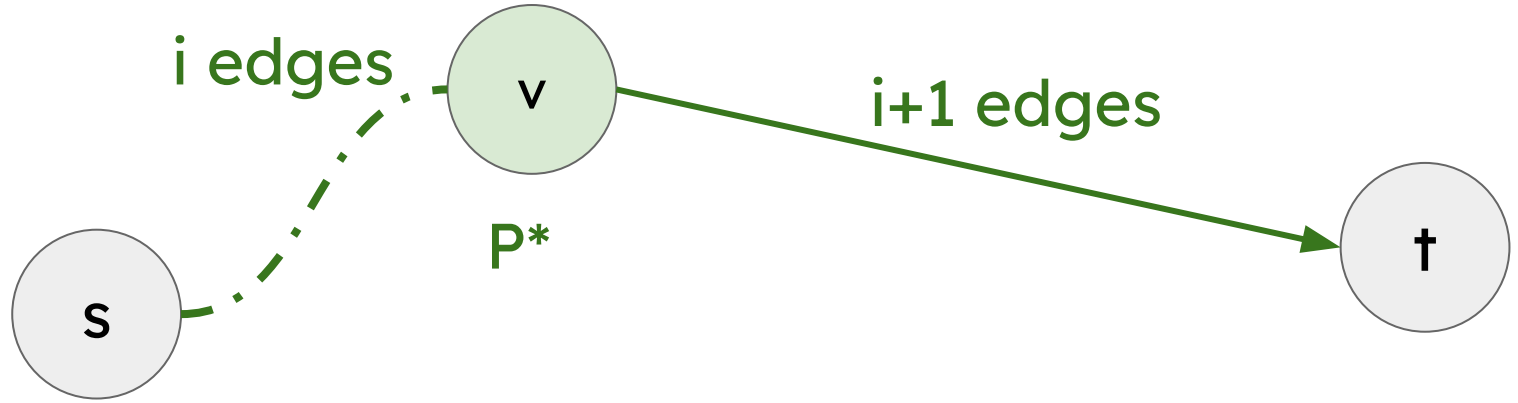


$$\text{dist}[t] = \min(d1 + d1, d2 + w2, d3 + w3, d4 + w4)$$



Let's try to make another observation. Let's say v is in $P(t)$, and is on a shortest path to t . So far we know this path also gives us a shortest path from s to v .

If a shortest path from s to v takes i edges, how many edges does a shortest path from s to v take?



It takes just 1 more edge, so $i + 1$ edges.

A few observations:

Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

A few observations:

Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

Assume not,

A few observations:

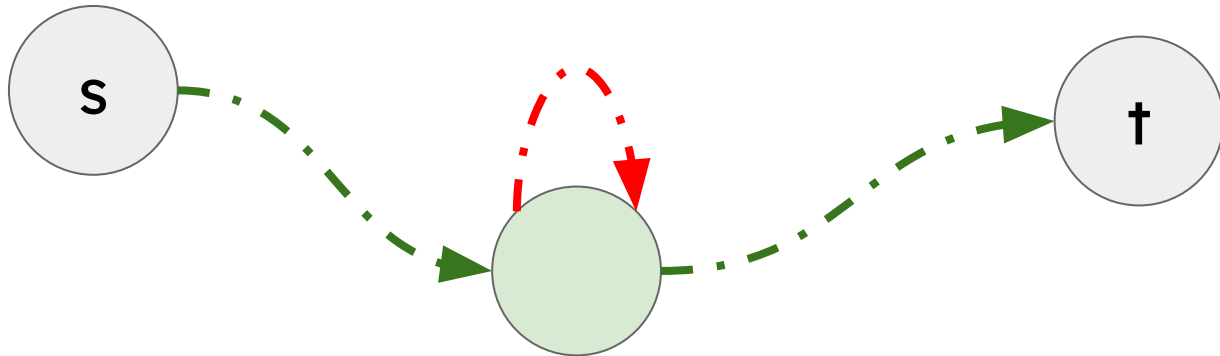
Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

Assume not, then by pigeonhole principle, our path has at least 1 repeated node, i.e. there is a cycle somewhere.

A few observations:

Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

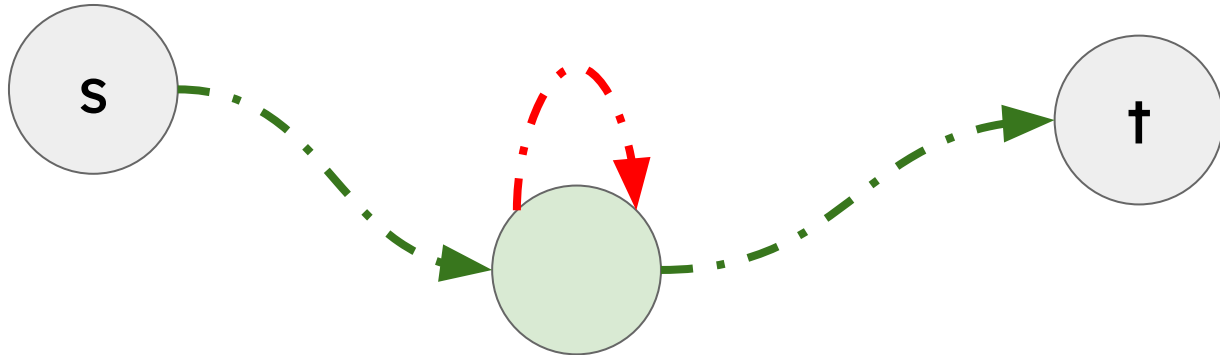
Assume not, then by pigeonhole principle, our path has at least 1 repeated node, i.e. there is a cycle somewhere.



A few observations:

Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

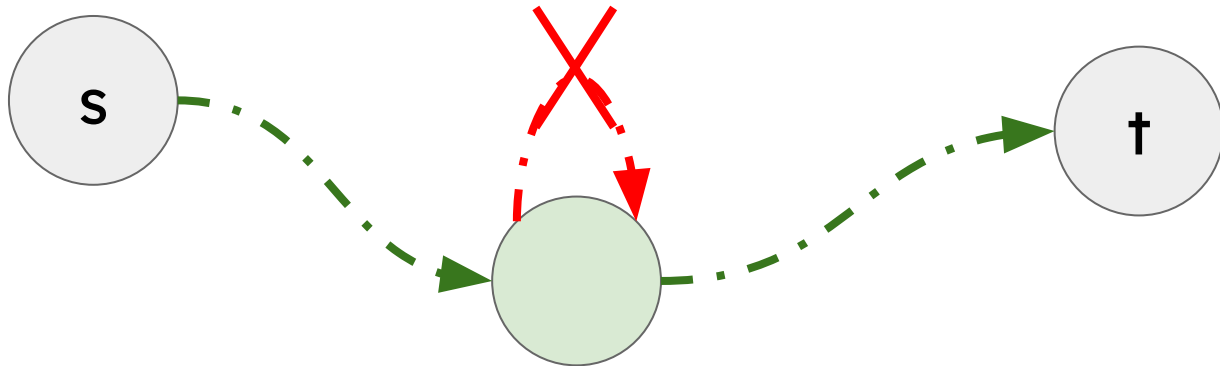
By our assumption, the cycle is non-negative. So the cycle must cause a distance of 0 or above.



A few observations:

Conjecture #2: There exists a shortest path from s to any node v in the graph uses at most $n - 1$ edges.

So if we consider the path from s to t but without traversing the cycle, our distance won't ever worsen.



A few observations:

Observation 1: To solve the shortest distance from s to t , we first need to solve shortest distance from s to all nodes $P(t)$.

Observation 2: If a shortest path from s to t goes through node v (where v is from $P(t)$), then we know that this also gives us a shortest path from s to v .

If that path to v uses i edges, then the path to t uses $i + 1$ edges.

Observation 3: There exists a shortest path that uses at most $n - 1$ edges.

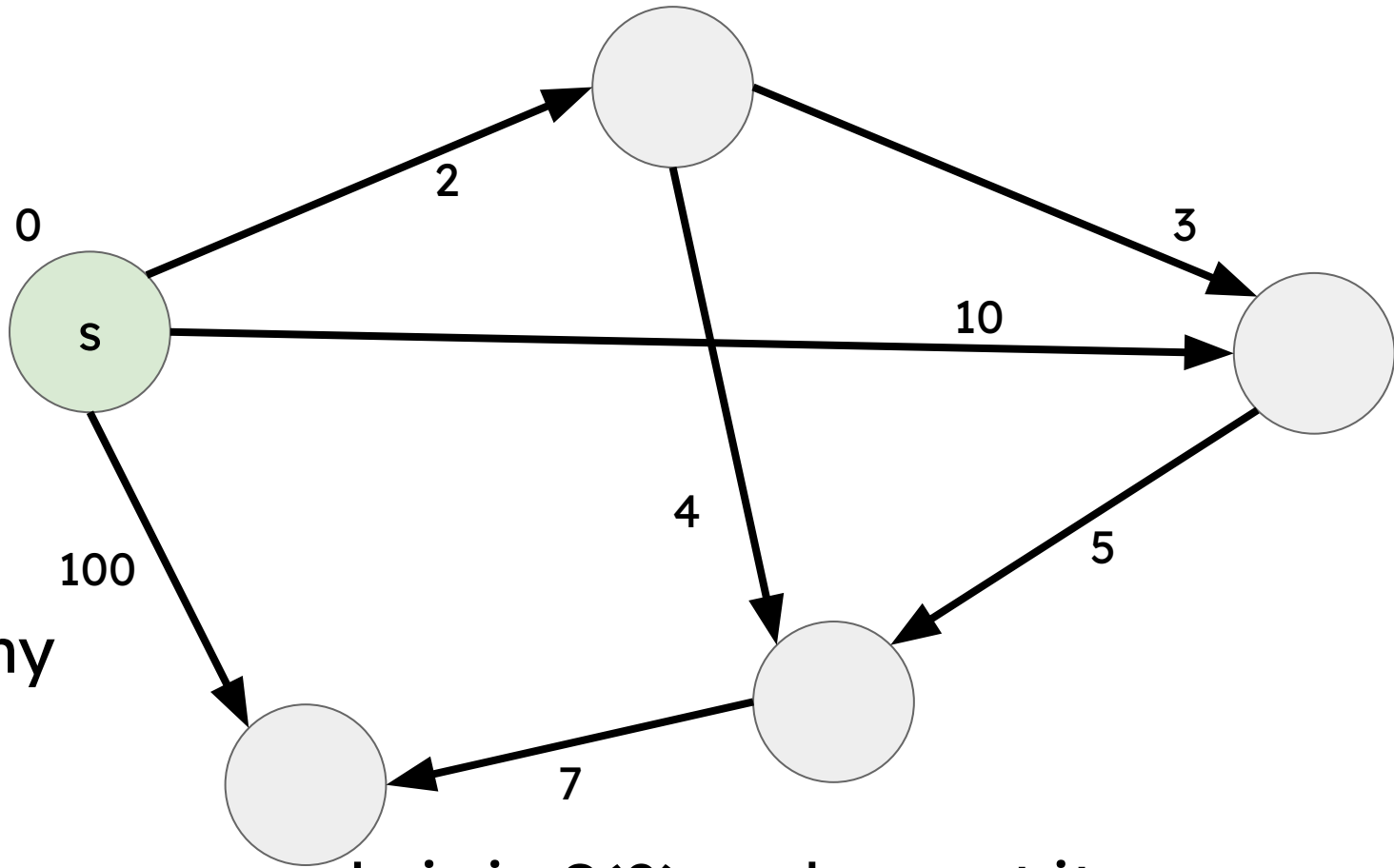
Approach #1:

If we group the nodes by how many edges their shortest path needs.

I.e. $S(1)$ is the subset of nodes that need 1 edge in the shortest path from s , $S(2)$ is the subset that needs 2 edges, and so on.

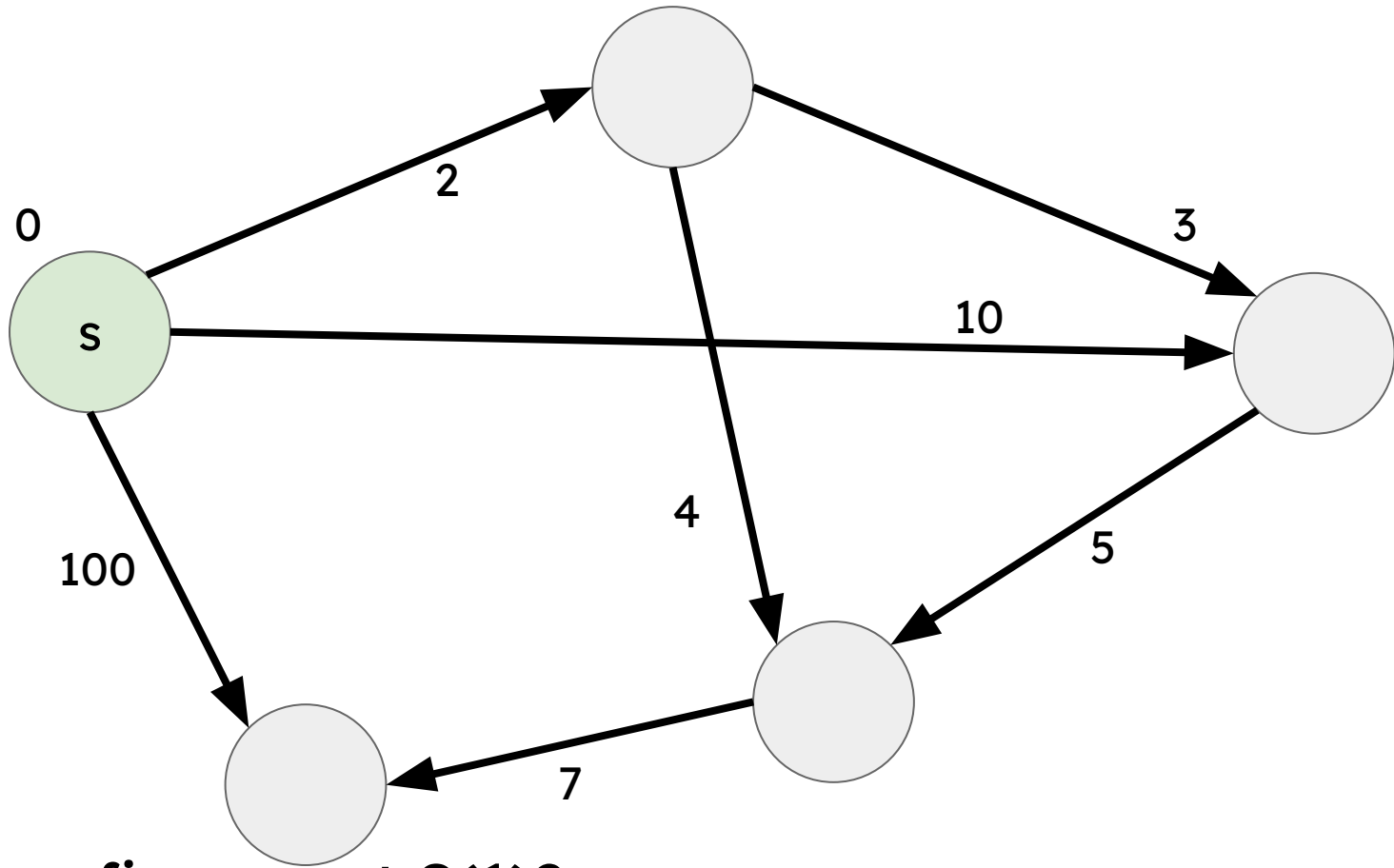
Then:

- $S(0)$ is already solved.
- Assuming $S(i)$ is solved, we should use it to solve $S(i + 1)$
- Do this until we solve $S(n - 1)$



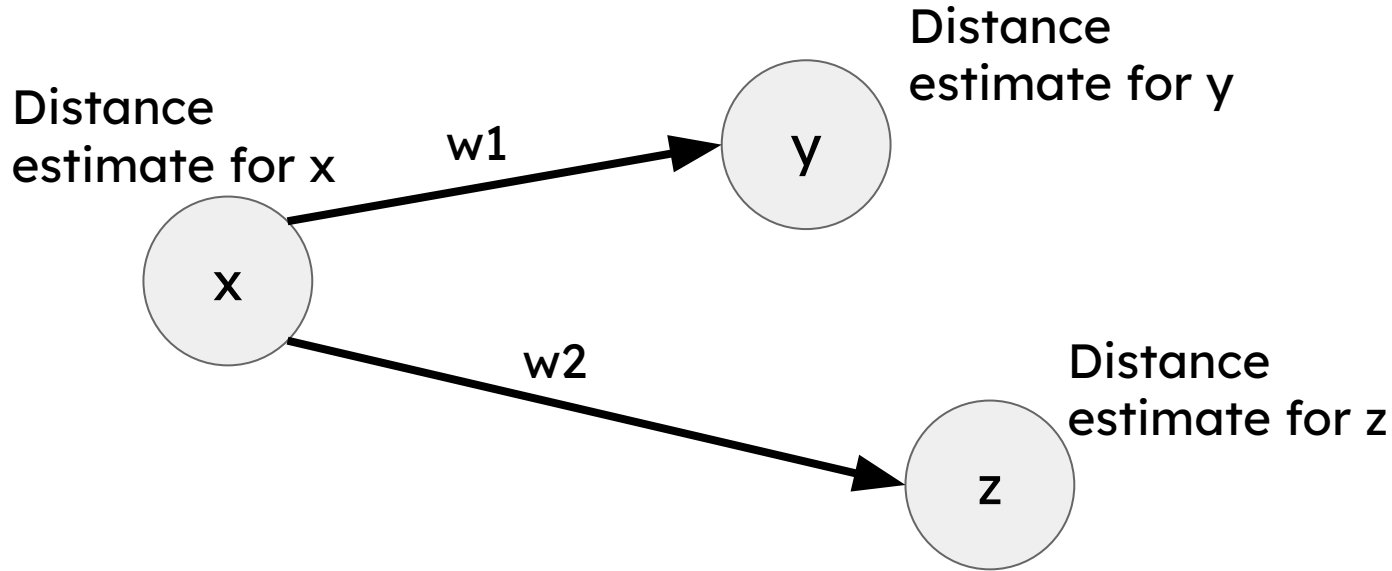
So before any iterations,

we know the source node is in $S(0)$ and we set its distance to 0.



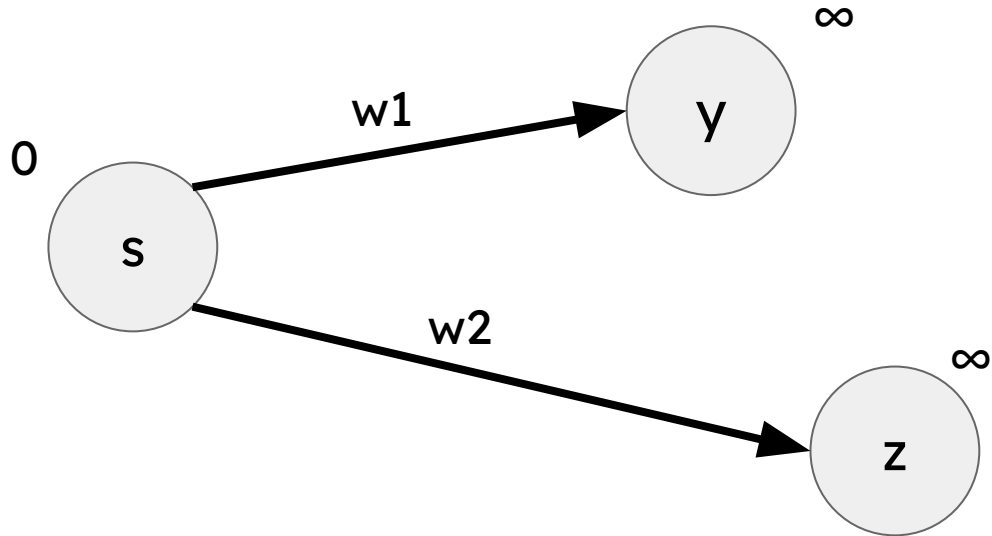
But how do we figure out $S(1)$?

Idea: Path Relaxation



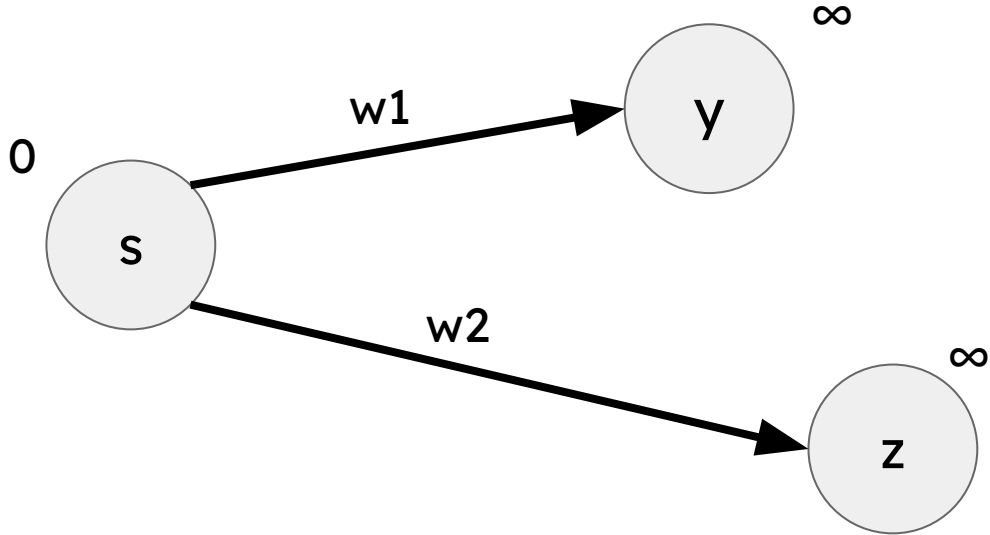
Let's say that for every node, we firstly maintain a distance estimate.

Idea: Path Relaxation



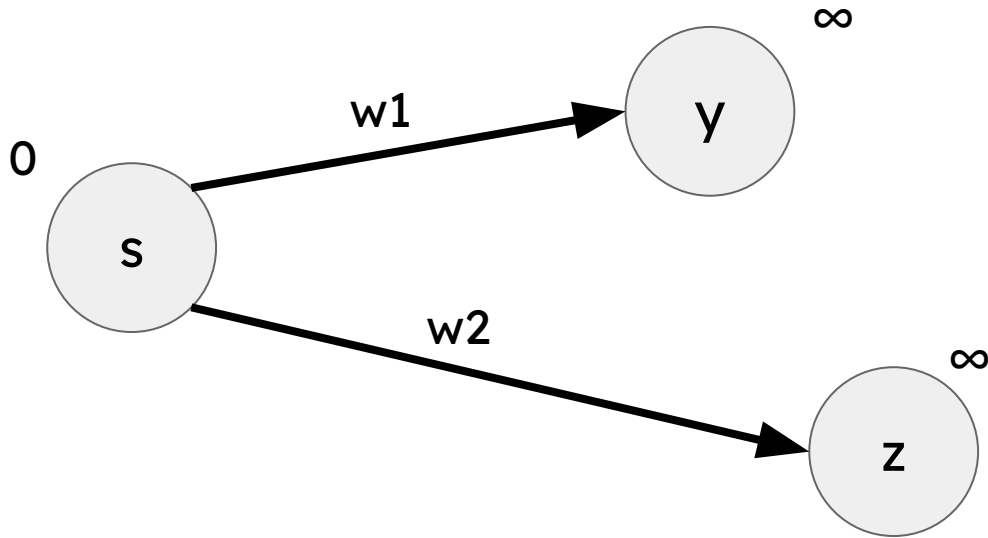
We'll start every distance estimate at ∞ , except for the source node, whose distance estimate is 0.

Idea: Path Relaxation



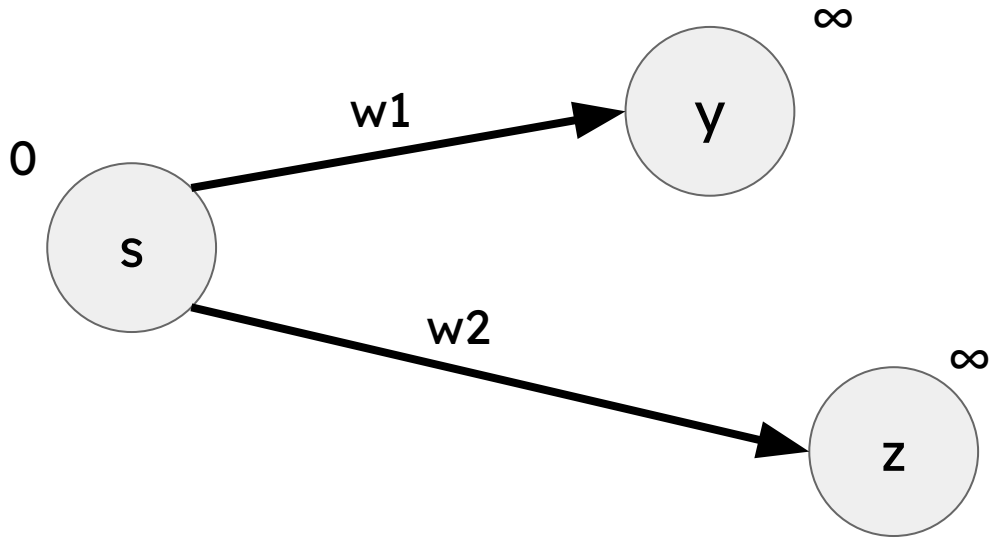
At every iteration, our distance estimate for a node v has to be $\geq$ the shortest possible distance to node v .

Idea: Path Relaxation



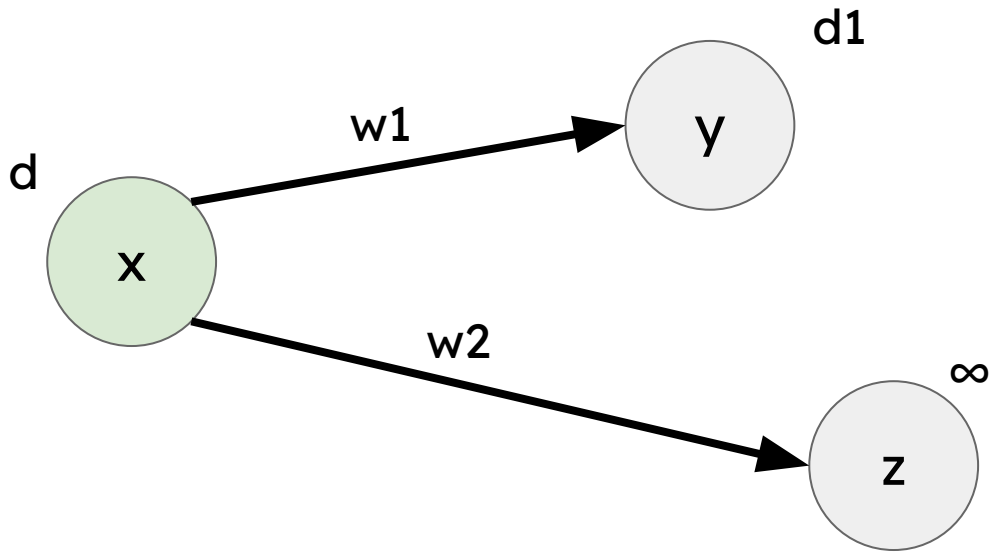
Intuitively, if we can somehow find a path from s to a node v with a cost/weight l , then we should compare it against its current estimate. If l is shorter than the current estimate, we should update our estimate to l .

Idea: Path Relaxation



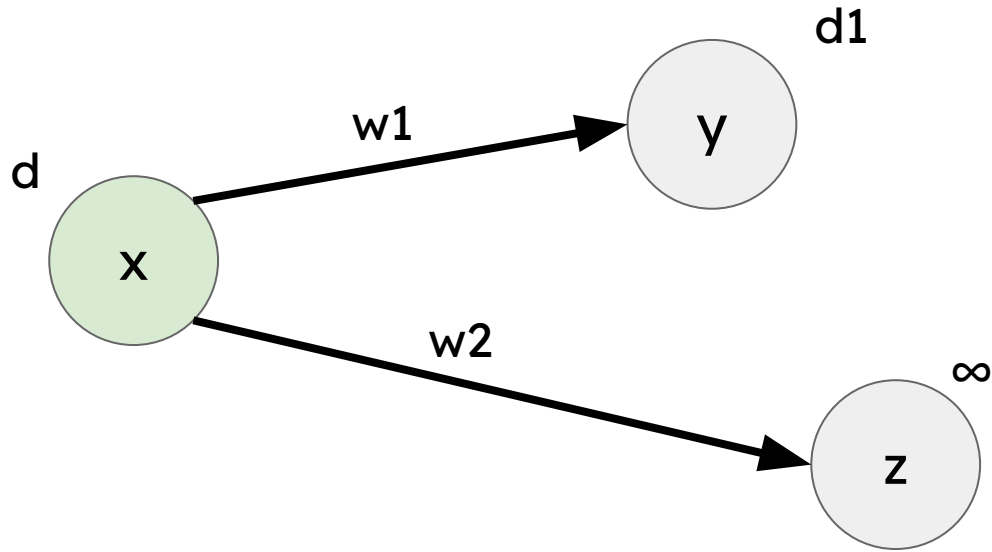
So the moment we've somehow discovered the shortest path, the estimate is set to the true shortest distance.

Idea: Path Relaxation

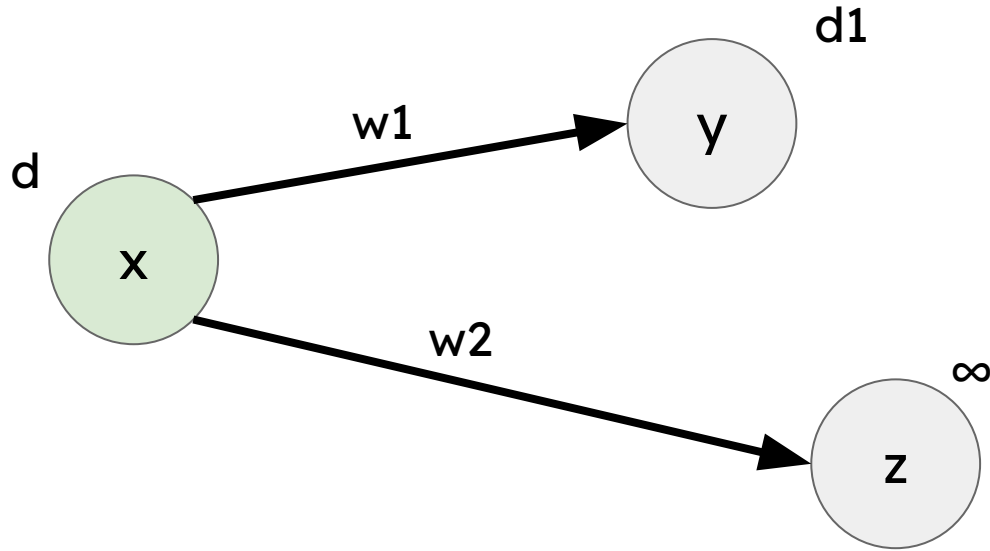


For example, let's say we're considering some node x , with distance estimate d . We know this means we've found some path with distance d to node x .

Idea: Path Relaxation

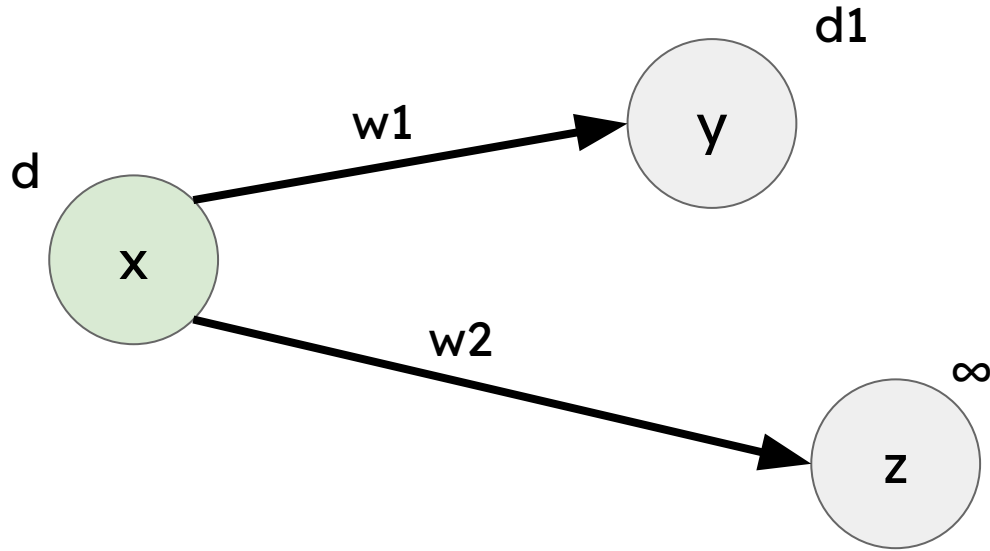


We should try to see if via node x we can somehow find a shorter distance to its neighbouring nodes y and z .



If $d + w1 \geq d1$, what should we set y 's distance estimate to?

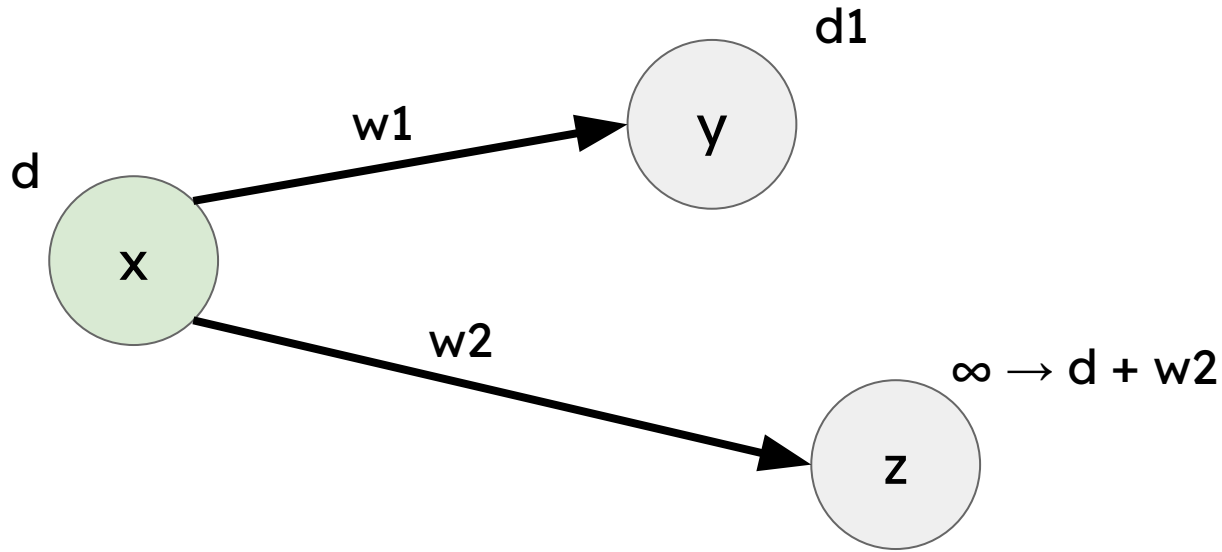
1. d
2. $d + w1$
3. $d1$
4. $w1$



If $d + w1 < d1$, what should we set y 's distance estimate to?

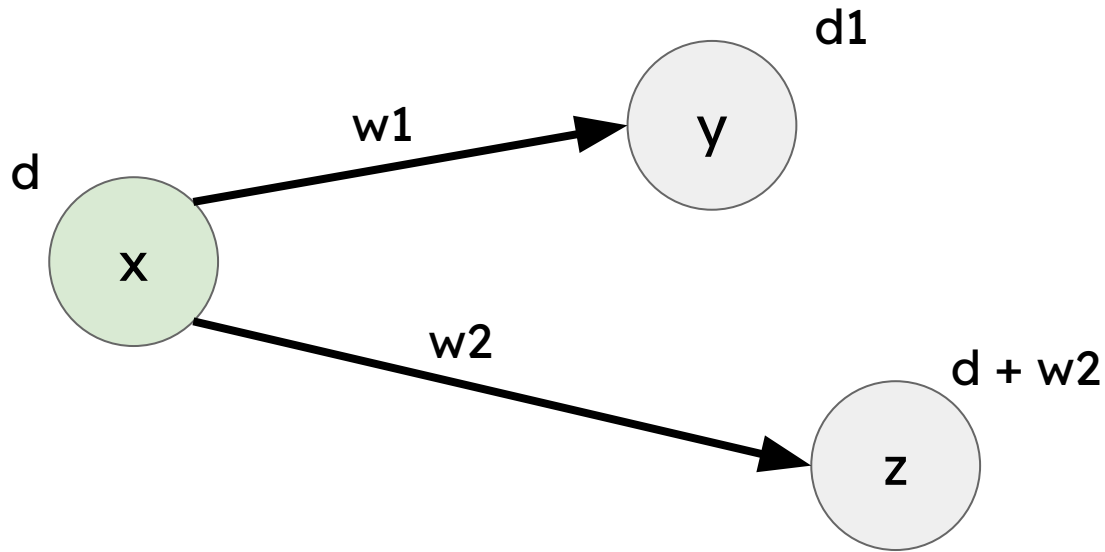
1. d
2. $d + w1$
3. $d1$
4. $w1$

Idea: Path Relaxation



On the other hand, anything is smaller than infinity, so if we look at neighbour z , we should just set it to $d + w2$.

Idea: Path Relaxation



In doing so, we've basically discovered a new path from s to z, via node x, that has distance $d + w2$.

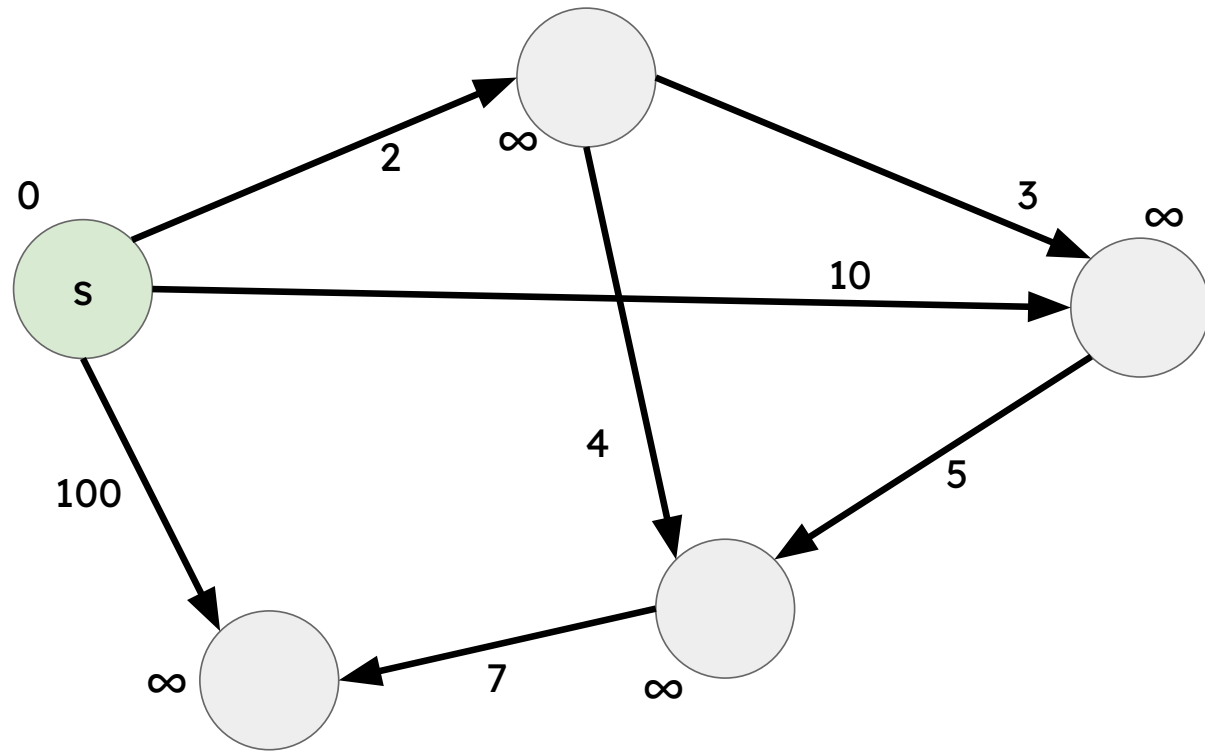
Idea: Path Relaxation

```
void relax(Graph g, int dist_est[], int u, int v):
```

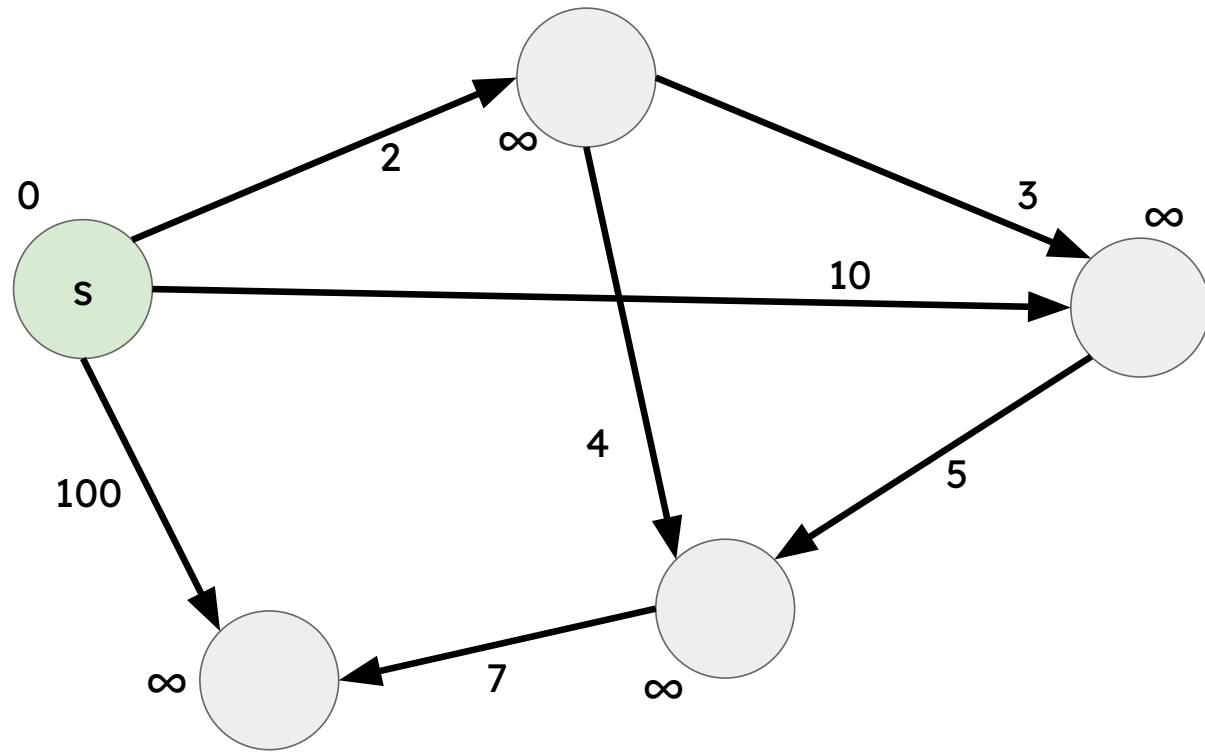
```
    if dist_est[u] + g.dist(u, v) < dist_est[v]:
```

```
        dist_est[v] = dist_est[u] + g.dist(u, v)
```

Initially, all distance estimates are ∞ (except for our source node, with estimate 0).



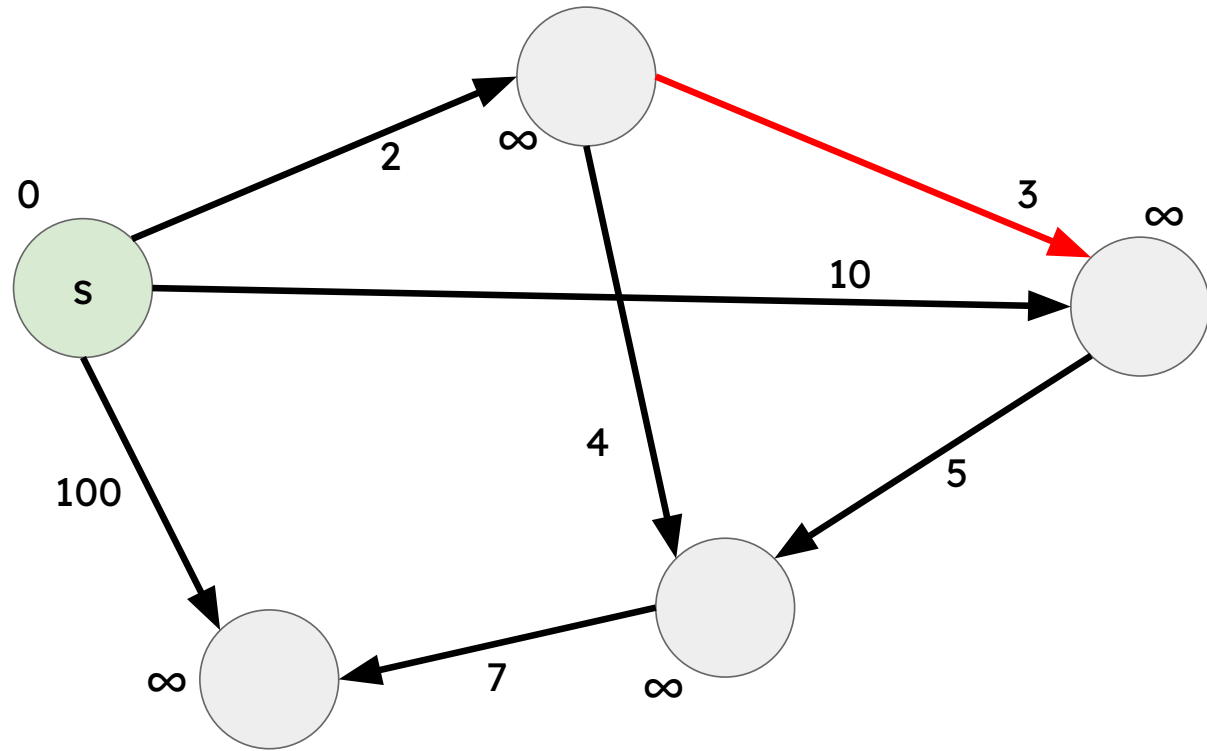
Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .



for $e = (u, v)$ in edges:

`relax(G, dist_est, u, v)`

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

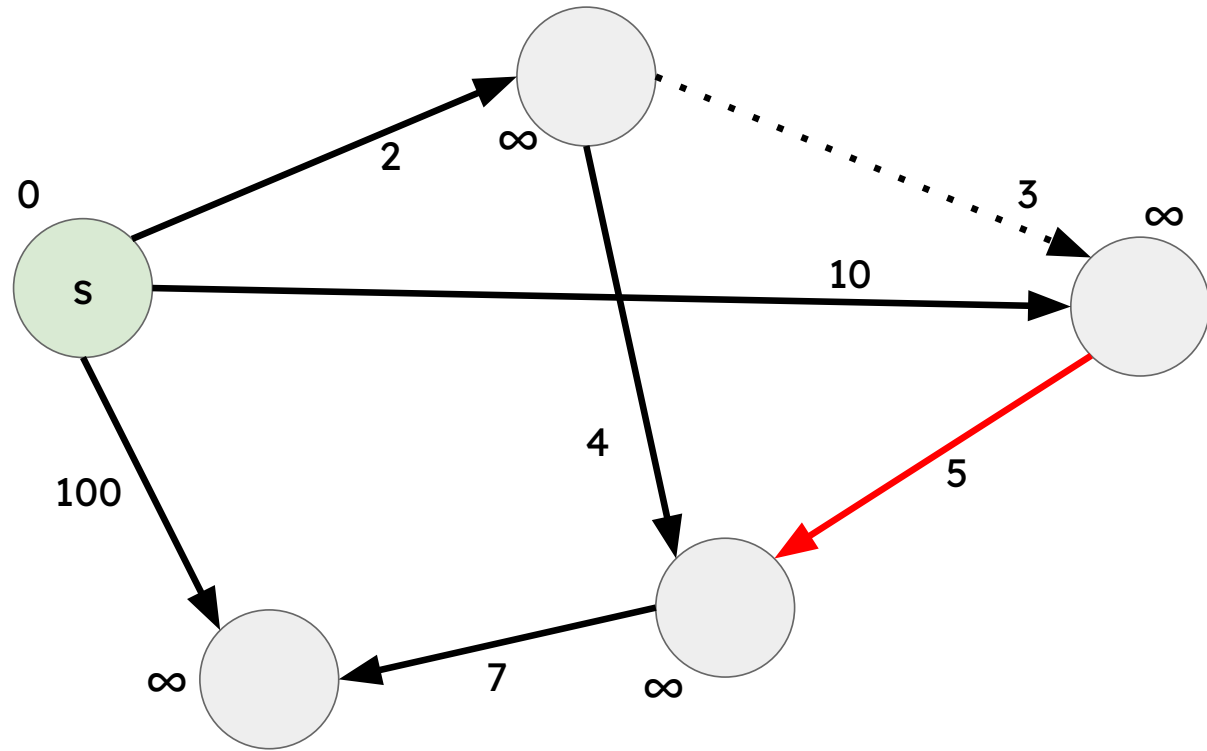


for $e = (u, v)$ in edges:

`relax(G, dist_est, u, v)`

No change to the estimates

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

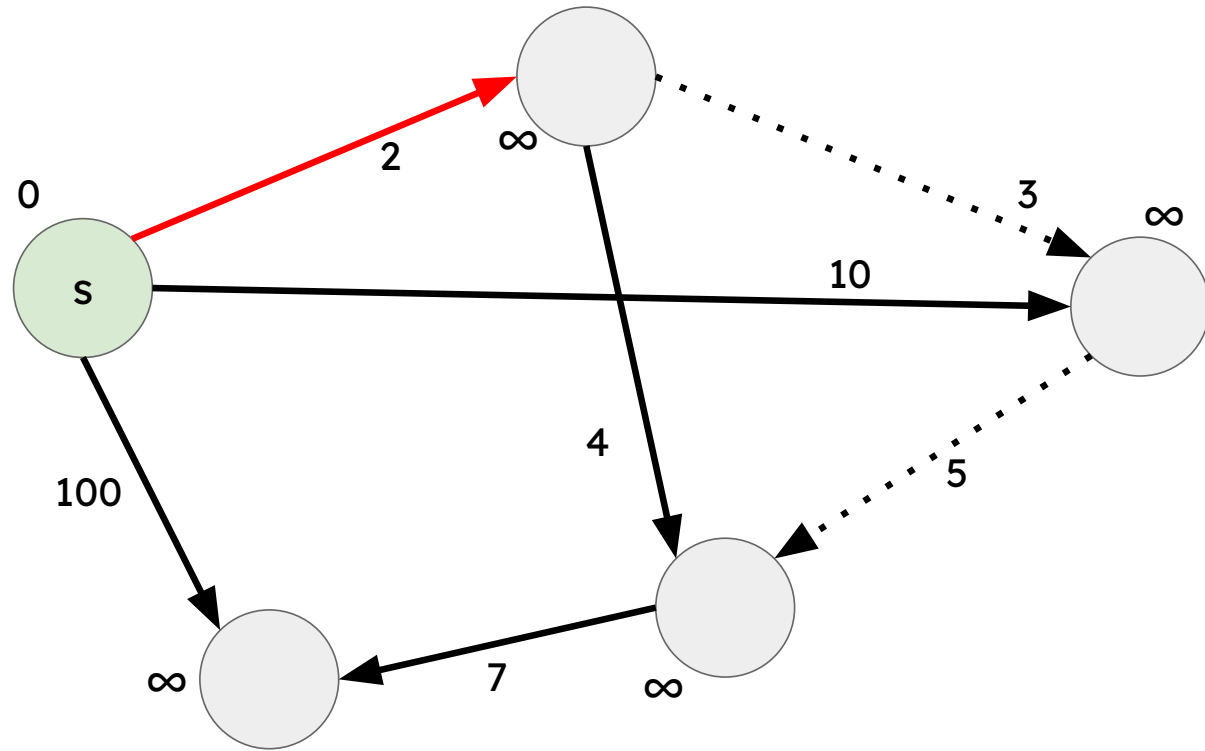


for $e = (u, v)$ in edges:

`relax(G, dist_est, u, v)`

No change to the estimates

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

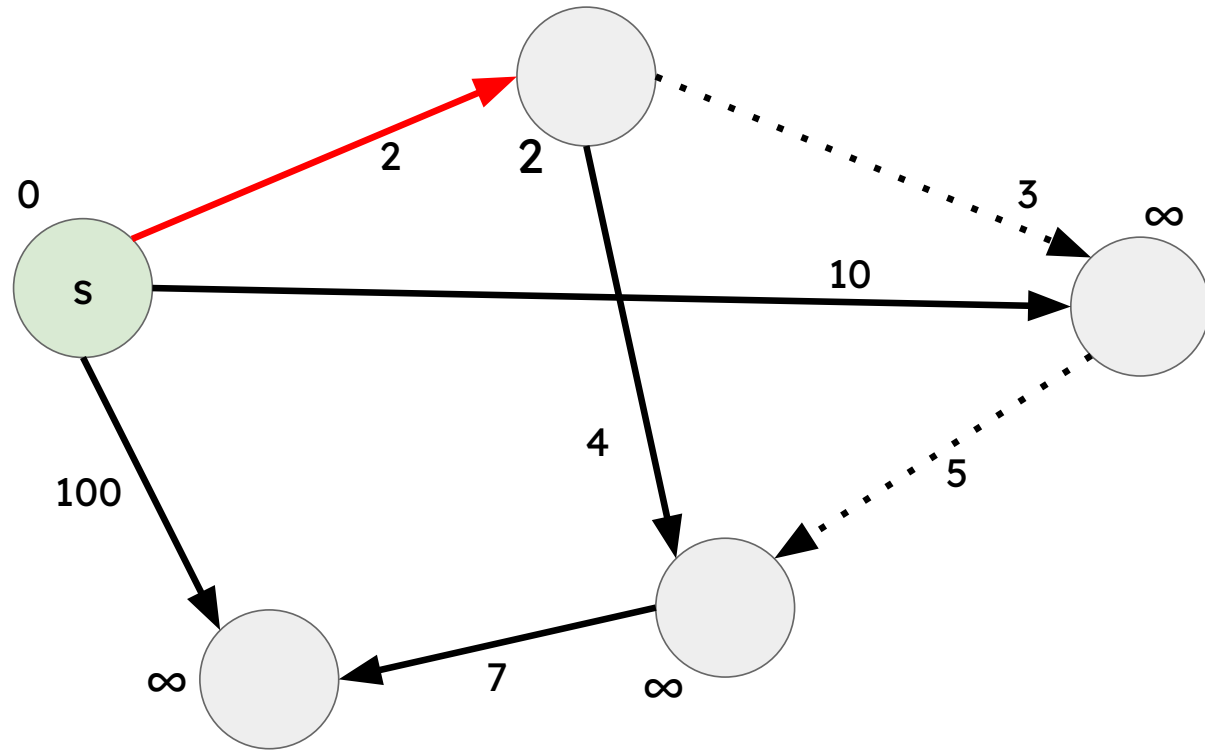


for $e = (u, v)$ in edges:

$\text{relax}(G, \text{dist_est}, u, v)$

$0 + 2 < \infty$
So update the estimate to 2

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

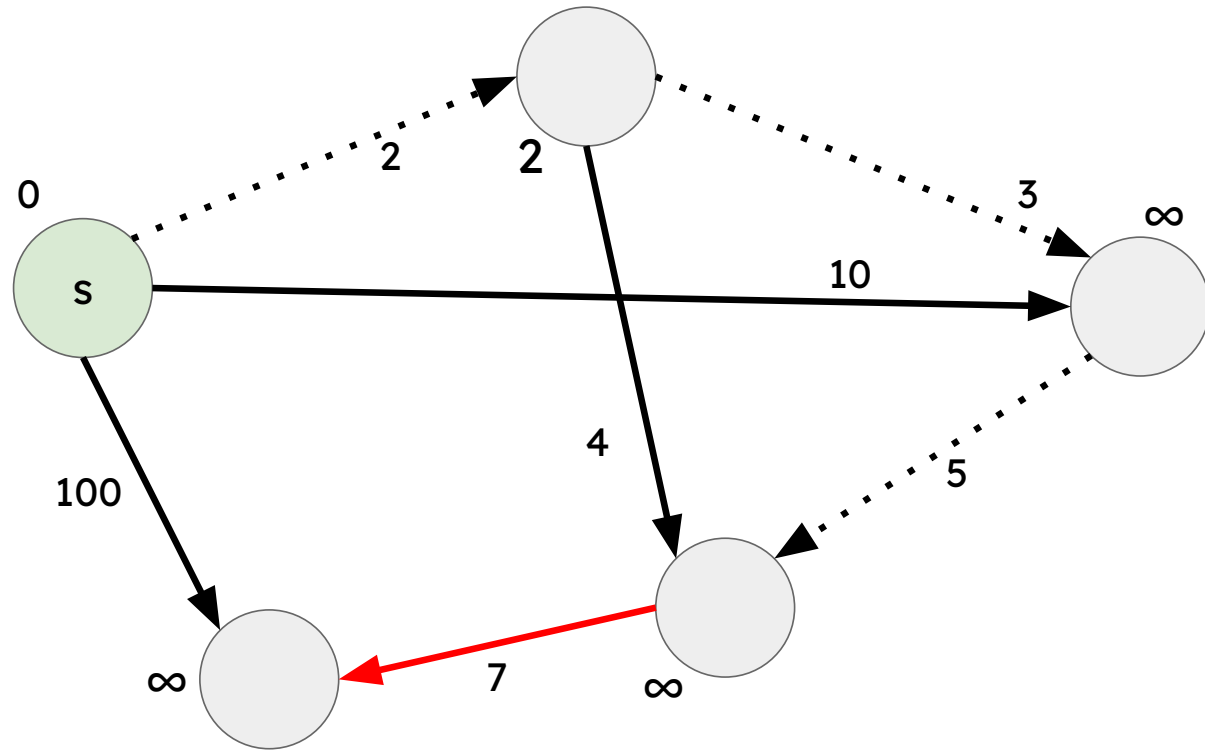


for $e = (u, v)$ in edges:

$\text{relax}(G, \text{dist_est}, u, v)$

$0 + 2 < \infty$
So update the estimate to 2

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

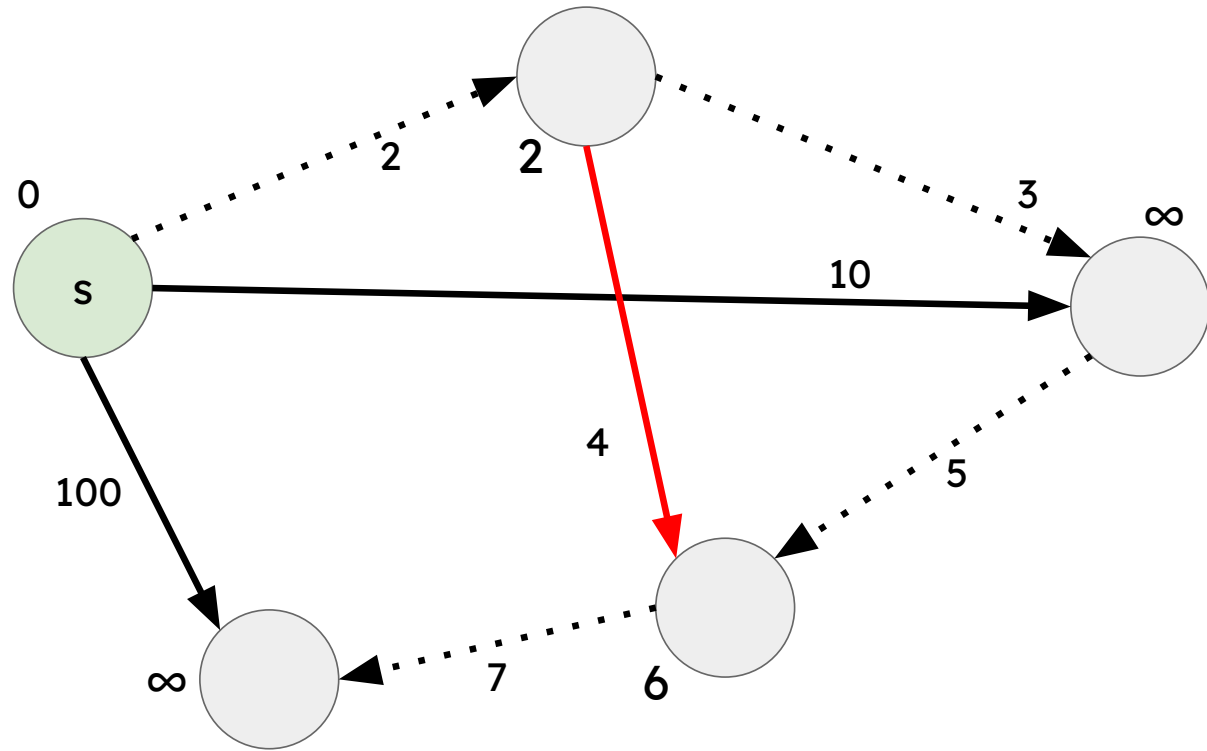


for $e = (u, v)$ in edges:

`relax(G, dist_est, u, v)`

Again, no change.

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

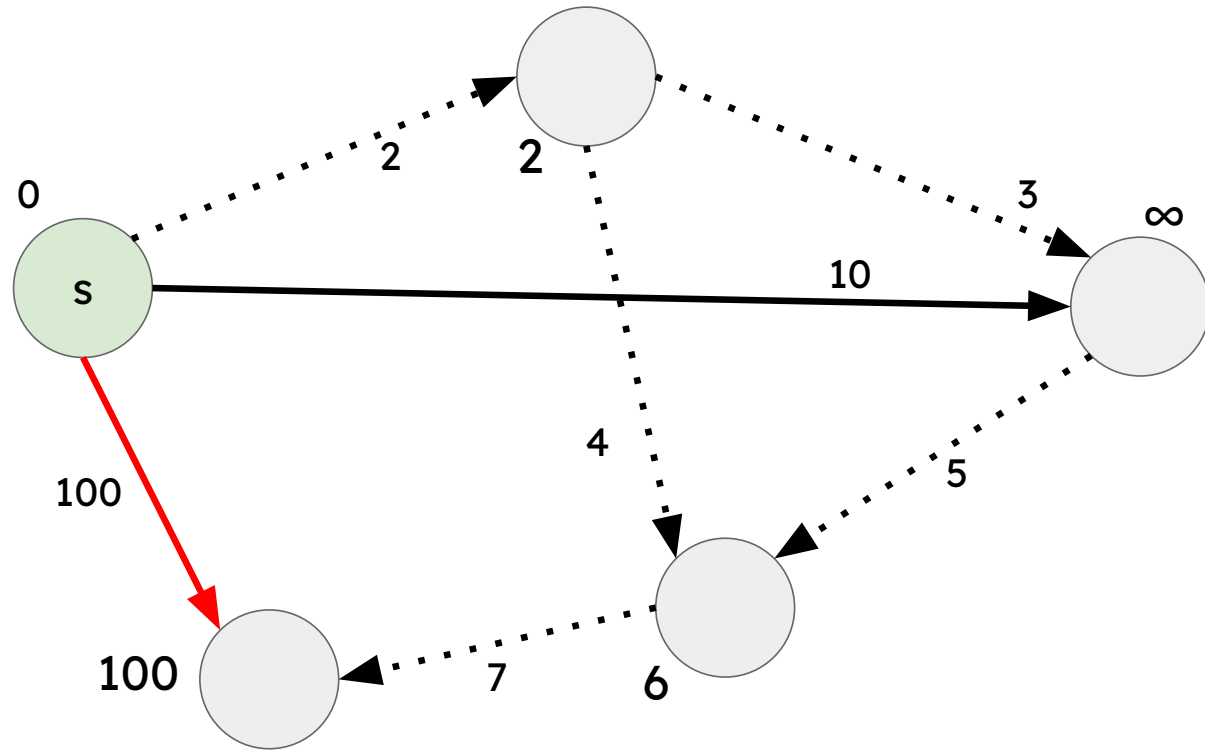


for $e = (u, v)$ in edges:

$\text{relax}(G, \text{dist_est}, u, v)$

$2 + 4 < \infty$
So we update this to 6

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

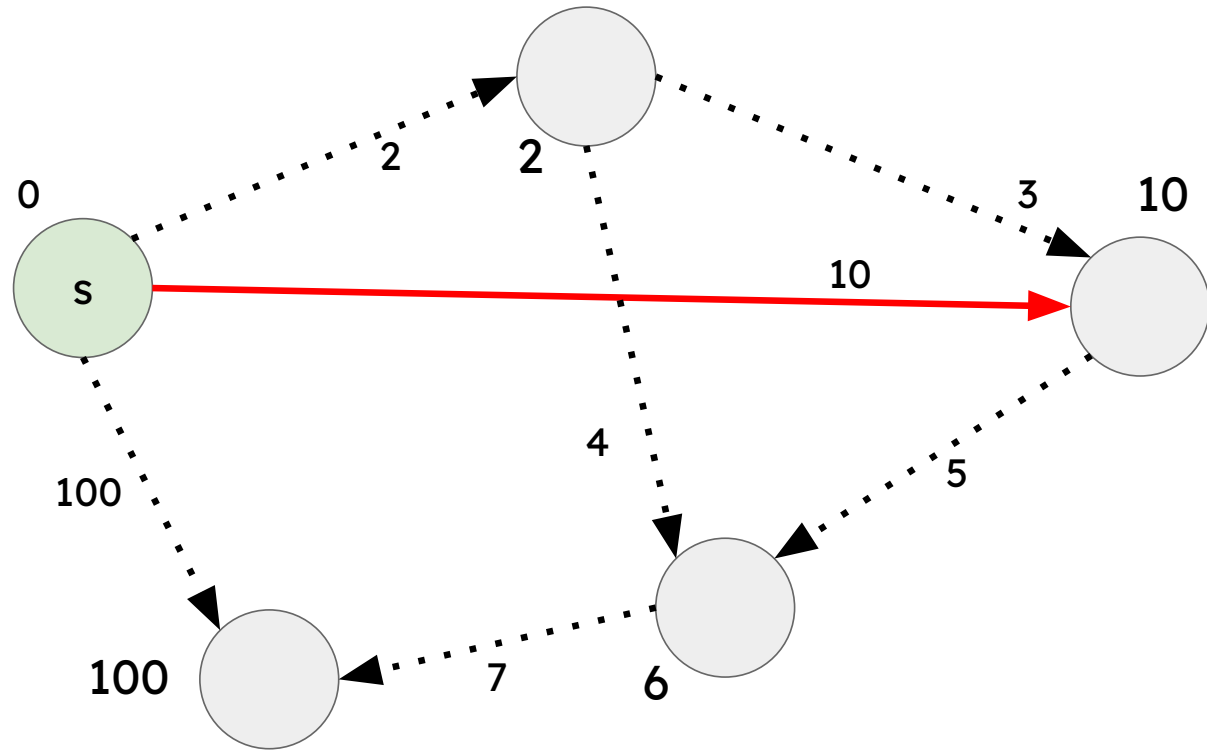


for $e = (u, v)$ in edges:

$\text{relax}(G, \text{dist_est}, u, v)$

$0 + 100 < \infty$
So we update this to 100

Let's say that we went through every edge $e = (u, v)$, and tried to relax from node u to node v .

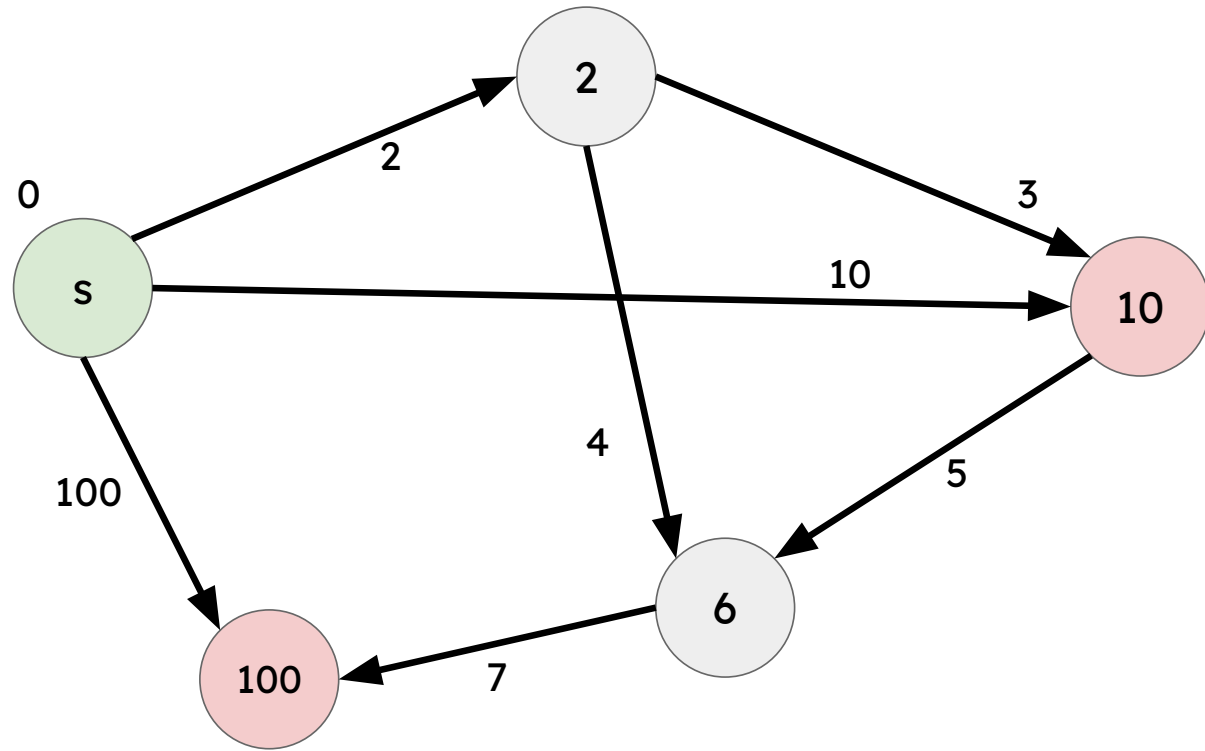


for $e = (u, v)$ in edges:

$\text{relax}(G, \text{dist_est}, u, v)$

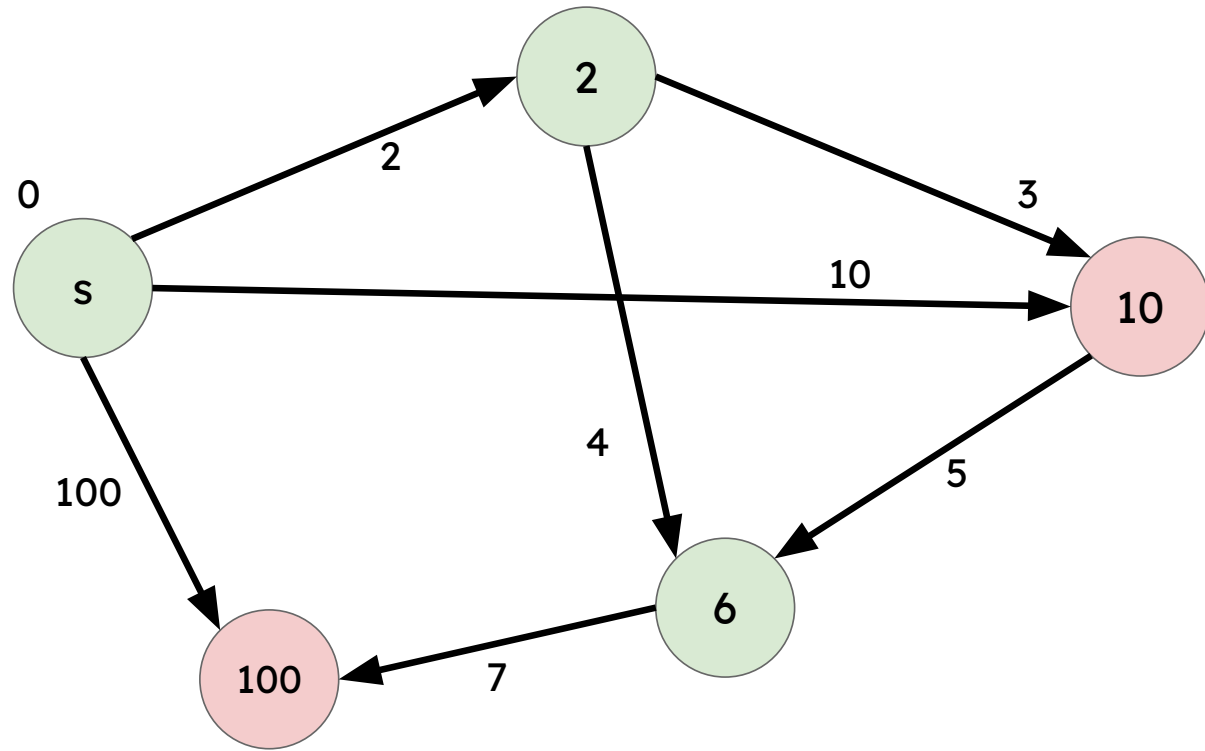
$0 + 10 < \infty$
So we update this to 10

So we've done a single round of relax on edges.



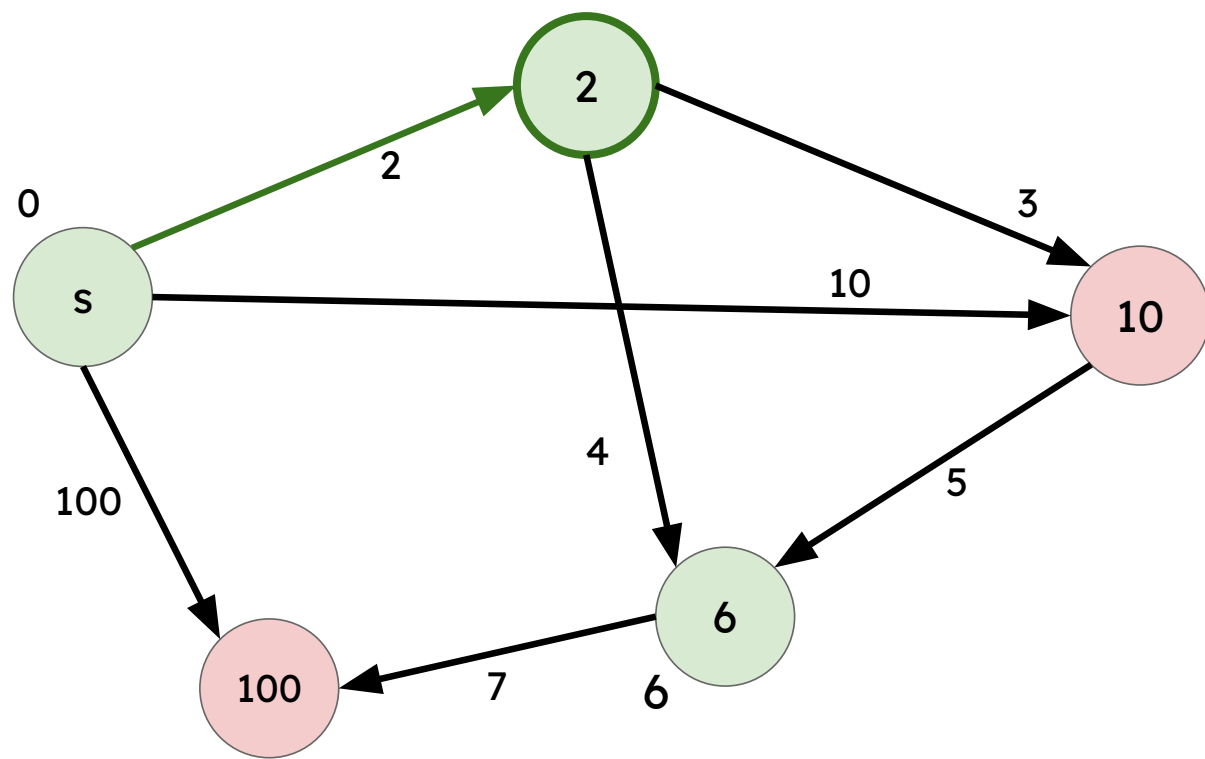
You might want to notice that **not** every node has their distance estimates = their shortest path distance yet.

So we've done a single round of relax on edges.



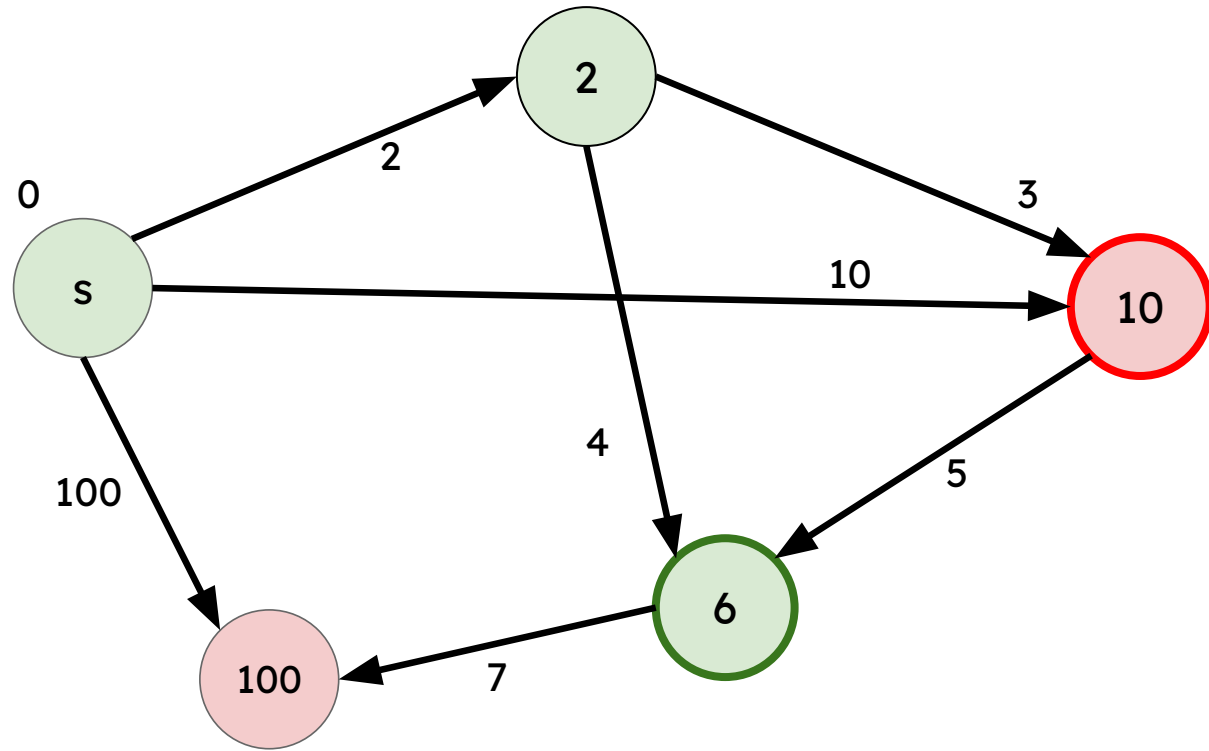
Whereas some nodes already have the correct shortest distances computed.

So we've done a single round of relax on edges.



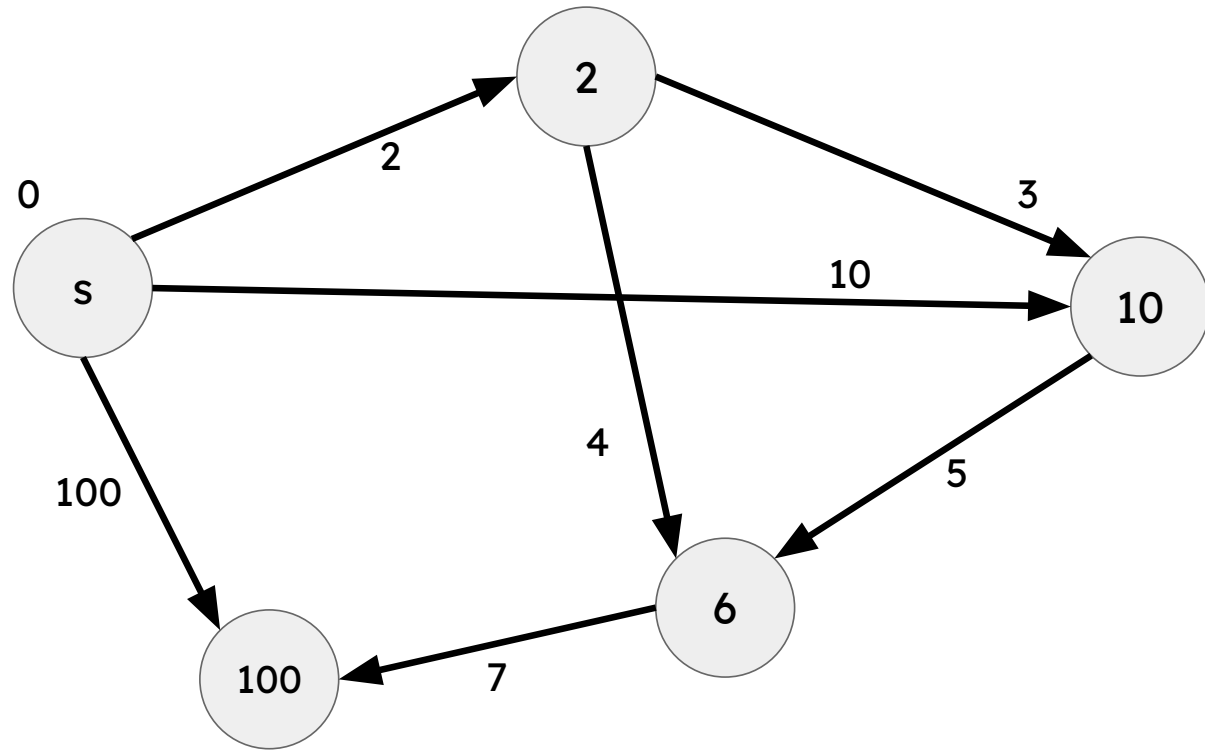
Importantly, notice that for any node whose shortest path uses a single edge, it's guaranteed we have found its shortest distance.

So we've done a single round of relax on edges.



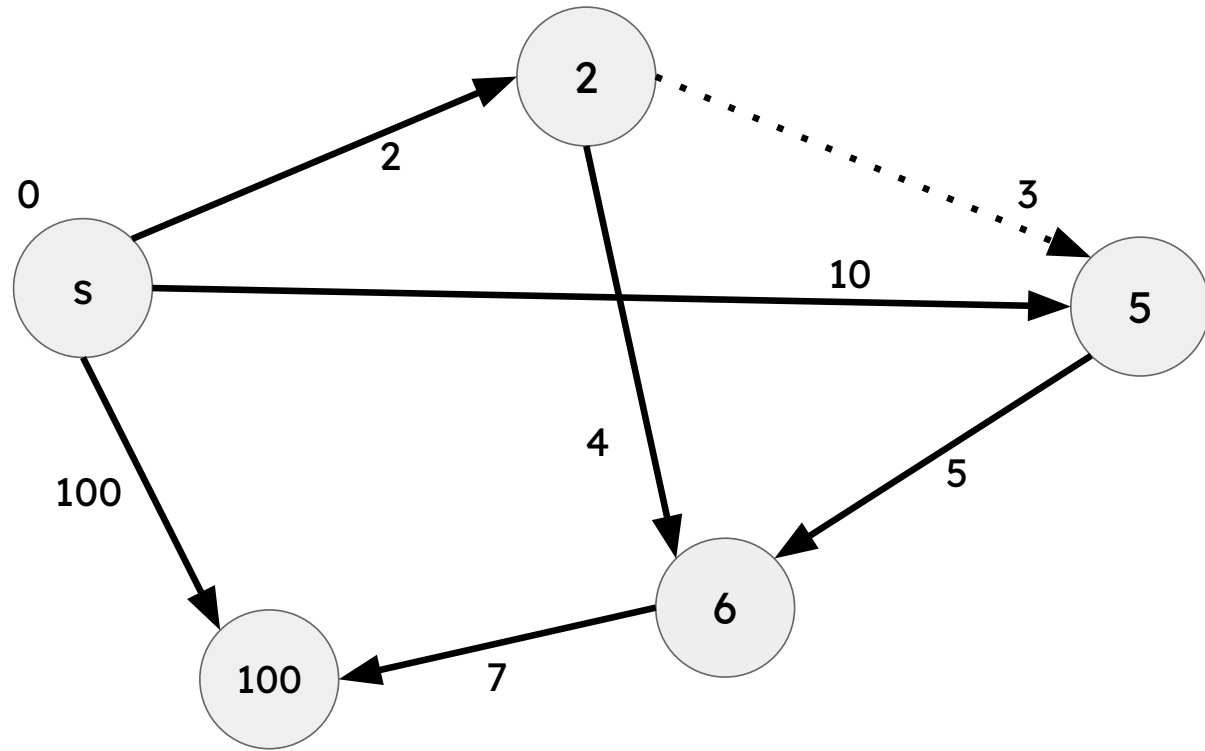
On the other hand, for other nodes whose shortest path uses 2 edges or more, we don't have a guarantee on whether its shortest distance has been found yet or not.

Basically, we've computed $S(1)$ using $S(0)$.



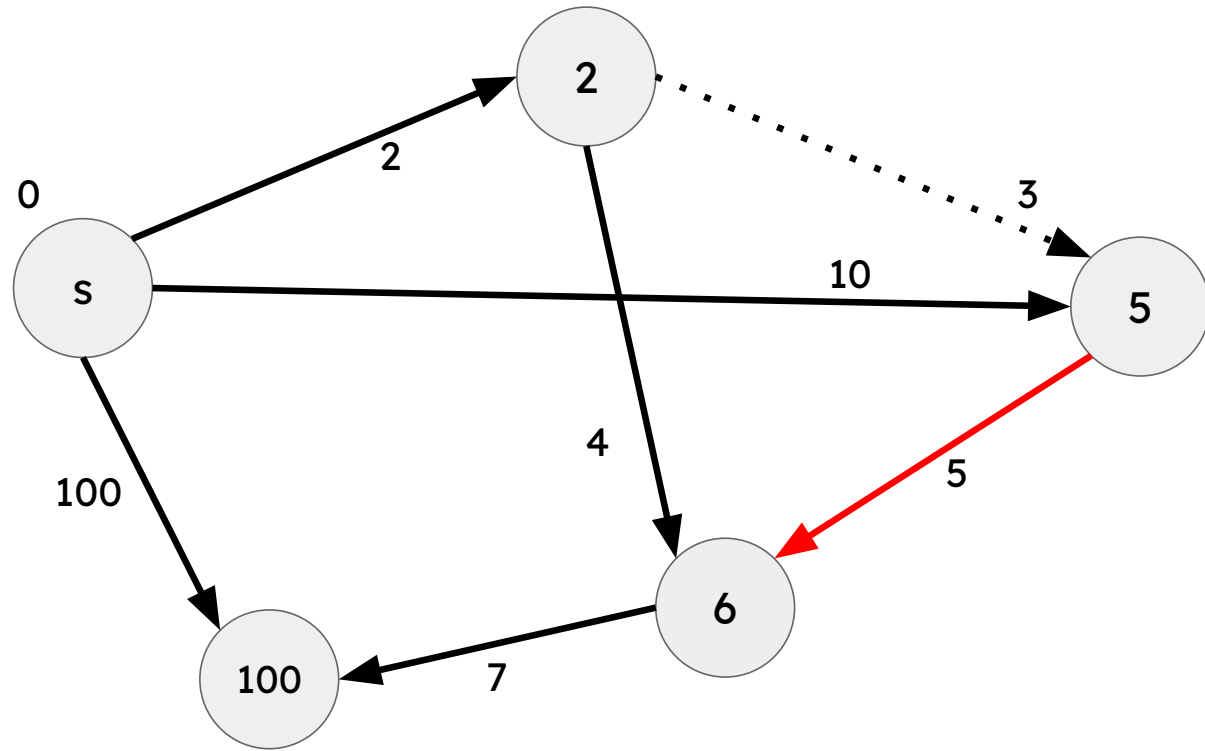
So if we repeated this round of edge relaxations again, we'd compute the shortest distances for nodes in $S(2)$ using $S(1)$.

So after one more round of edge relaxations, we might get something like:



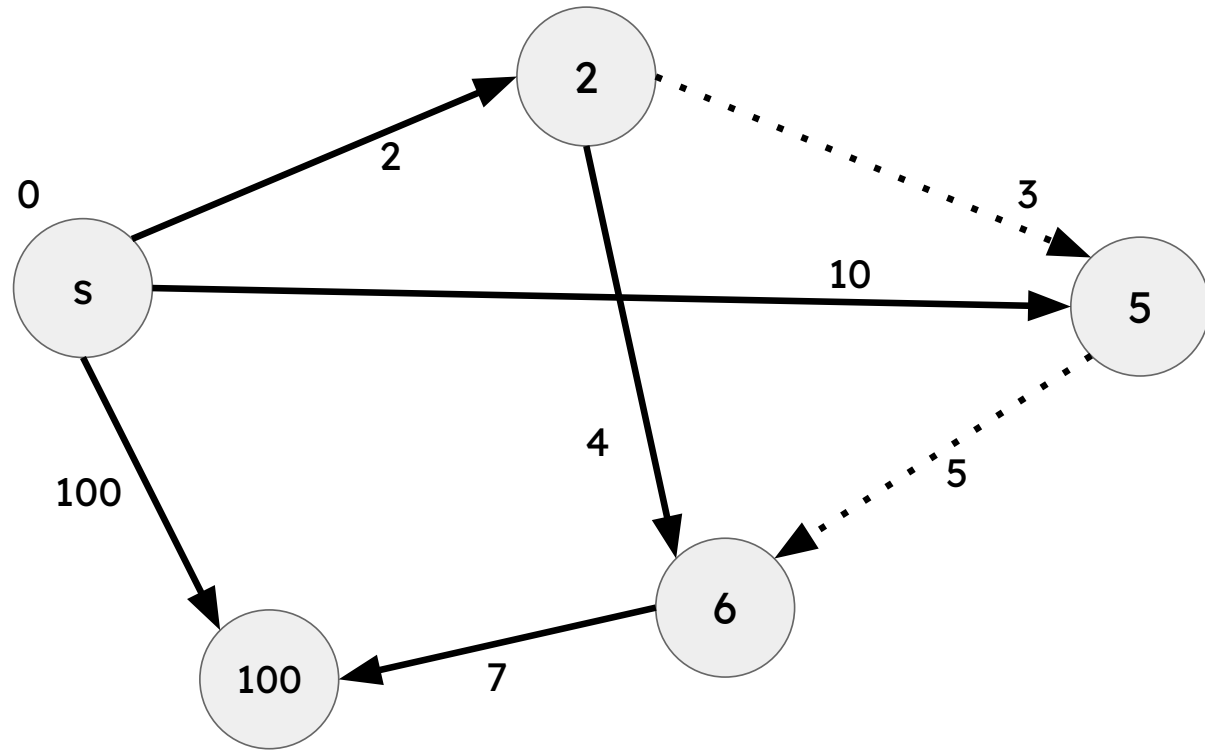
Again, since $2 + 3 < 10$, we can update the distance estimate from 10 to $2 + 3 = 5$.

So after one more round of edge relaxations, we might get something like:



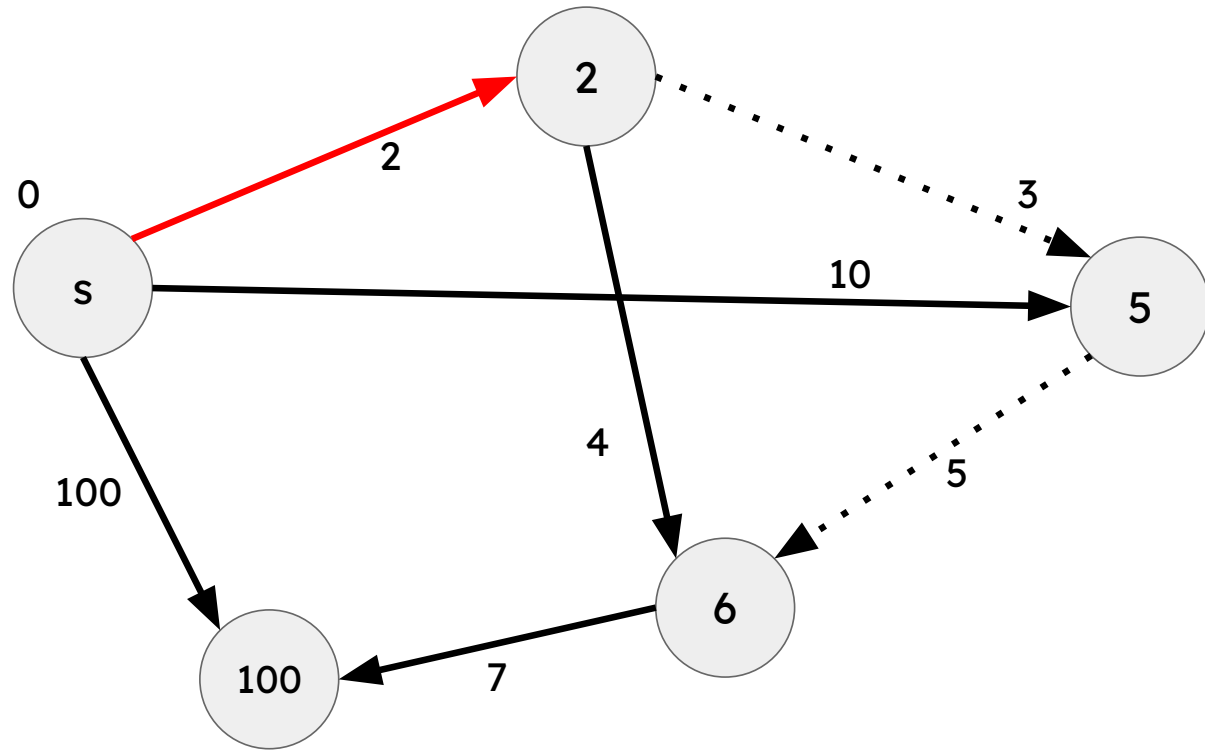
$5 + 5 > 6$ so we don't update this estimate.

So after one more round of edge relaxations, we might get something like:



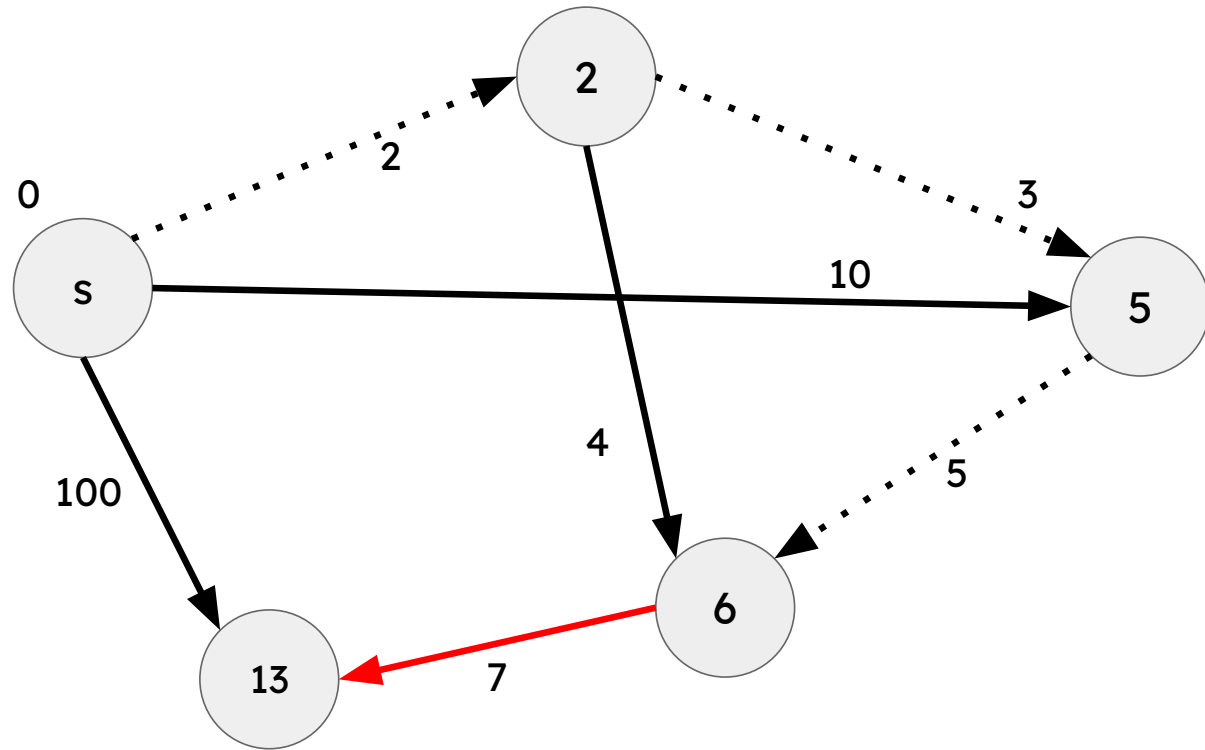
$5 + 5 > 6$ so we don't update this estimate.

So after one more round of edge relaxations, we might get something like:



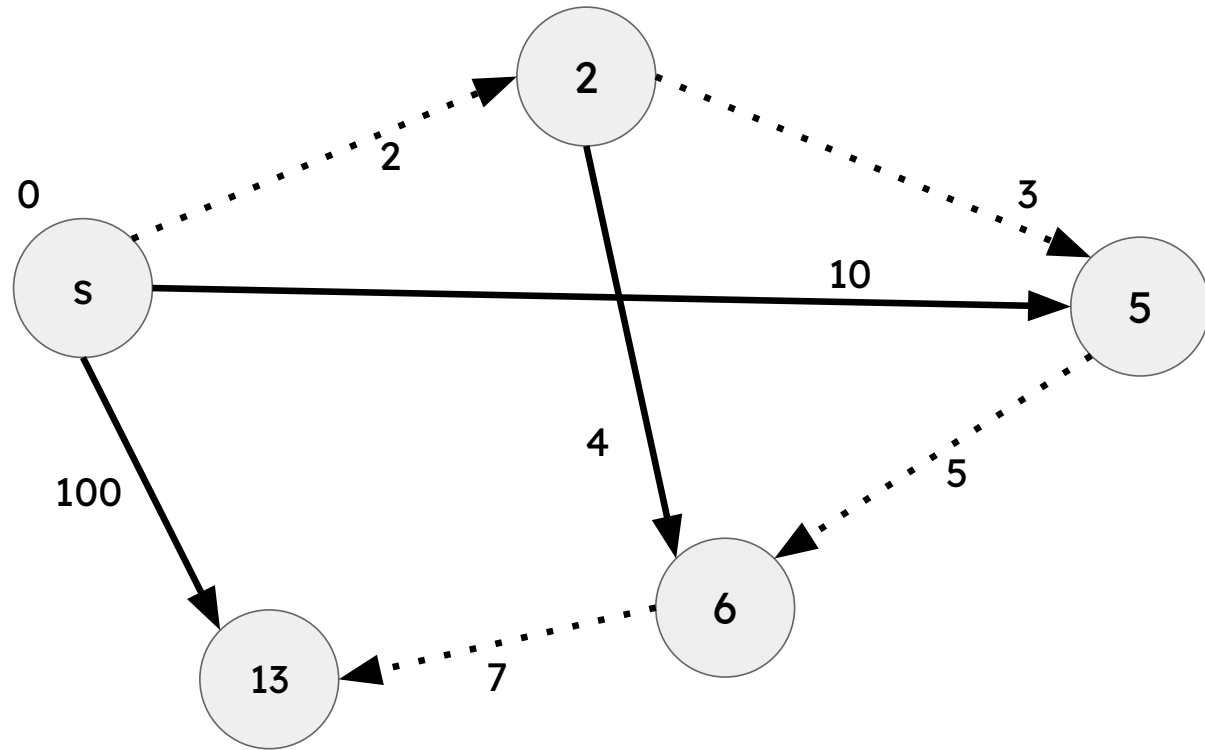
$0 + 2$ is not < 2 , so we keep this the same.

So after one more round of edge relaxations, we might get something like:



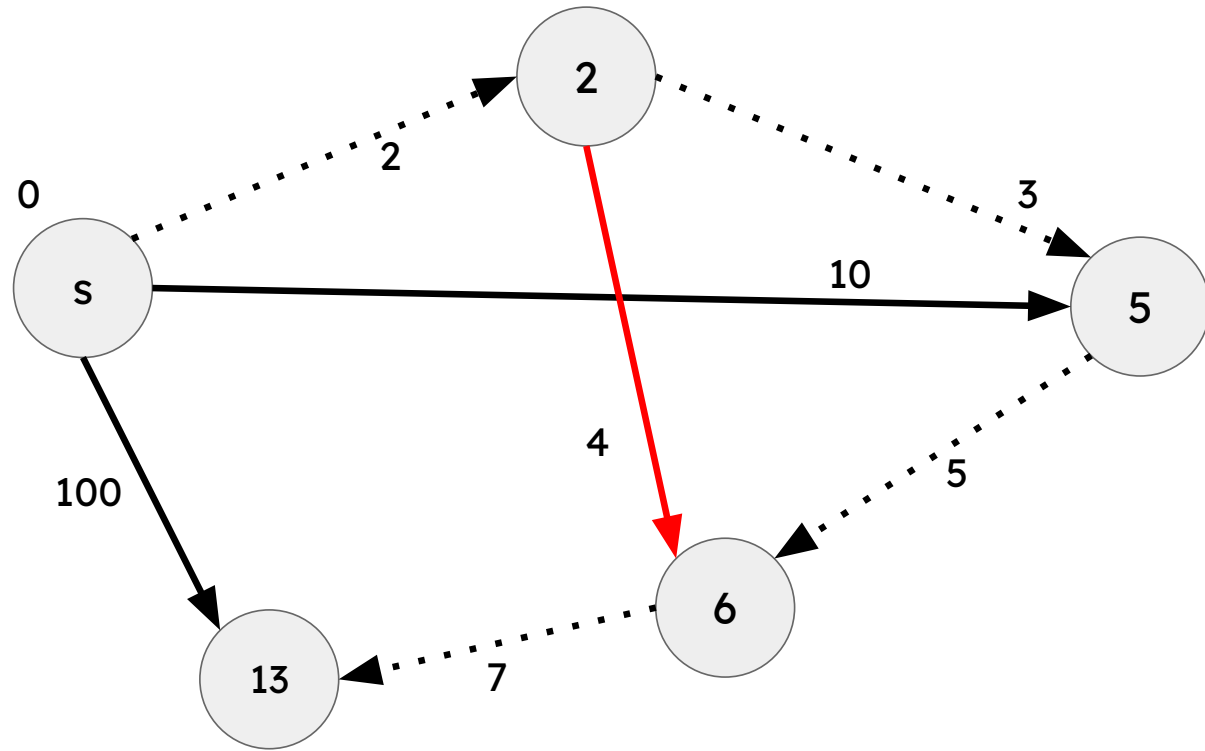
$6 + 7 = 13 < 100$, so we update its distance estimate.

So after one more round of edge relaxations, we might get something like:



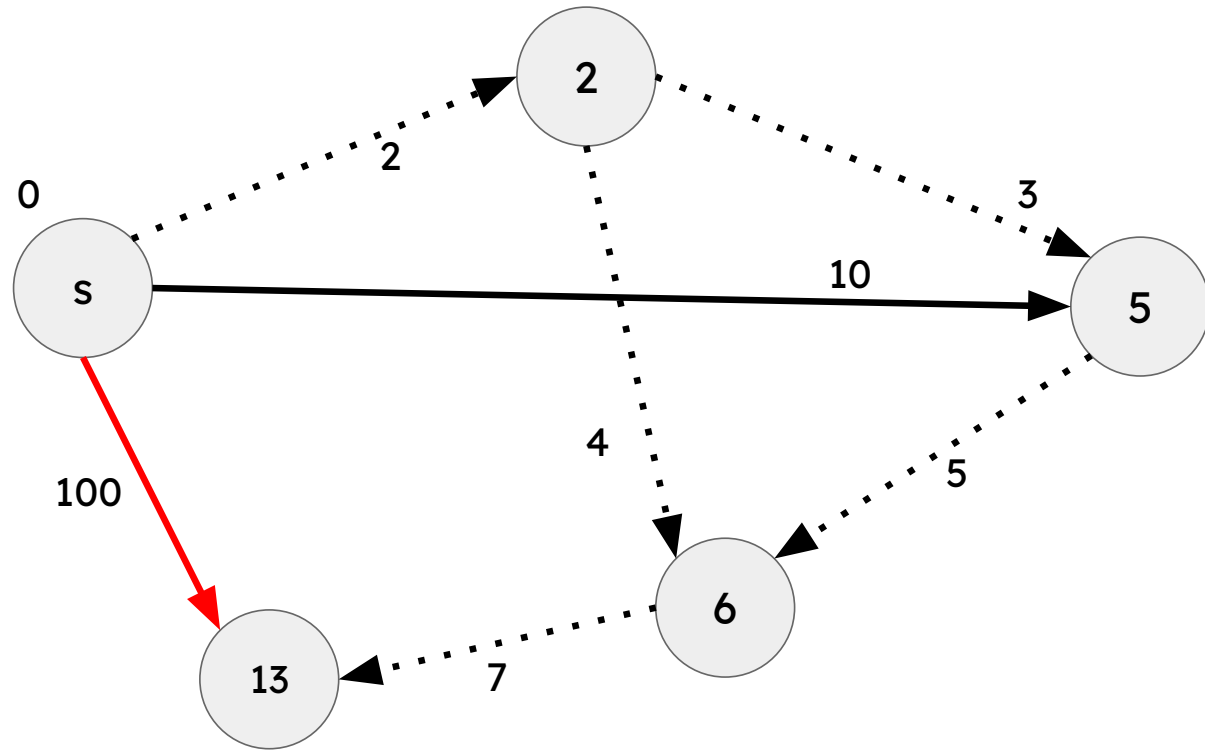
$6 + 7 = 13 < 100$, so we update its distance estimate.

So after one more round of edge relaxations, we might get something like:



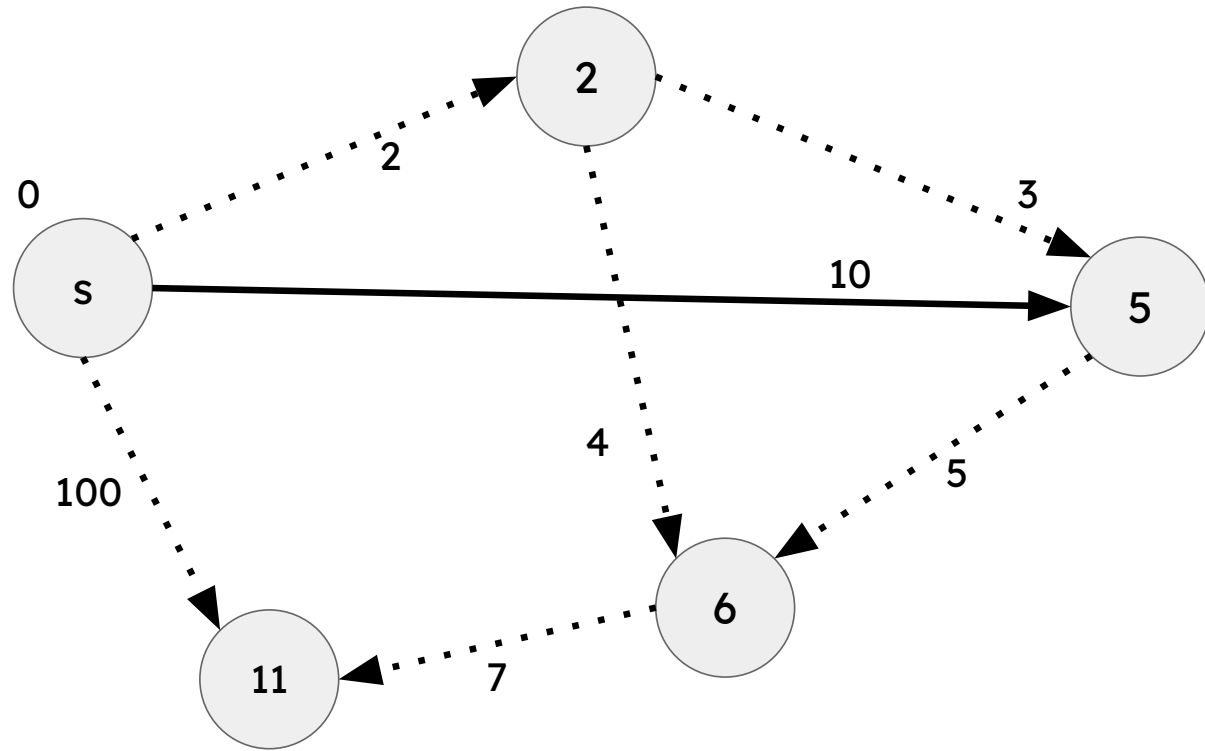
Since $2 + 4$ is not < 6 , we can leave this distance estimate unchanged.

So after one more round of edge relaxations, we might get something like:



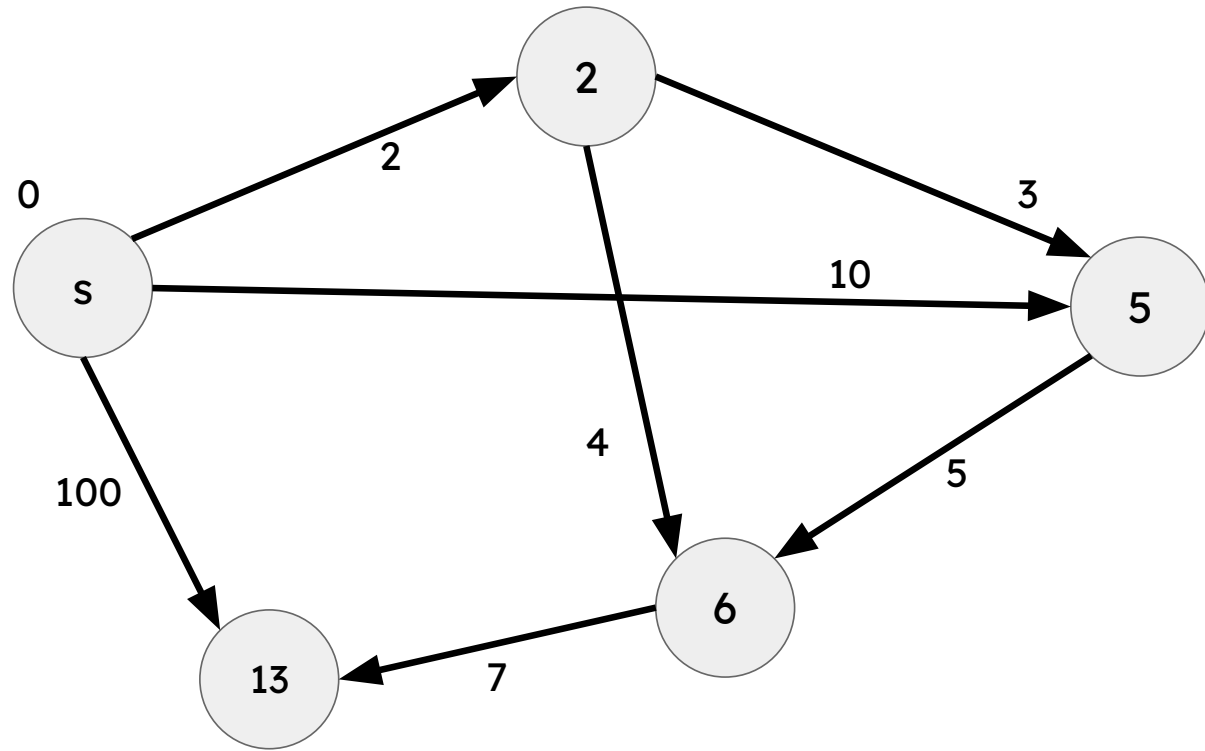
Since $0 + 13$ is not < 13 , we leave this distance estimate unchanged.

So after one more round of edge relaxations, we might get something like:



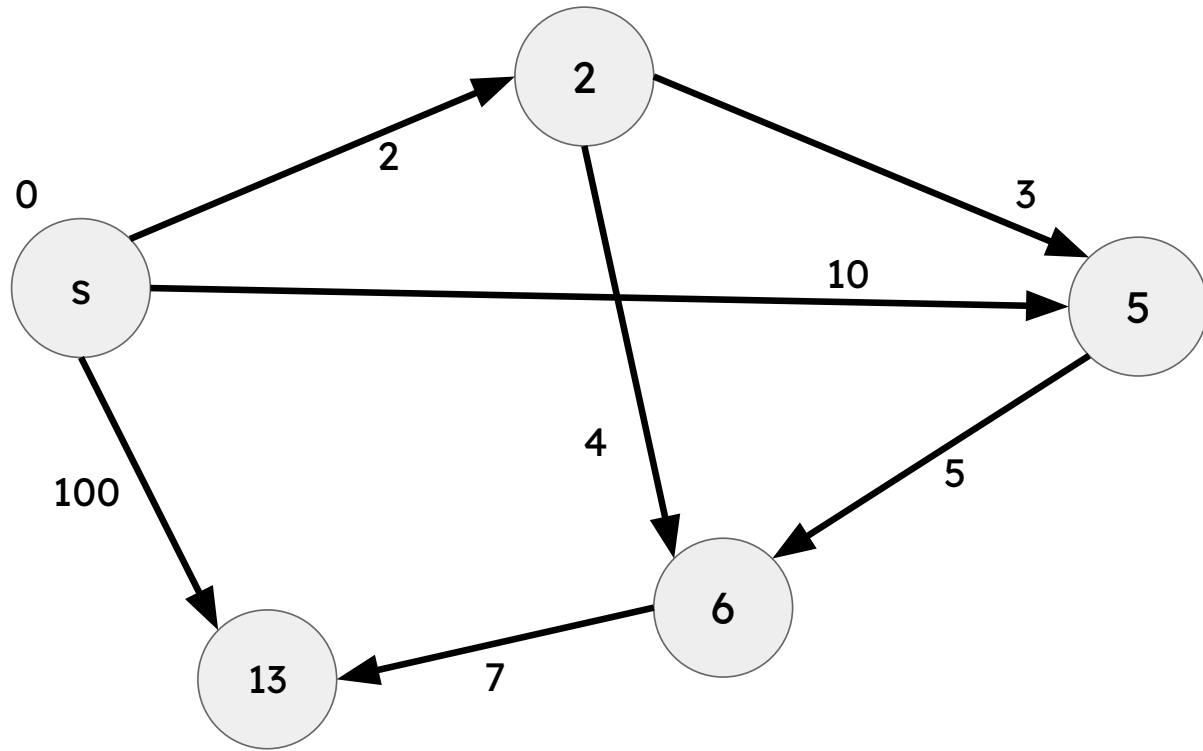
Since $0 + 13$ is not < 13 , we leave this distance estimate unchanged.

After the second round of updates, notice that some distance estimates didn't change, whereas some got reduced.



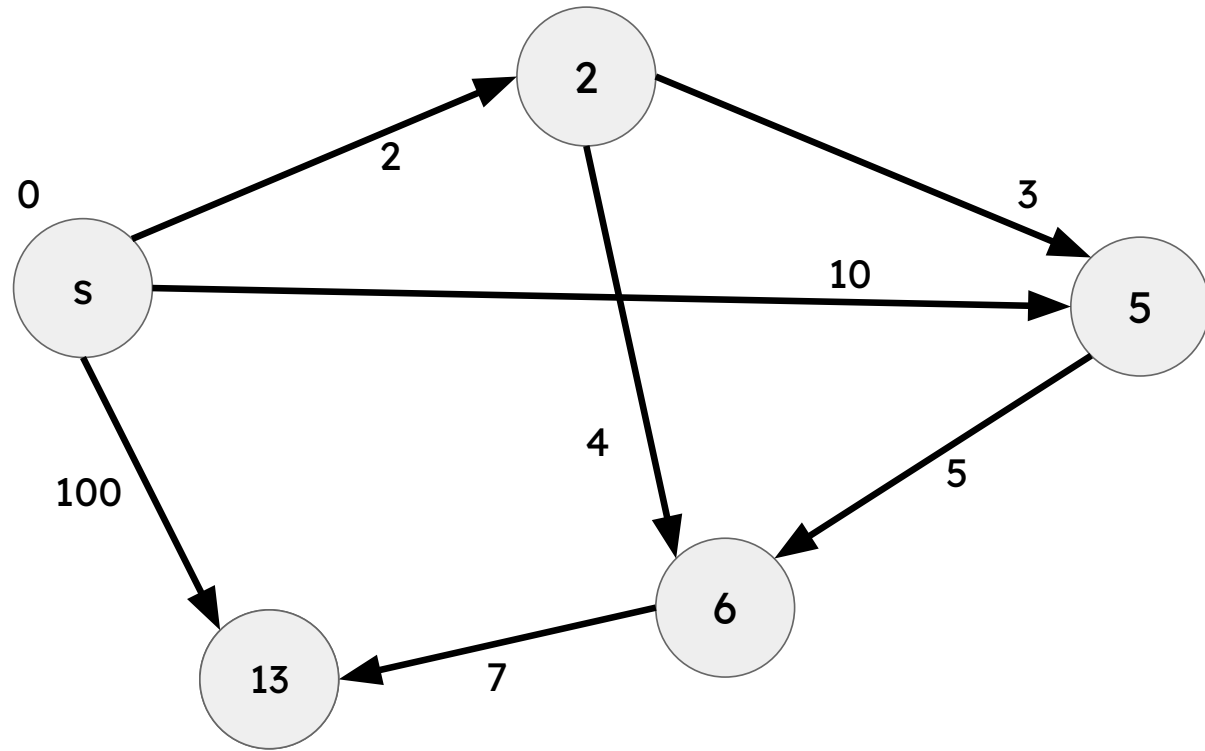
In fact, we're now guaranteed that any nodes whose shortest path uses either 0, 1, or 2 edges are such that their distance estimates are the shortest path distances.

You might notice that if we ran this for another 2 rounds, the distance estimates won't change again.



That's because all of the estimates are actually the same as the shortest path distance now.

But in general, since the shortest path might use all $n - 1$ edges, we might need to run this for $n - 1$ rounds.



Bellman Ford:

bellman-ford(**G**, **s**):

dist_est = [an array of all ∞]; set **dist_est**[**s**] = 0

for n - 1 iterations:

for all edges $e = (u, v)$ in edge_list:

relax(**G**, **dist_est**, **u**, **v**)

Bellman Ford:

bellman-ford(G , s):

dist_est = [an array of all ∞]; set $\text{dist_est}[s] = 0$

for $n - 1$ iterations:

for all edges $e = (u, v)$ in edge_list :

relax(G , dist_est , u , v)

What's the runtime?

1. $O(n)$
2. $O(n + m)$
3. $O(m)$
4. $O(nm)$

Summary so far:

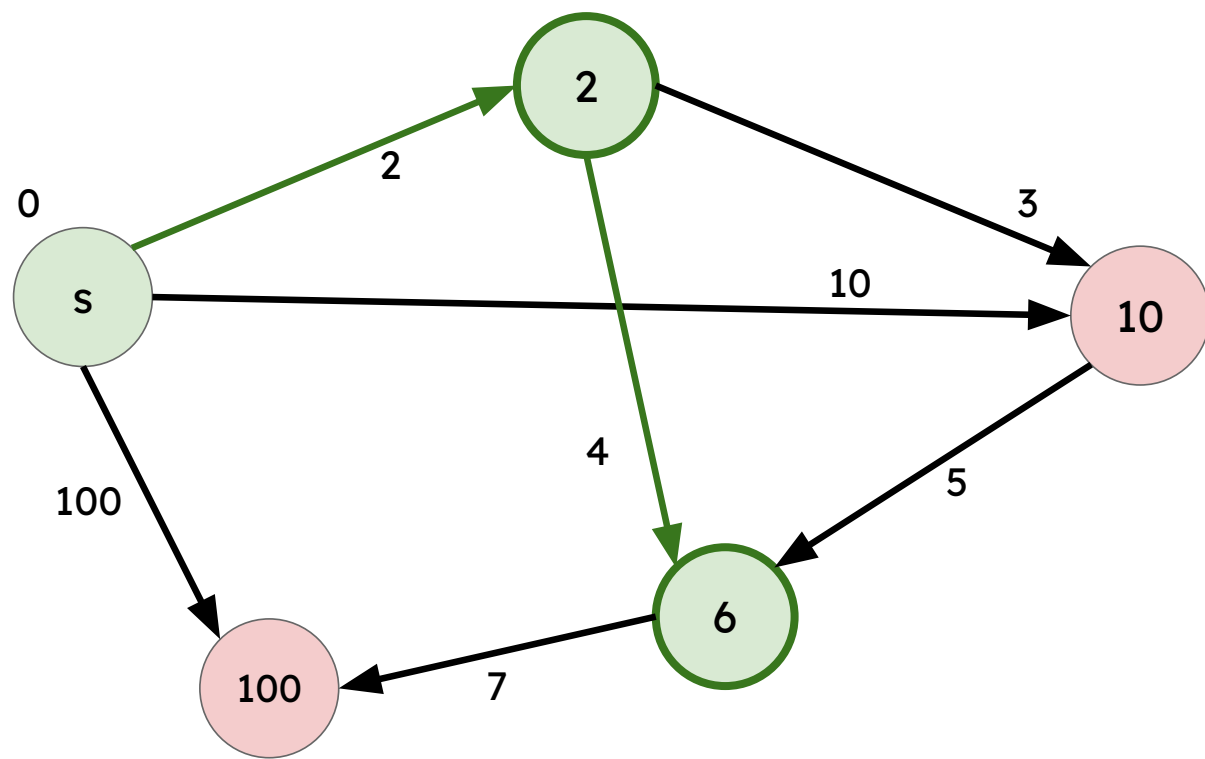
Case	Complexity
Directed, unweighted	$O(m + n)$
Directed, potentially negative weights	$O(mn)$

SSSP on a DAG

Speeding up Bellman Ford

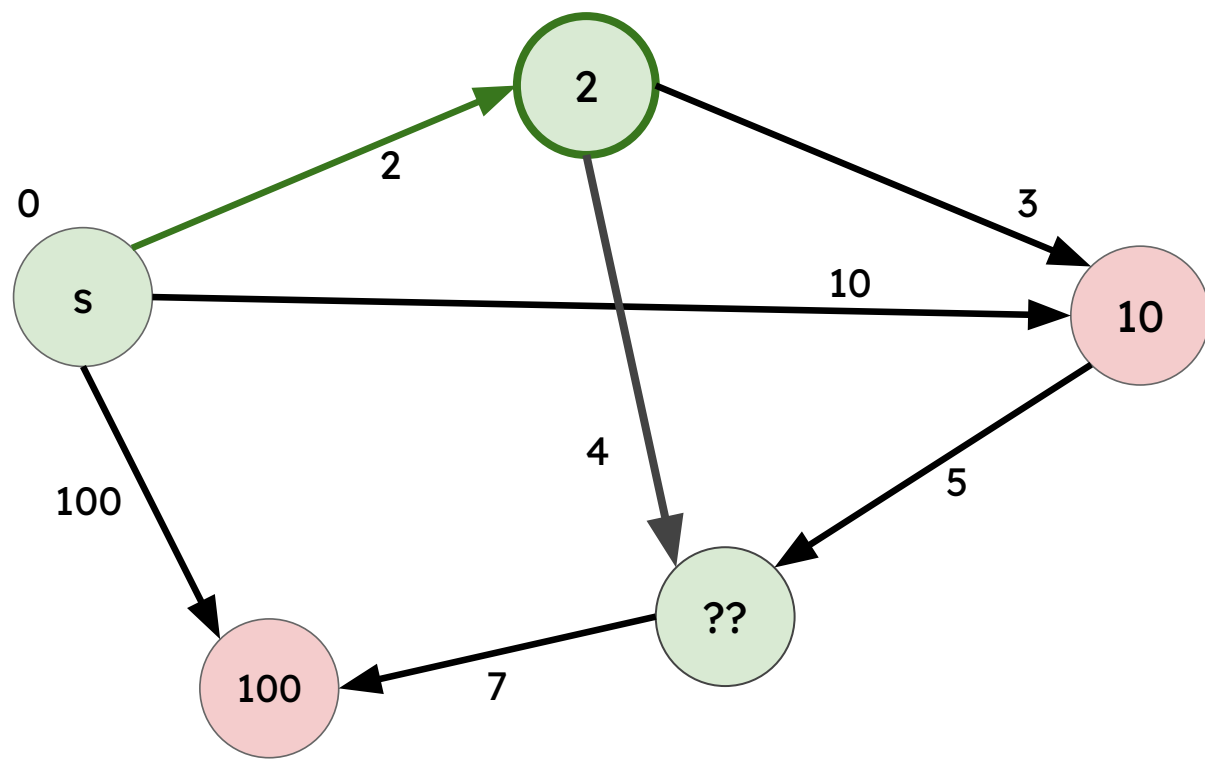
Let's be a lil' smarter
about the ordering.

Recall that after 1 round of relaxations, there was actually a chance for a node that was 2 edges away to have the correct distance computed.



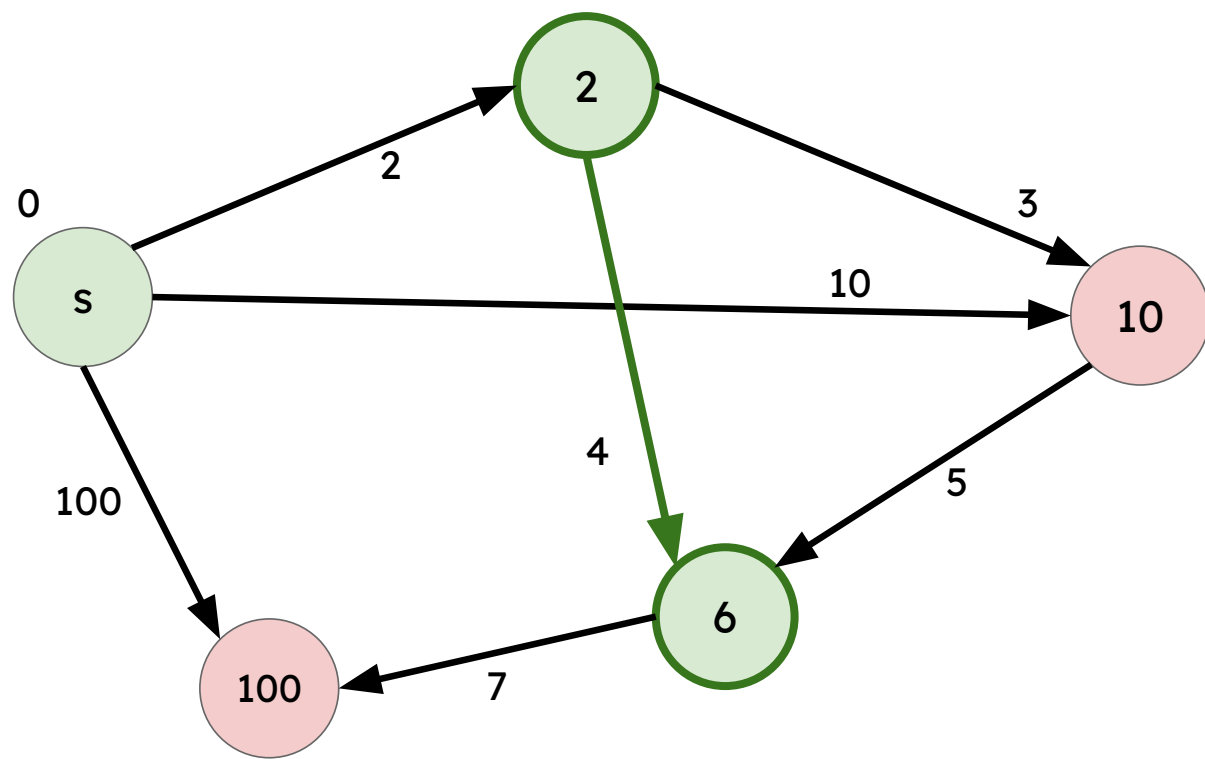
Even though this wasn't a hard guarantee, it was technically possible.

This is because in this case we actually relaxed its edges in the “correct order”.



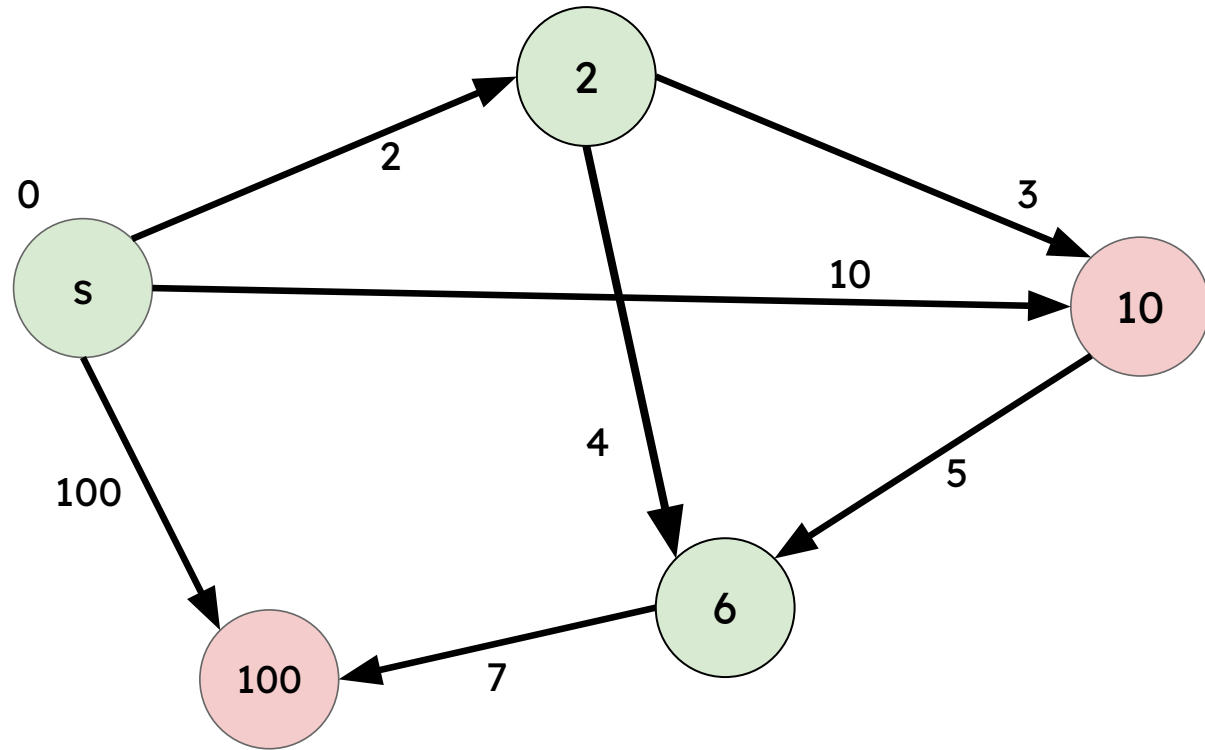
If we relaxed the edge from node s **first**...

This is because in this case we actually relaxed its edges in the “correct order”.



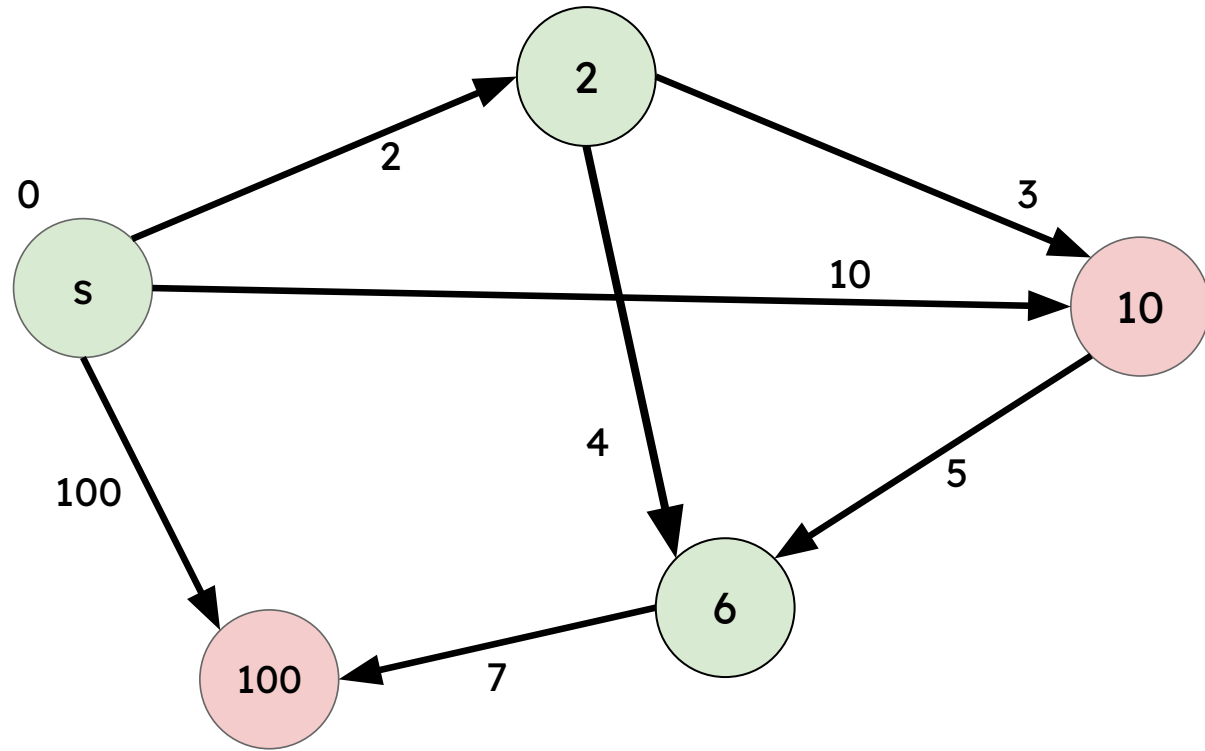
If we relaxed the edge from node **s** **first**... since it has the correct shortest distance already, any neighbour of that node that uses it for a shortest path can also get its correct distance.

So is there a way for us to abuse this to only run 1 round of edge relaxations?



We essentially need to find a good way to “order” the edges so that if we relaxed them in this sequence, we get all the correct distances in a single round.

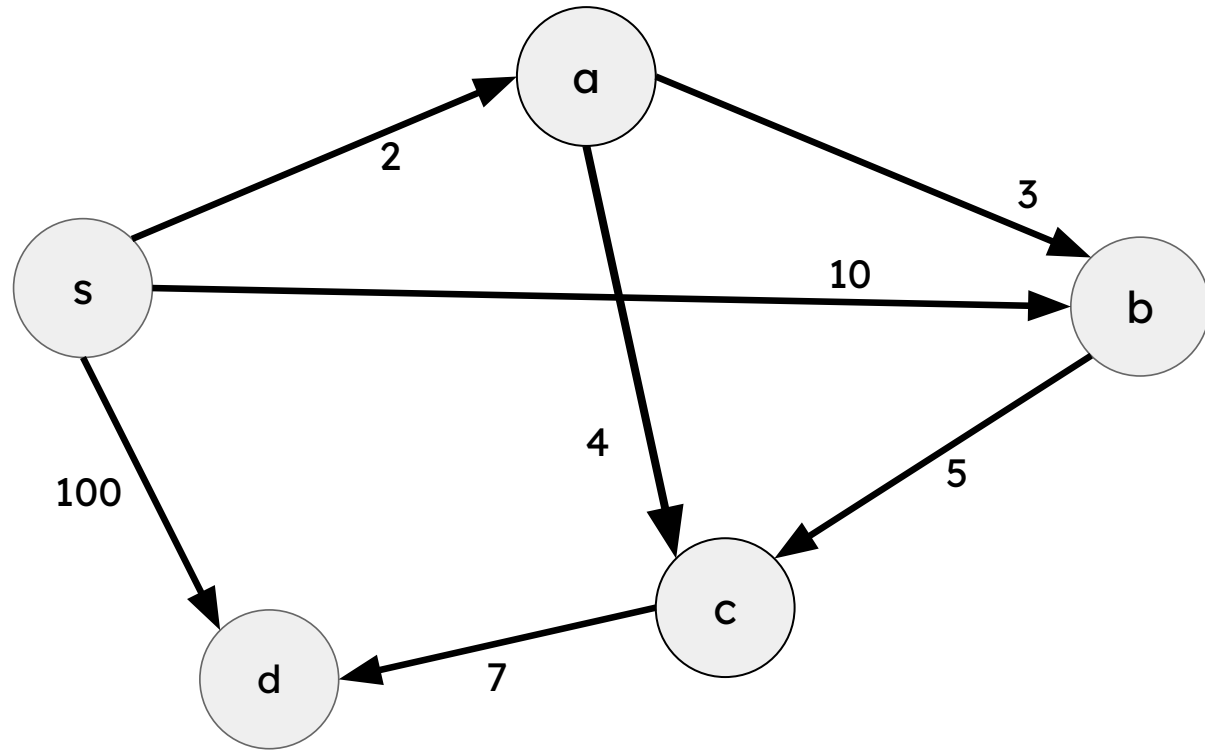
So is there a way for us to abuse this to only run 1 round of edge relaxations?



It turns out that for a DAG, if we toposort the nodes, and relax them in toposort order, we would only need to do this in 1 pass!

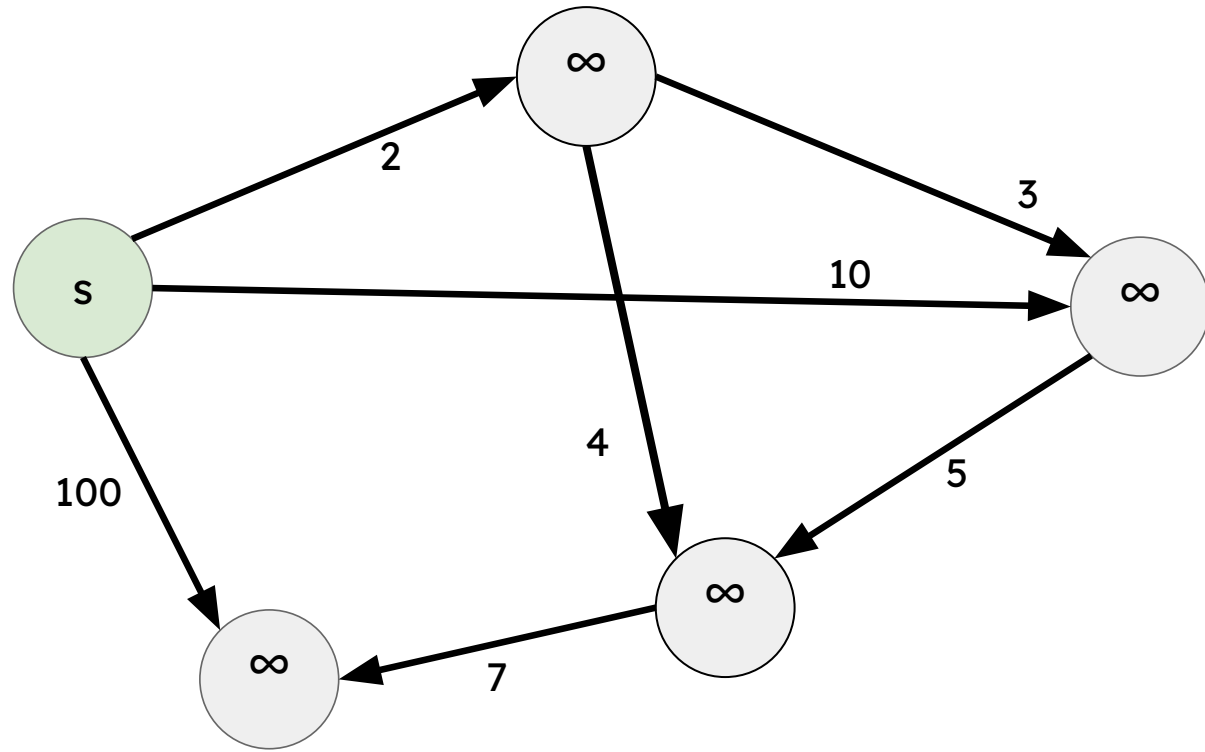
Let's say that we ran
toposort first, we'd
probably get a toplist
like so:

[s, a, b, c, d]



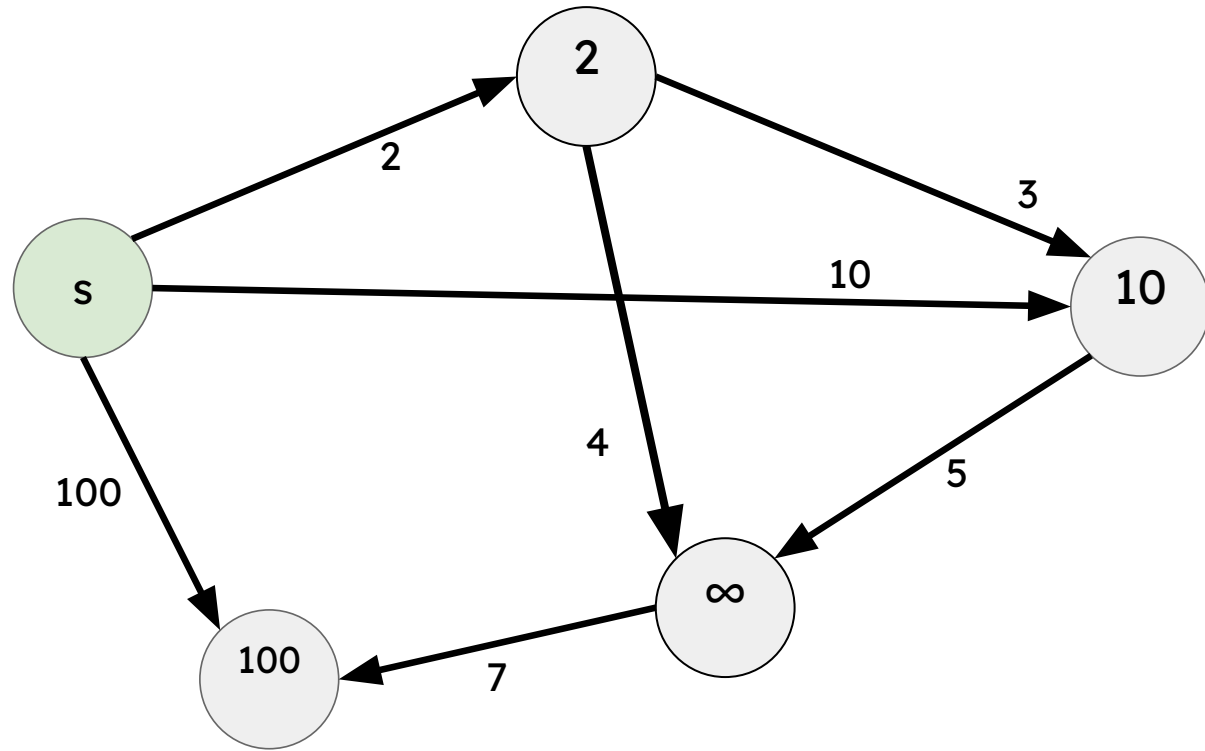
Then we relax outgoing edges from node s first, then all outgoing edges from node a, then all outgoing edges from node b, then all outgoing edge from node c, then all outgoing edges from node d.

So starting from s ,
relax out its outgoing
edges.



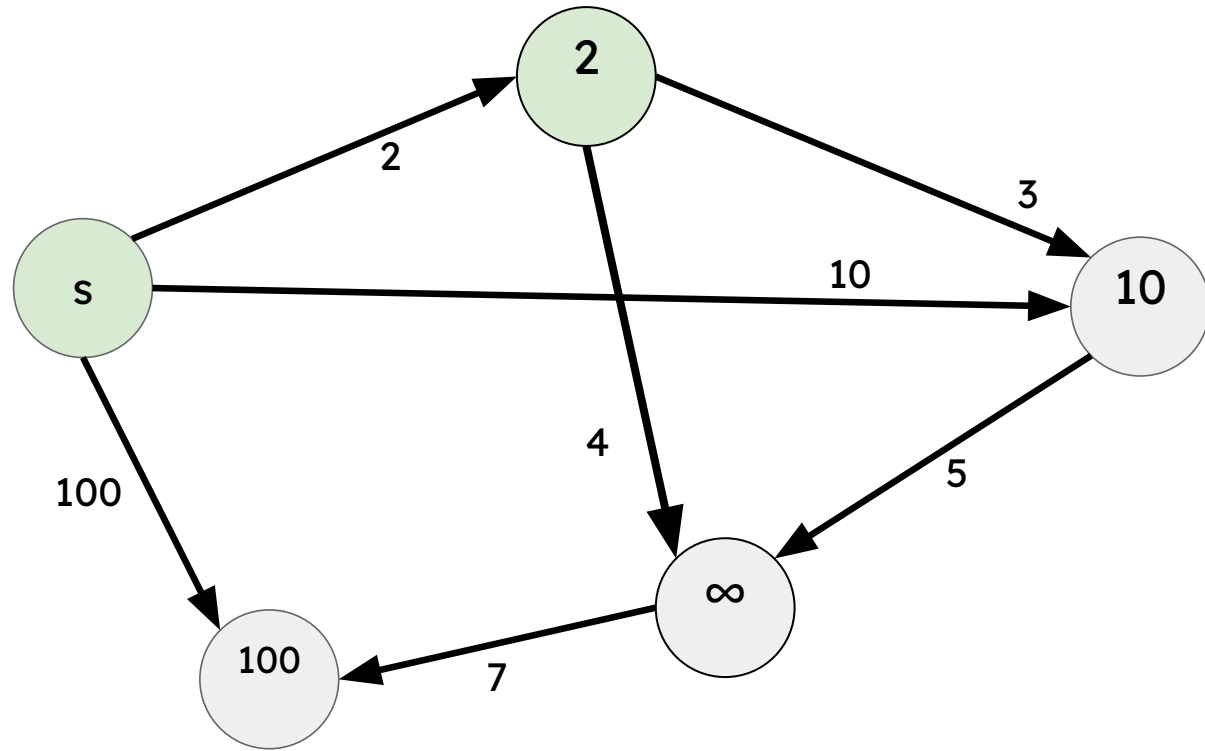
It has 3 neighbours, which get updated to 2, 10, and 100
respectively.

So starting from s ,
relax out its outgoing
edges.



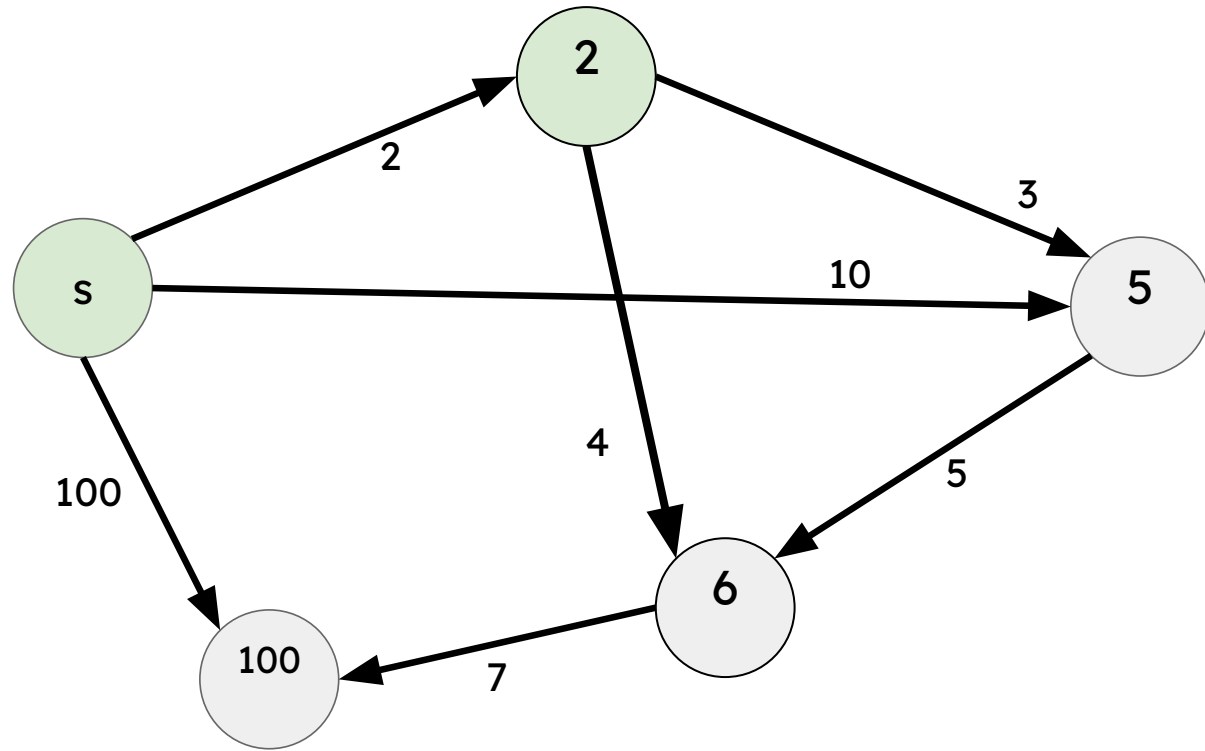
It has 3 neighbours, which get updated to 2, 10, and 100 respectively.

Next, relax from node
a.



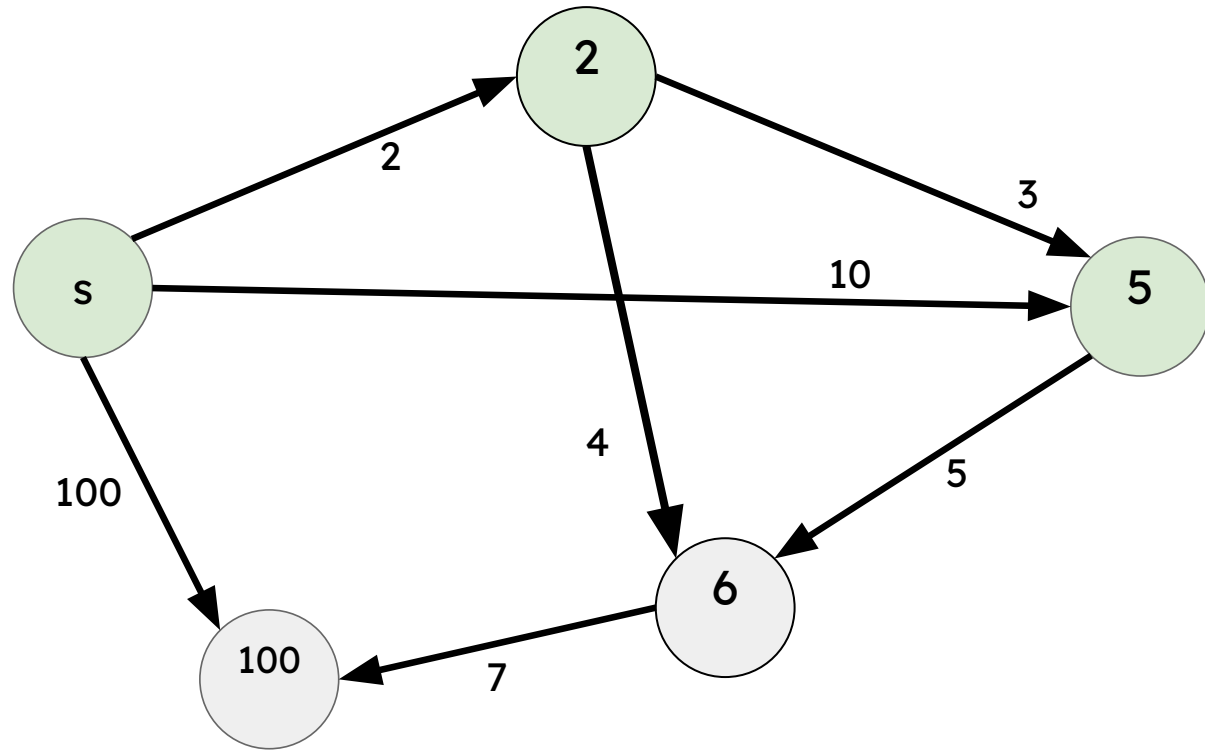
So it has 2 neighbours, which get updated to distances 5 and 6 respectively.

Next, relax from node
a.



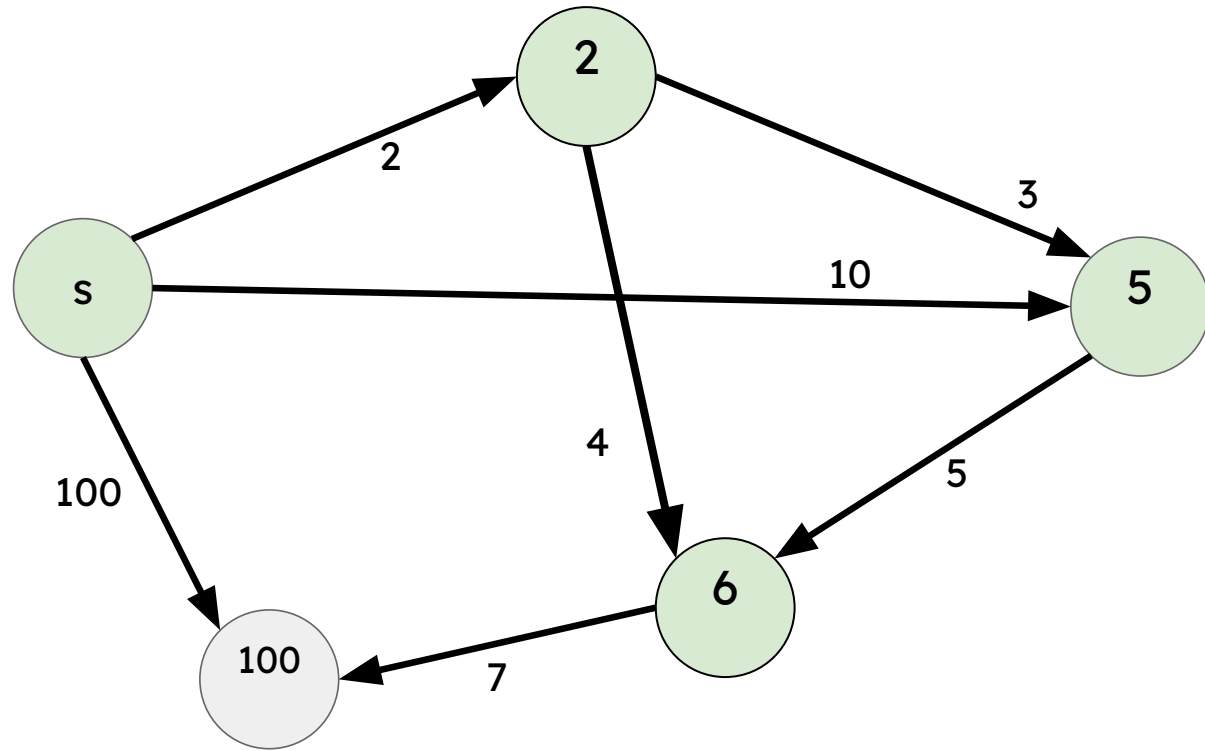
So it has 2 neighbours, which get updated to distances 5 and 6 respectively.

Next, relax from node
c.



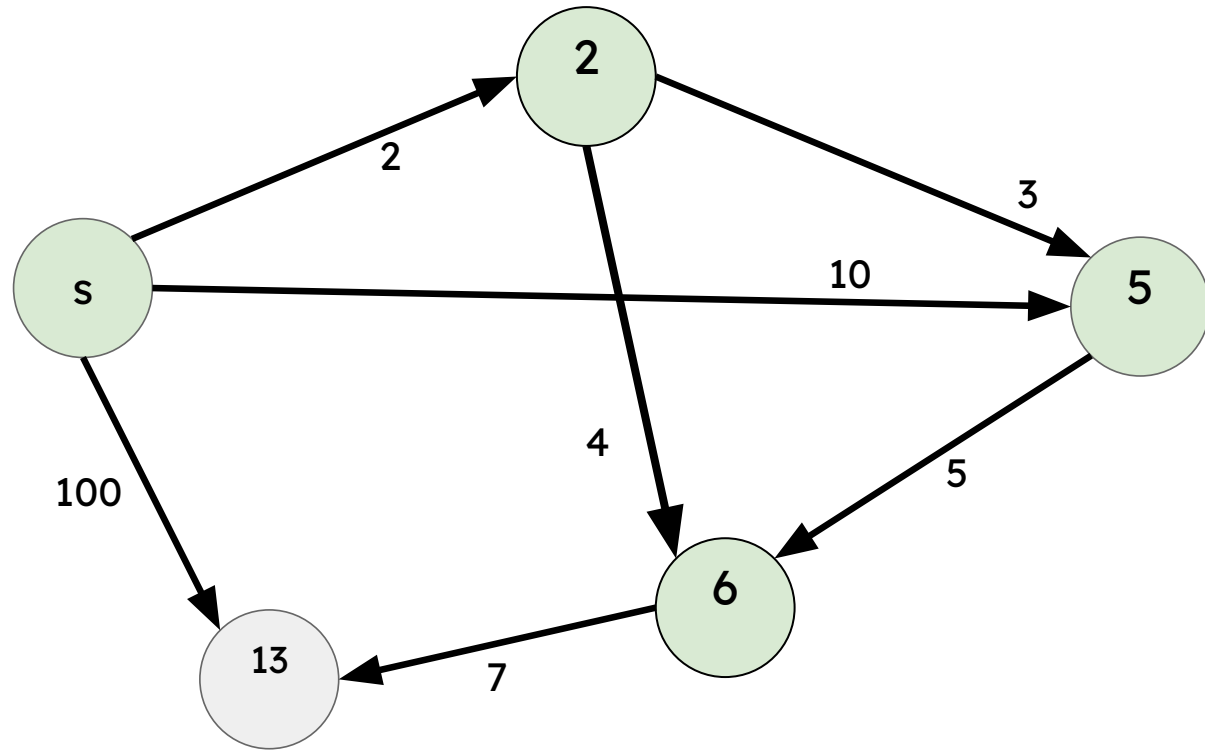
$5 + 5 = 10 > 6$, so no changes needed.

Next, relax from node
d.



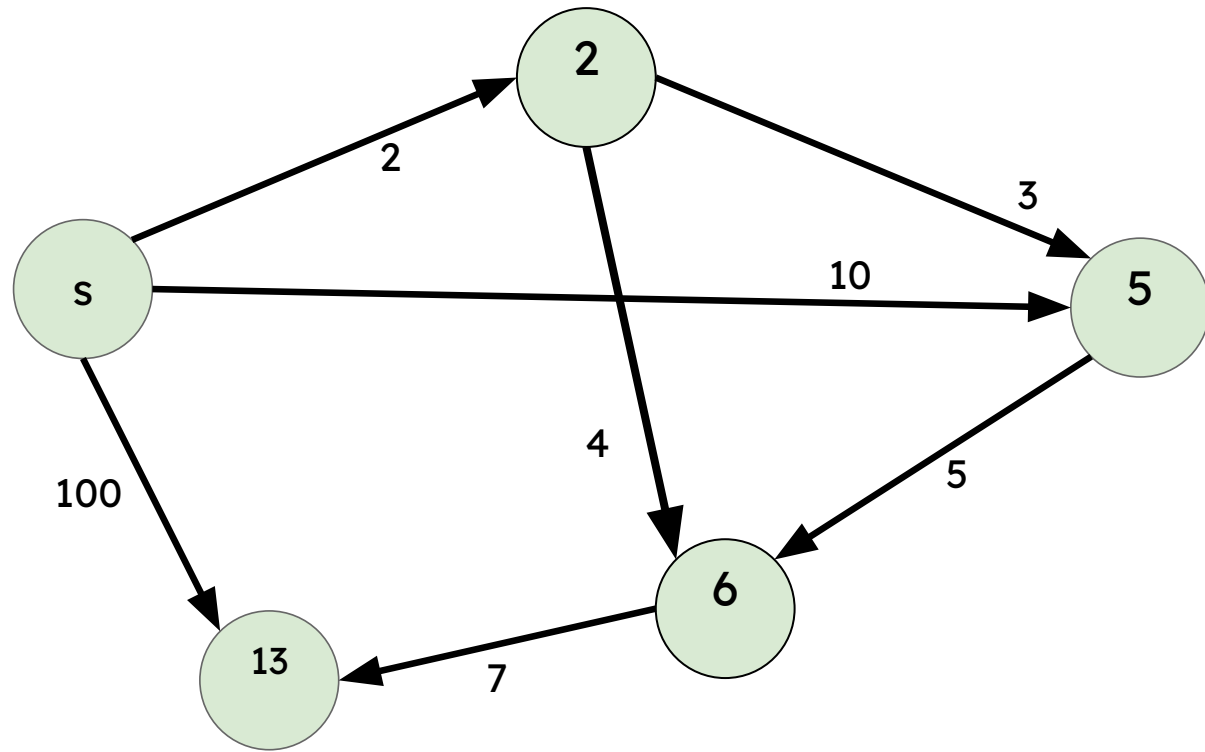
$6 + 7 = 13 < 100$, so we update its neighbours distance from 100 to 13.

Next, relax from node
d.



$6 + 7 = 13 < 100$, so we update its neighbours distance from 100 to 13.

We can skip node e, it has no outgoing edges.



And all shortest path distances have been computed.

SSSP on a DAG

sssp_on_dag(G, s):

 topolist = toposort(G, s)

 for each node v in topolist (from start to end):

 Relax all outgoing edges for node v

SSSP on a DAG

sssp_on_dag(G, s):

$topolist = \text{toposort}(G, s)$

 for each node v in $topolist$ (from start to end):

 Relax all outgoing edges for node v

What's the runtime?

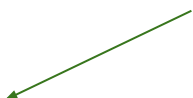
1. $O(n)$
2. $O(n + m)$
3. $O(m)$
4. $O(nm)$

SSSP on a DAG

sssp_on_dag(G, s):

topolist = toposort(G, s)

$O(m + n)$



for each node v in topolist (from start to end):

Relax all outgoing edges for node v

$O(m)$



Summary so far:

Case	Complexity
Directed, unweighted	$O(m + n)$
Directed, potentially negative weights	$O(mn)$
Directed acyclic, potentially negative weights	$O(m + n)$

SSSP on non-negative weights

Speeding up SSSP even more

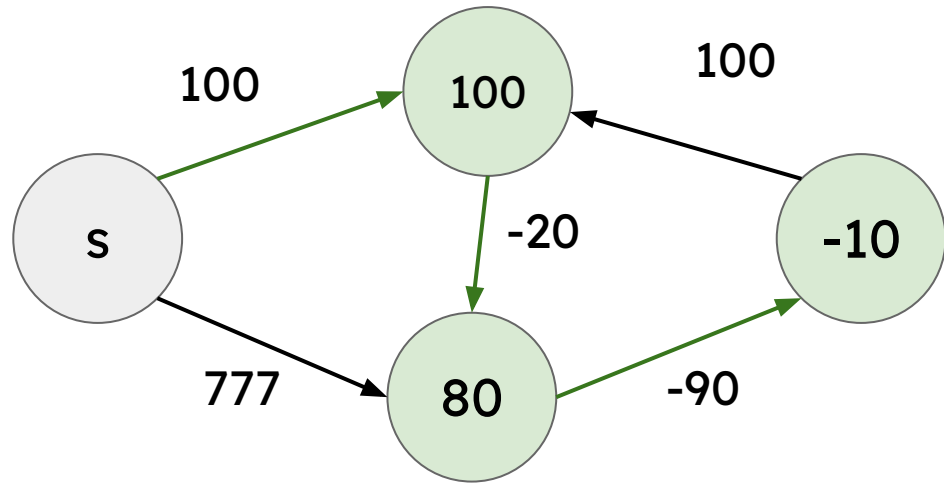
Did you know the original
Dijkstra paper wasn't as
efficient as the modern
version being presented?

Problem statement:

Input: Given a **directed**, **non-negatively weighted** graph $G = (V, E)$ in the form of an adjacency list, with **n** nodes and **m** edges, and a source node **s**.

Output: An array **dist[]** such that for all nodes **v**, **dist[v]** is the shortest distance from node **s** to node **v**.

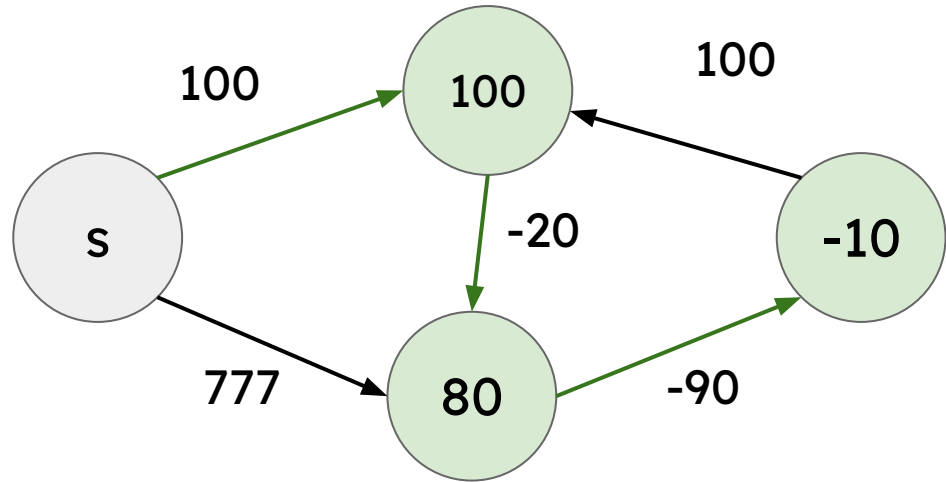
What Bellman-Ford handles:



Even if edges are negative, our distance estimations work fine.

What Bellman-Ford handles:

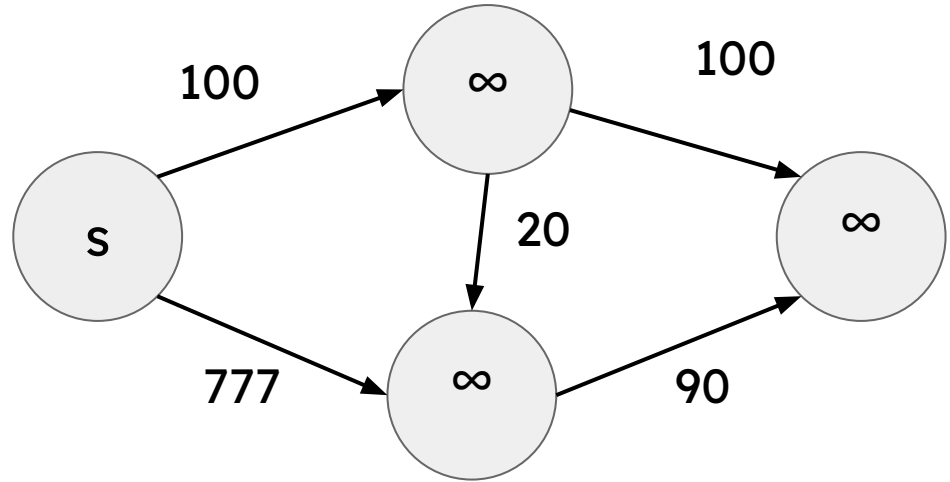
We're using the fact that:



Every round of edge relaxations helps us correctly find the shortest path of even more nodes, and eventually we will find it for all of them.

A different approach:

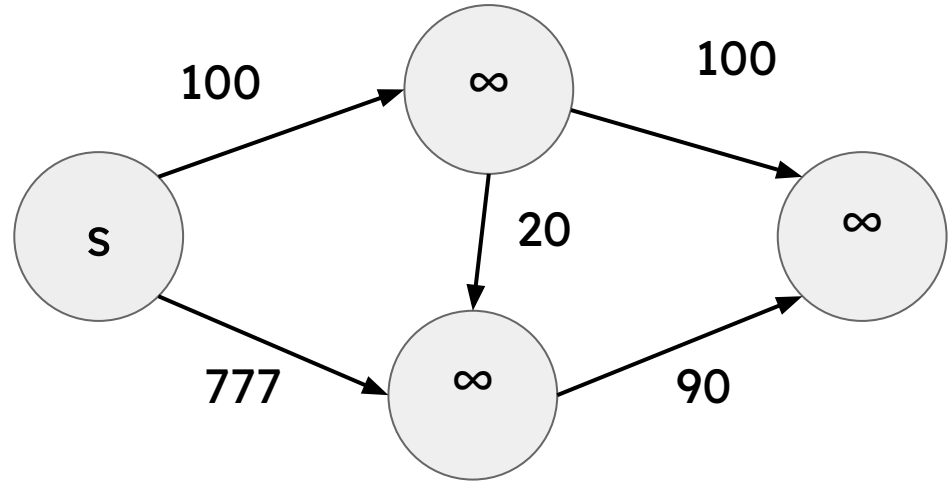
Since our edges have a non-negative weight, let's try a completely different approach here:



What if we visited nodes based on their distance estimates?

A different approach:

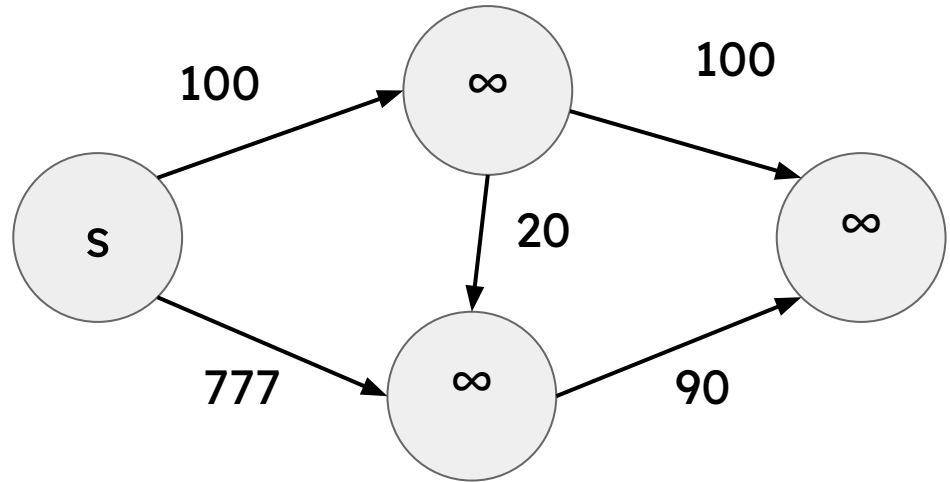
Since our edges have a non-negative weight, let's try a completely different approach here:



I.e. Keep a priority queue of the nodes based on their distance estimates so we know which node has the smallest distance estimate so far.

A different approach:

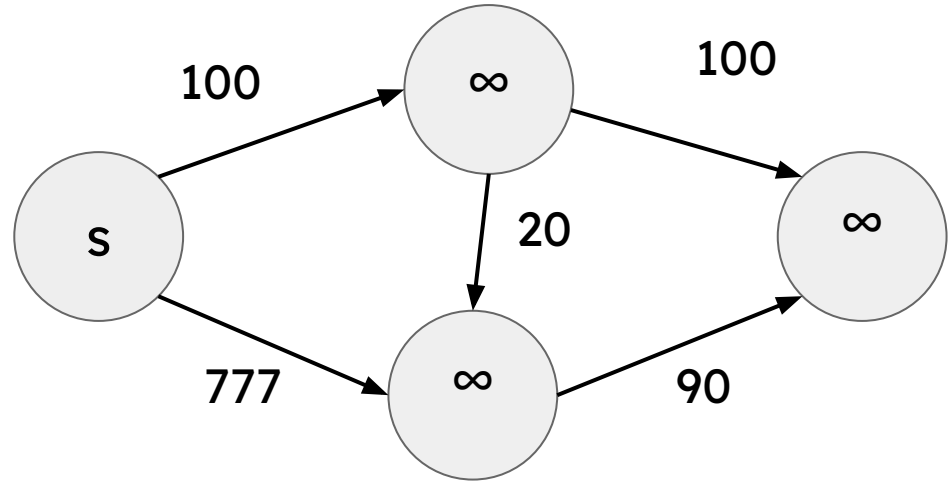
Since our edges have a non-negative weight, let's try a completely different approach here:



Then, when we extract a node out of the priority queue, we'll look at its neighbours and see if we can relax any of the edges, thereby reducing distance estimates.

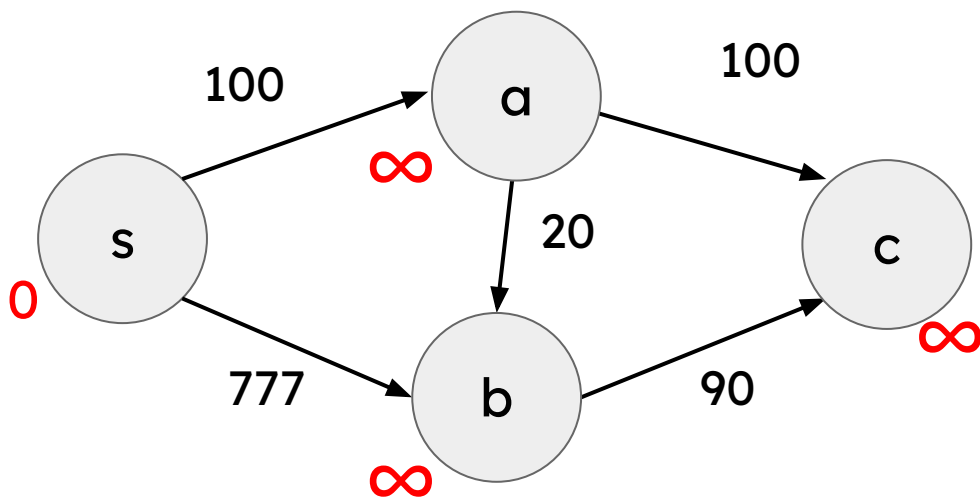
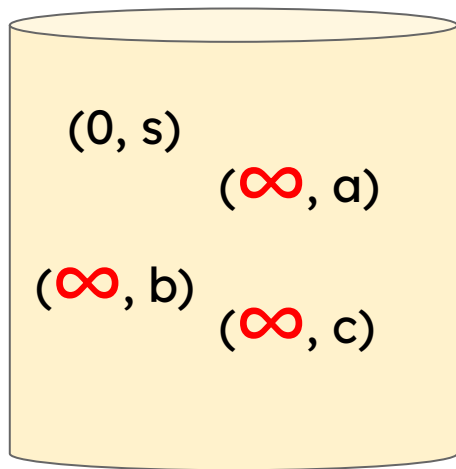
A different approach:

Since our edges have a non-negative weight, let's try a completely different approach here:



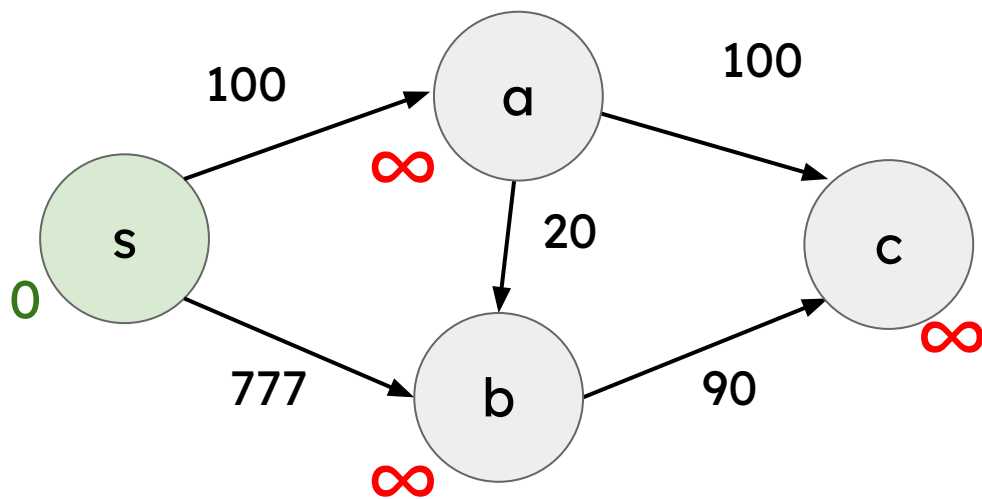
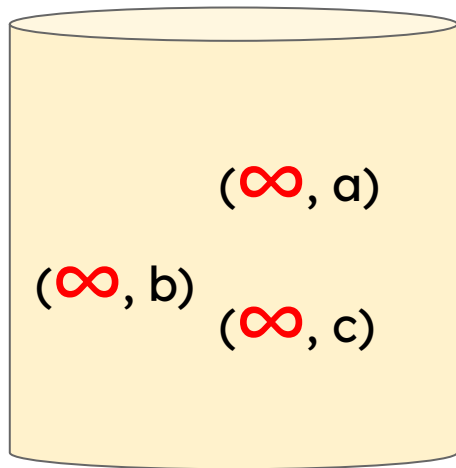
Then just keep doing this until the priority queue is empty.

Example run:



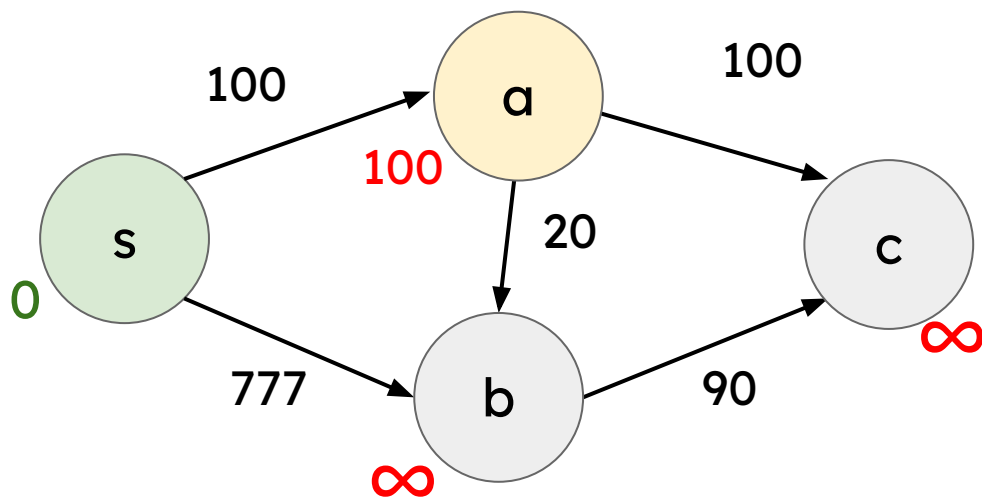
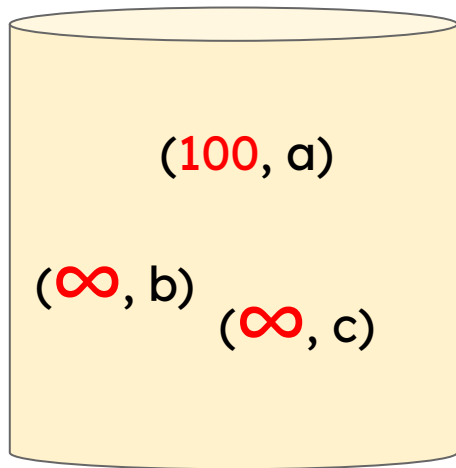
For example, we'll keep a priority queue for all the nodes where the priorities are the distance estimates. Initially $\text{dist_est}[s] = 0$, and otherwise it's ∞ .

Example run:



When we run extract min, the first node we'll get is s itself. So just remove it from the queue.

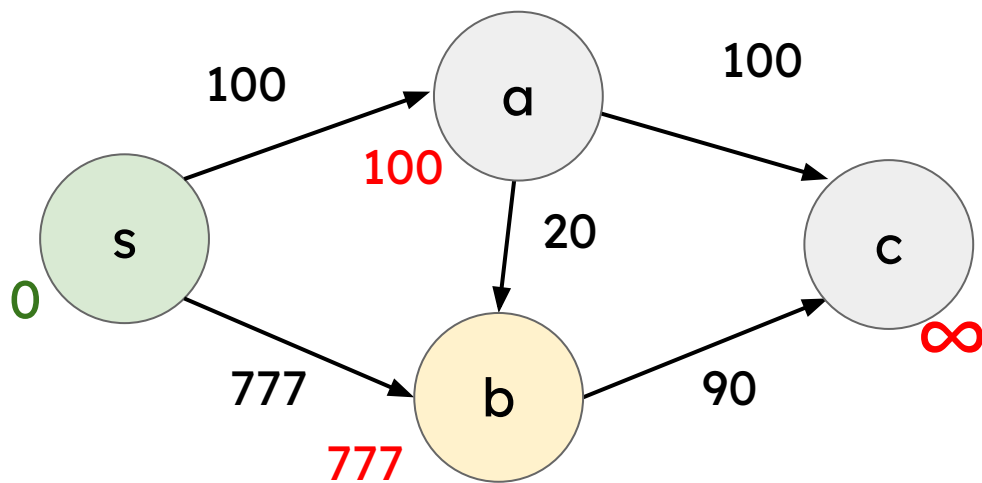
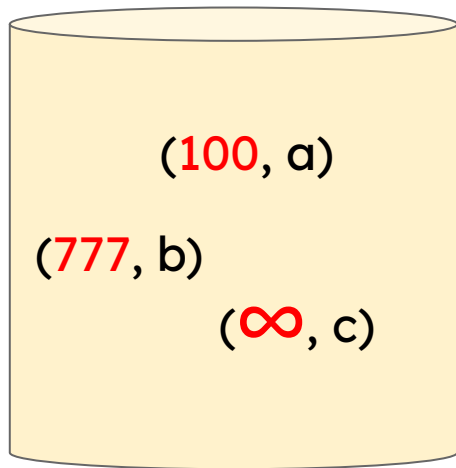
Example run:



Now, as before, run edge relaxation on all of its outgoing edges.

For neighbour a : $0 + 100 < \infty$, so we'll decrease its estimate to 100.

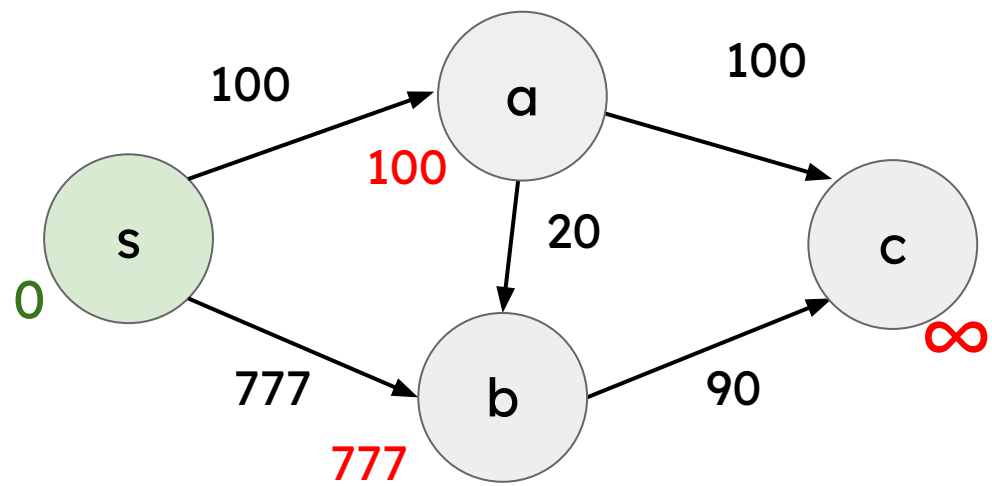
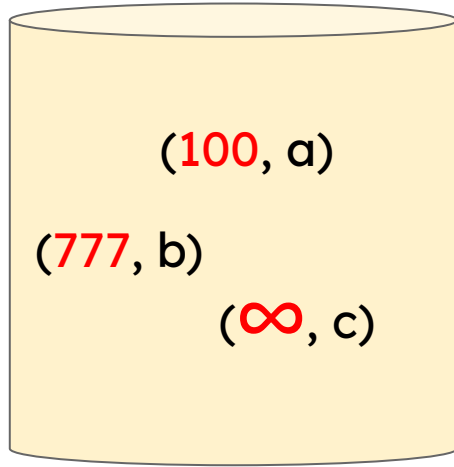
Example run:



Now, as before, run edge relaxation on all of its outgoing edges.

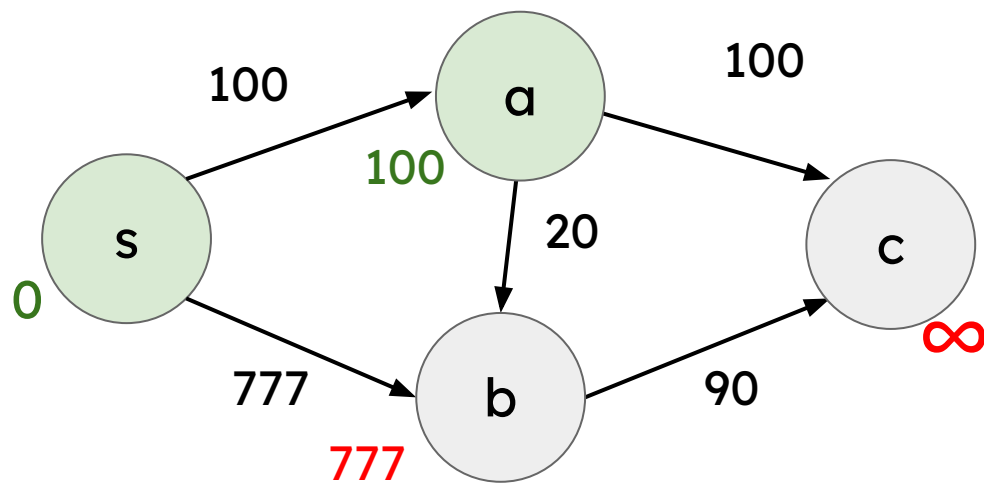
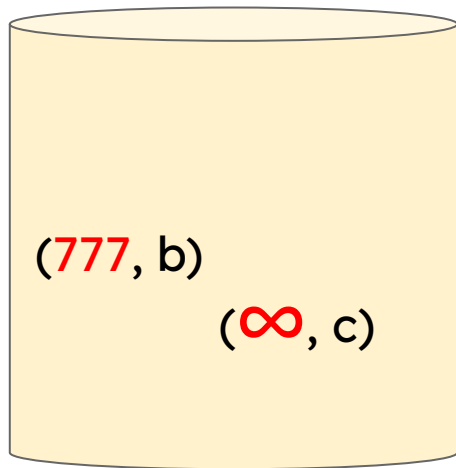
For neighbour b : $0 + 777 < \infty$, so we'll decrease its estimate to 777.

Example run:



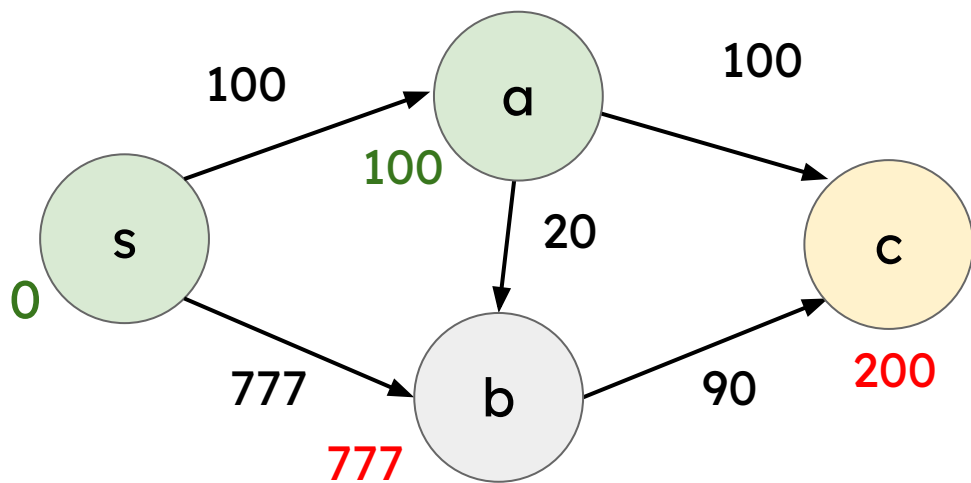
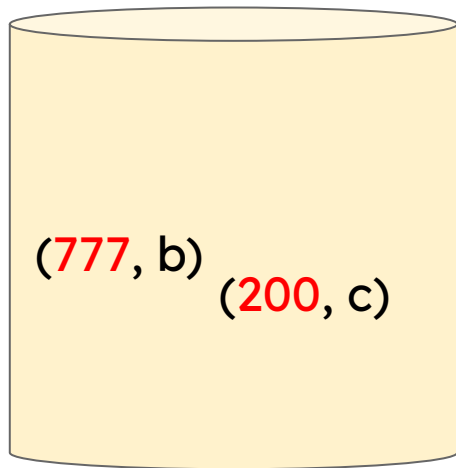
Now that we're done with that, we pop the next minimum element out of the priority queue. This time it happens to be node a.

Example run:



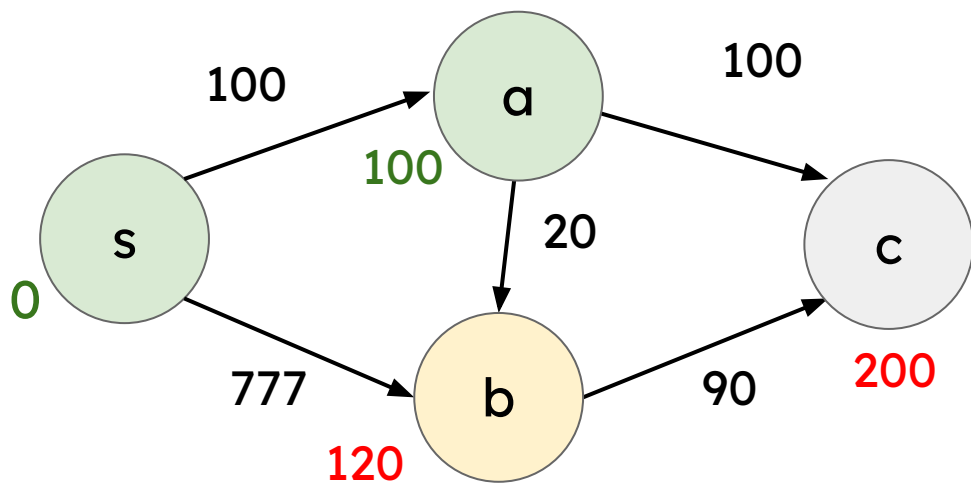
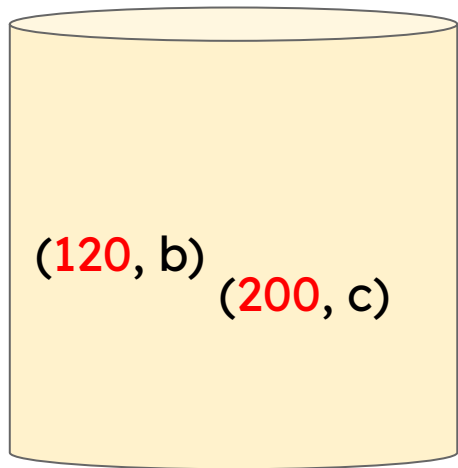
Again, we're going to relax the outgoing edges.

Example run:



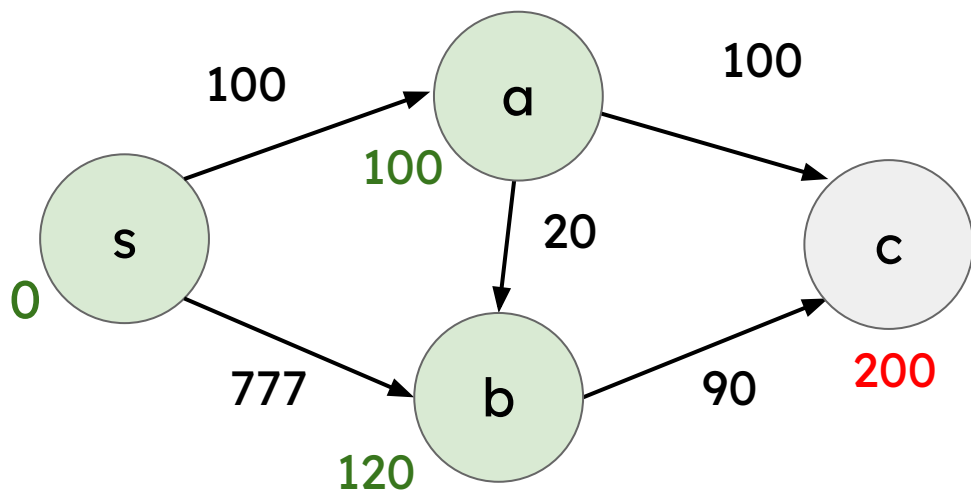
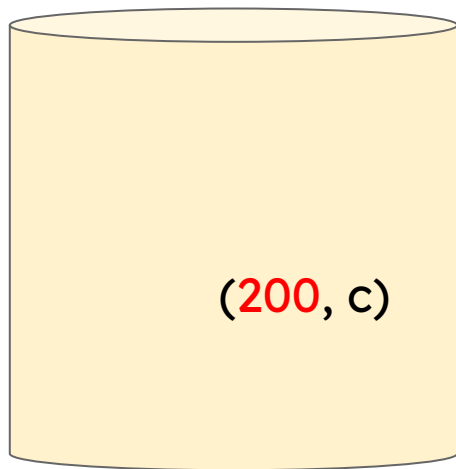
Considering neighbour c: $100 + 100 < \infty$, so we update its distance estimate to 200.

Example run:



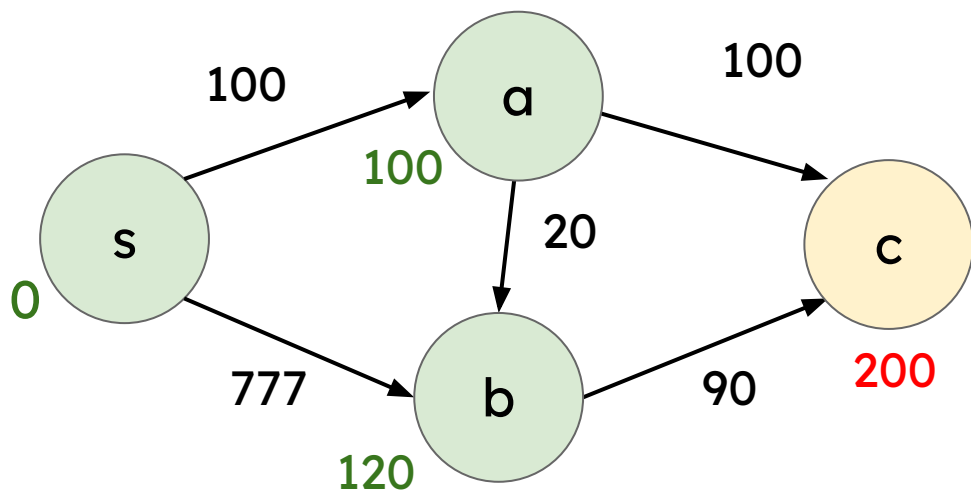
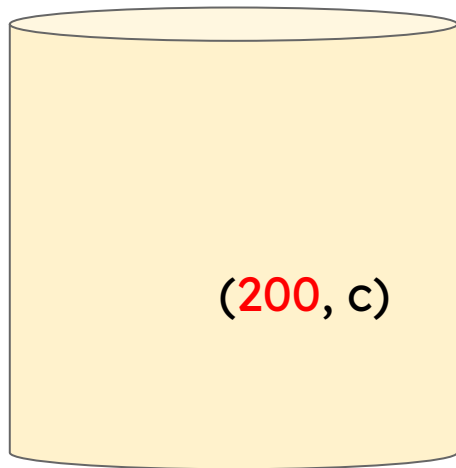
Also, for neighbour b, $100 + 20 < 777$, so we'll update its distance estimate to 120.

Example run:



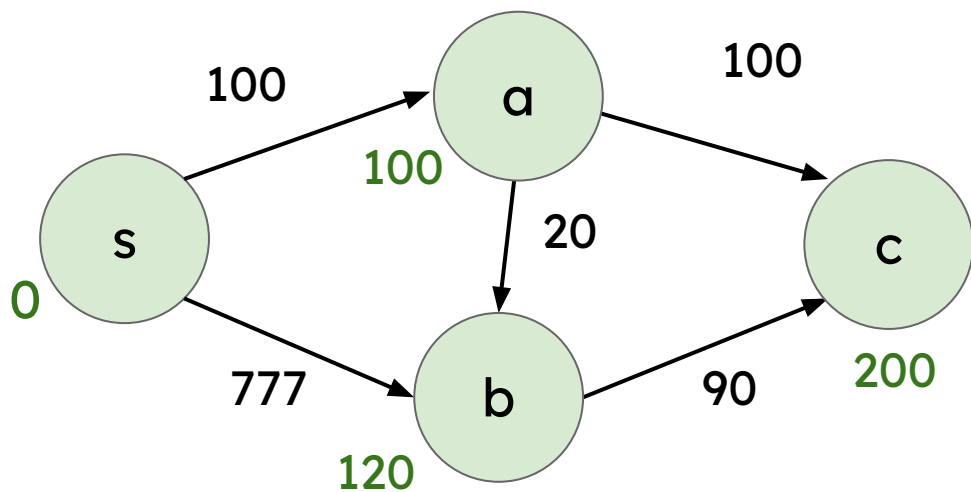
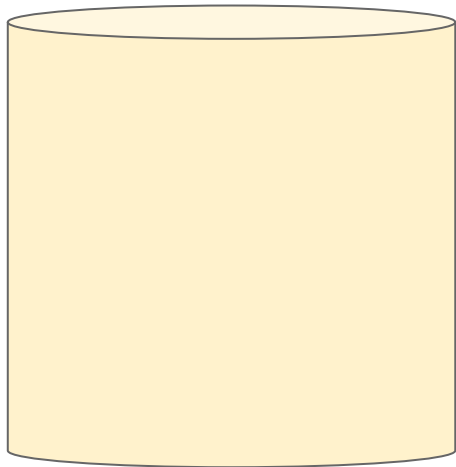
The priority queue isn't empty yet, so let's keep going. The next element to remove is node b, with a priority of 120.

Example run:

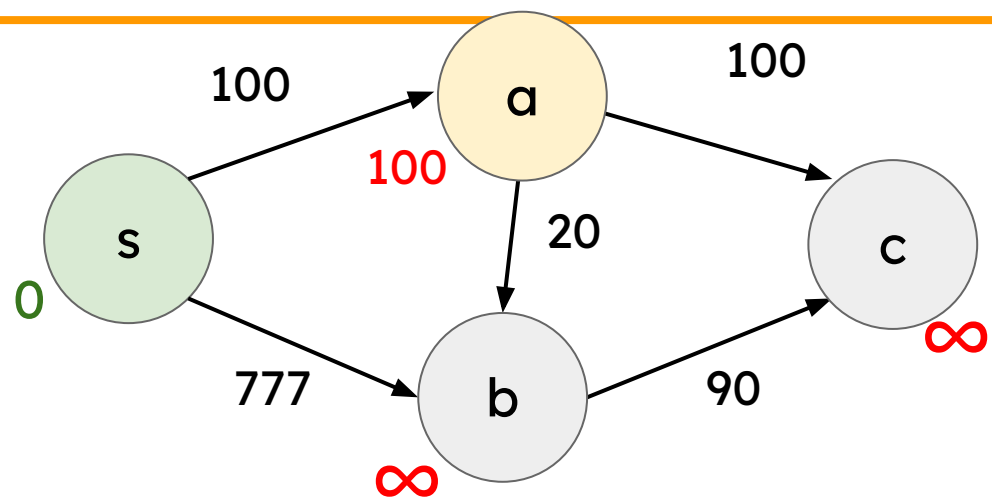
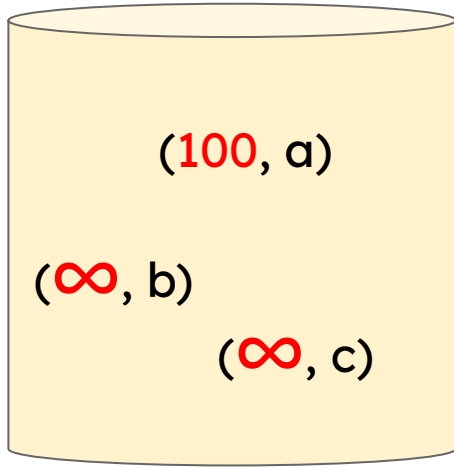


The only neighbour to consider here is node c. Since $120 + 90 \geq 200$, we won't decrease its distance estimate, and just leave it alone.

Example run:



Lastly, remove node c from the priority queue, and we are done!



When we were decreasing the distance estimates, we had to update these in the priority queue as well, which operation is most suitable for us here?

Decrease key!

Dijkstra: (Classic Version)

dijkstra(G, s):

dist_est = [an array of all ∞]; set dist_est[s] = 0

heap = heapify(copy(dist_est))

while heap is not empty:

 curr_node = heap.extract_min()

 for all neighbours (n, w) in G.get_neighbours(curr_node)

 if dist_est[curr_node] + w < dist_est[n]:

 heap.decrease_key(n, dist_est[curr_node] + w)

 dist_est[n] = dist_est[curr_node] + w

Dijkstra: (Classic Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`heap` = `heapify(copy(dist_est))`

while `heap` is not empty:

`curr_node` = `heap.extract_min()`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

if `dist_est[curr_node] + w < dist_est[n]`:

`heap.decrease_key(n, dist_est[curr_node] + w)`

`dist_est[n] = dist_est[curr_node] + w`

We do extract min at most n times, since the heap starts with n nodes.

Dijkstra: (Classic Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est[s] = 0`

`heap` = `heapify(copy(dist_est))`

while `heap` is not empty:

`curr_node` = `heap.extract_min()`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

if `dist_est[curr_node] + w < dist_est[n]`:

`heap.decrease_key(n, dist_est[curr_node] + w)`

`dist_est[n] = dist_est[curr_node] + w`

We do `decrease_key` at most m times, since we relax for each outgoing edge at most once.

Dijkstra: (Classic Version)

dijkstra(G, s):

Since each `extract_min` and `decrease_key` run in $O(\log n)$ time, the overall complexity is:

$$O(n \log n + m \log n)$$

Dijkstra: (Classic Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞];

`heap` = heapify(copy(`dist_est`))

while `heap` is not empty:

`curr_node` = `heap`.extract_min()

for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

if `dist_est`[`curr_node`] + `w` < `dist_est`[`n`]:

`heap`.decrease_key(`curr_node`, `dist_est`[`curr_node`] + `w`)

If you recall the headache we went through to implement decrease key with IDs, you might realise that to implement Dijkstra, you now need to also implement a heap.

Dijkstra: (Classic Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞];

`heap` = heapify(copy(`dist_est`))

while `heap` is not empty:

`curr_node` = `heap.extract_min()`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

if `dist_est[curr_node] + w < dist_est[n]`:

`heap.decrease_key(curr_node, dist_est[curr_node] + w)`

Most real life APIs (like from Java and C/C++) don't actually provide this.

Dijkstra: (Classic Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞];

`heap` = heapify(copy(`dist_est`))

while `heap` is not empty:

`curr_node` = `heap`.extract_min()

for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

if `dist_est`[`curr_node`] + `w` < `dist_est`[`n`]:

`heap`.decrease_key(`curr_node`, `dist_est`[`curr_node`] + `w`)

However, they do give priority queues that can insert and extract-min efficiently. But not delete nor decrease_key.

dijkstra(G, s):

Start with an **empty priority queue**. Set all distance estimates except s to ∞ . Set our estimate of s to 0.

Insert node s with priority 0 into the **pq**.

While our **pq** is not empty:

 Extract min from **pq** to get the next **current_node** and its **priority**.

 If has **current_node** been visited before, **skip this iteration**.
 Otherwise, mark it as **visited**.

 Update **distance estimate** to the extracted **priority**.

 For all its neighbours n , if we have edge weight w to it:

 Add node n with priority $\text{dist_est}[\text{current_node}] + w$ to **pq**

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

dist_est = [an array of all ∞]; set dist_est[s] = 0

visited = [an array of all false]; heap = [a heap containing (0, s)]

while heap is not empty:

(distance, curr_node) = heap.extract_min();

if visited[curr_node]: continue else visited[curr_node] = true

dist_est[curr_node] = distance

for all neighbours (n, w) in G.get_neighbours(curr_node)

heap.insert(dist_est[curr_node] + w, n)

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

 (`distance`, `curr_node`) = `heap`.extract_min();

 if visited[`curr_node`]: continue else visited[`curr_node`] = true

`dist_est`[`curr_node`] = `distance`

 for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

`heap`.insert(`dist_est`[`curr_node`] + `w`, `n`)

We will only visit each node once.

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

 (`distance`, `curr_node`) = `heap`.extract_min();

 if visited[`curr_node`]: continue else visited[`curr_node`] = true

`dist_est`[`curr_node`] = `distance`

 for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

`heap`.insert(`dist_est`[`curr_node`] + `w`, `n`)

But notice here that there may be multiple copies of the same node in the heap.

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

 (`distance`, `curr_node`) = `heap`.extract_min();

 if visited[`curr_node`]: continue else visited[`curr_node`] = true

`dist_est`[`curr_node`] = `distance`

 for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

`heap`.insert(`dist_est`[`curr_node`] + `w`, `n`)

But we will only update its distance the first time we ever see it.

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

(`distance`, `curr_node`) = `heap.extract_min()`;

if `visited[curr_node]`: `continue` else `visited[curr_node]` = `true`

`dist_est[curr_node]` = `distance`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

`heap.insert(dist_est[curr_node] + w, n)`

And we won't update the distance again for any other time we see it.

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

 (`distance`, `curr_node`) = `heap`.extract_min();

 if visited[`curr_node`]: continue else visited[`curr_node`] = `true`

`dist_est`[`curr_node`] = `distance`

 for all neighbours (`n`, `w`) in G.get_neighbours(`curr_node`)

`heap`.insert(`dist_est`[`curr_node`] + `w`, `n`)

Notice in this version, the heap holds at most m items

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est`

`visited` = [an array of all `false`]; `heap` =

while `heap` is not empty:

 (`distance`, `curr_node`) = `heap.extract_min()`;

 if `visited[curr_node]`: `continue` else `visited[curr_node]` = `true`

`dist_est[curr_node]` = `distance`

 for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

`heap.insert(dist_est[curr_node] + w, n)`

Each node can only be inserted as many time as it has incoming edges. So the heap size is at most $O(m)$

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all ∞]; set `dist_est[s] = 0`

`visited` = [an array of all `false`]; `heap` = `PriorityQueue()`

while `heap` is not empty:

`(distance, curr_node) = heap.extract_min();`

if `visited[curr_node]`: `continue` else `visited[curr_node] = true`

`dist_est[curr_node] = distance`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

`heap.insert(dist_est[curr_node] + w, n)`

This also means we do `extract_min` at most m times.

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

`dist_est` = [an array of all

`visited` = [an array of all t

while `heap` is not empty:

`(distance, curr_node) = heap.extract_min();`

if `visited[curr_node]`: `continue` else `visited[curr_node] = true`

`dist_est[curr_node] = distance`

for all neighbours (`n`, `w`) in `G.get_neighbours(curr_node)`

`heap.insert(dist_est[curr_node] + w, n)`

So the running time is $O(m \log m)$

Dijkstra: (Lazy Deletion Version)

dijkstra(G, s):

dist_est = [an array of all ∞]; set dist_est[s] = 0

visited = [an array of all false]; heap = [a heap containing (0, s)]

while heap is not empty:

(distance, curr_node) = heap.extract_min();

if visited[curr_node]: continue else visited[curr_node] = true

dist_est[curr_node] = distance

for all neighbours (n, w) in G.get_neighbours(curr_node)

heap.insert(dist_est[curr_node] + w, n)

Summary so far:

Case	Complexity
Directed, unweighted	$O(m + n)$
Directed, potentially negative weights	$O(mn)$
Directed acyclic, potentially negative weights	$O(m + n)$
Directed, nonnegative weights	$O(m \log m)$ or $O(m \log n)$

Case	Complexity
Directed, unweighted	$O(m + n)$
Directed, potentially negative weights	$O(mn)$
Directed acyclic, potentially negative weights	$O(m + n)$
Directed, nonnegative weights	$O(m \log m)$ or $O(m \log n)$

There's actually no real difference between $O(m \log m)$ and $O(m \log n)$, why?

Wednesday Plan:

Eldon will be back. We will solve a few problems as examples for midterms.